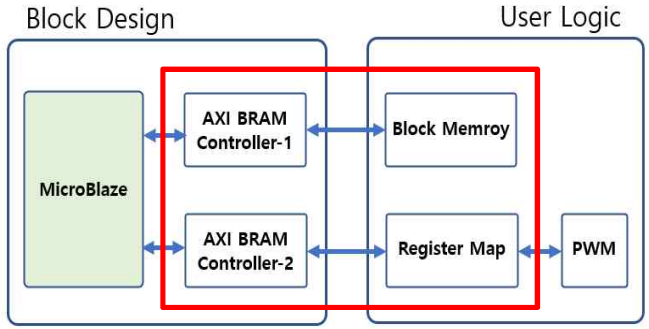
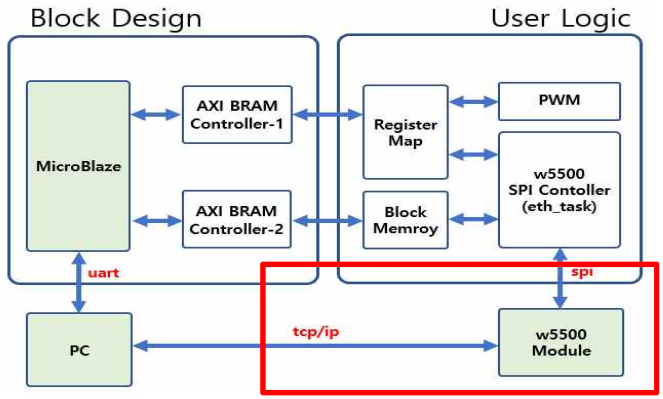
SoC 를 위한 *Peripheral* 설계

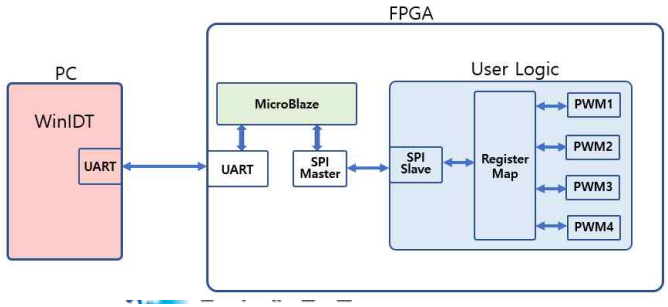
Reference : *MicroBlaze.v15 [IHIL]*

2025-06-24

Table of Contents







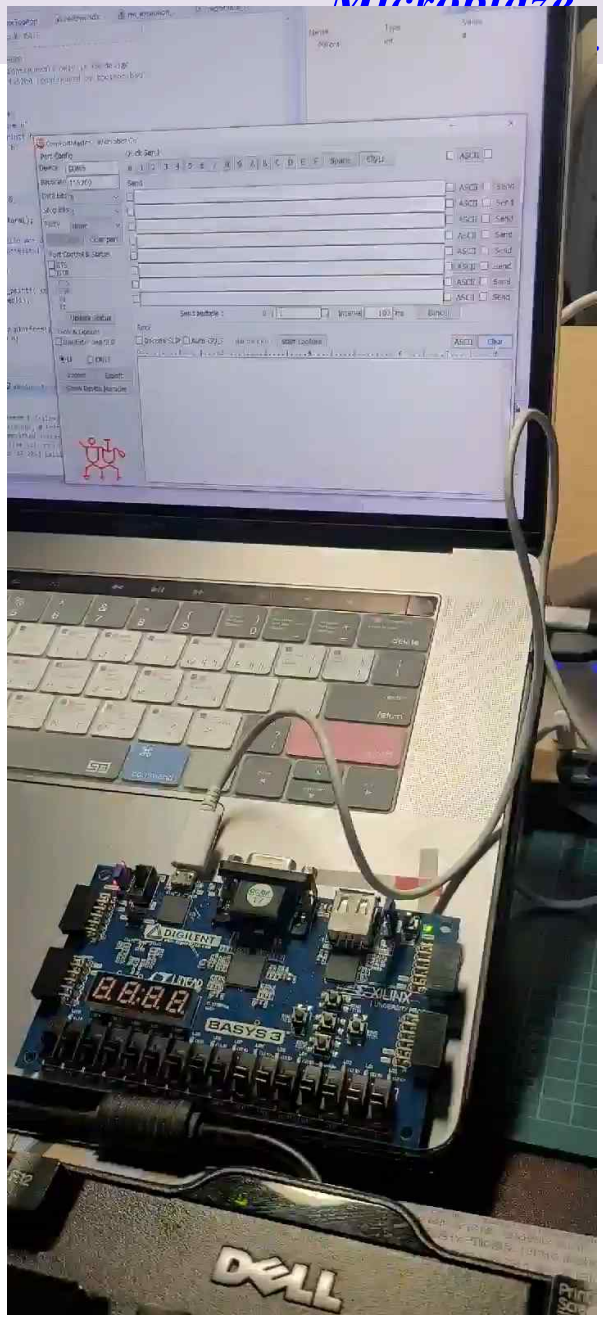
LED_Counter Implementation

- *MicroBlaze*를 이용하여 *Hello World* 출력하고, 간단한 사용자 로직*(User Logic)*으로 *LED*를 *On/Off* 하는 것을 구현
  - *MicroBlaze*는 *Processor Core*와 *Peripheral* 이 분리
  - **원하는 *Peripheral* 추가 및 개수도 필요한 만큼 추가하여 사용 가능**
  - *MicroBlaze Processor Core*와 *Peripheral* 중에 *UART* 를 추가하여 *UART* 포트를 통하여 *Hello World* 를 출력 기능 구현

- *FPGA*에서 *MicroBlaze*를 사용하는 목적
  - 사용자가 만든 *User Logic*을 제어하고, *Logic* 으로 구현할 수 없는 부분을 *MicroBlaze* 로 구현
  - *FPGA*에서 ***IP* 와 *User Logic* 을 생성해서 *MicroBlaze*사용하는 궁극적인 목적**은 ***HW*로 구현하는 부분**과 ***SW*로 구현하는 부분**을 나누어서 구현하고 **서로 간에 인터페이스를 구현**하여 외부에 추가적인 부품을 사용하지 않는 *SoC (System on Chip)*를 구현

LED_Counter Implementation

- *The Goal ➔ LED_Counter*
  - *Check The Result*
    - *LD0 ~ LD3* 이 **0.5**초 간격으로 *On/Off*
    - 기본적인 동작이 완료 ➔ [*LED_Counter Module*]에서 구현한 *LED on/off* 동작과 **SW**에서 구현한 **1**초마다 메시지를 표시



LED_Counter HW/SW Design and Implementation Flow

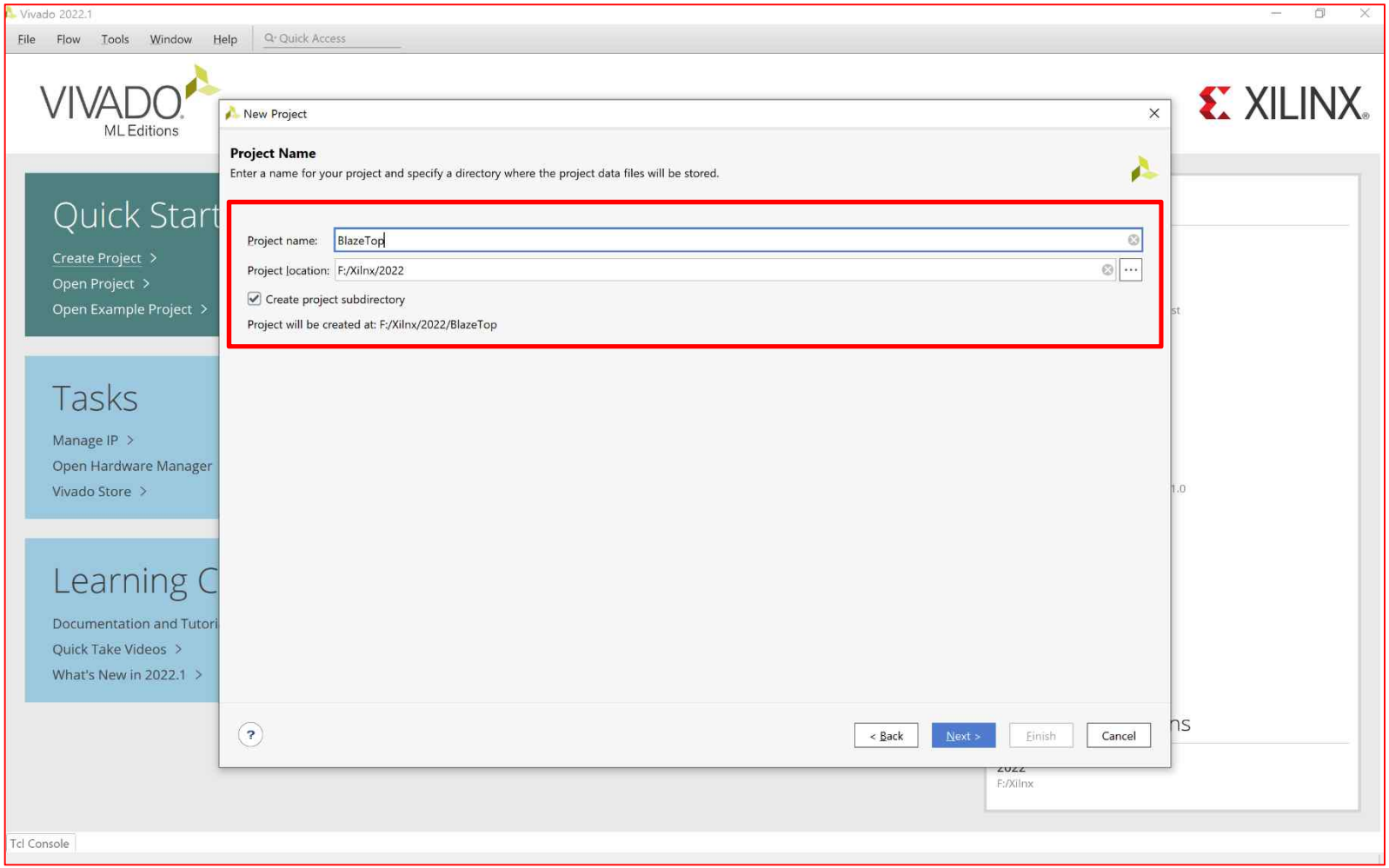
1. Create Project
2. Create Block Design
3. Add IP → MicroBlaze (Processor Core & Memory & Clock & Reset & …)
4. Add IP → AXI Uartlite (Uart IP and Baudrate)
5. Change Port Name and Interrupt Pin Connection
6. Validate Design and Regenerate Layout
7. Create HDL Wrapper
8. User Logic Implementation(1) → Led_On/Off_Logic → led_control
9. User Logic Implementation(2) → Led_On/Off_Logic → BlazeTop
10. XDC Implementation
11. Generate Bitstream
12. Export Hardware (XSA File Extraction)
13. SW Implementation(1) → Launch Vitis IDE & Create Application Project
14. SW Implementation(2) → [helloworld.c] Code Implementation
15. SW Implementation(3) → Build Project , Debug and Program Device
16. Check The Result

LED_Counter Implementation

- *Create Project*

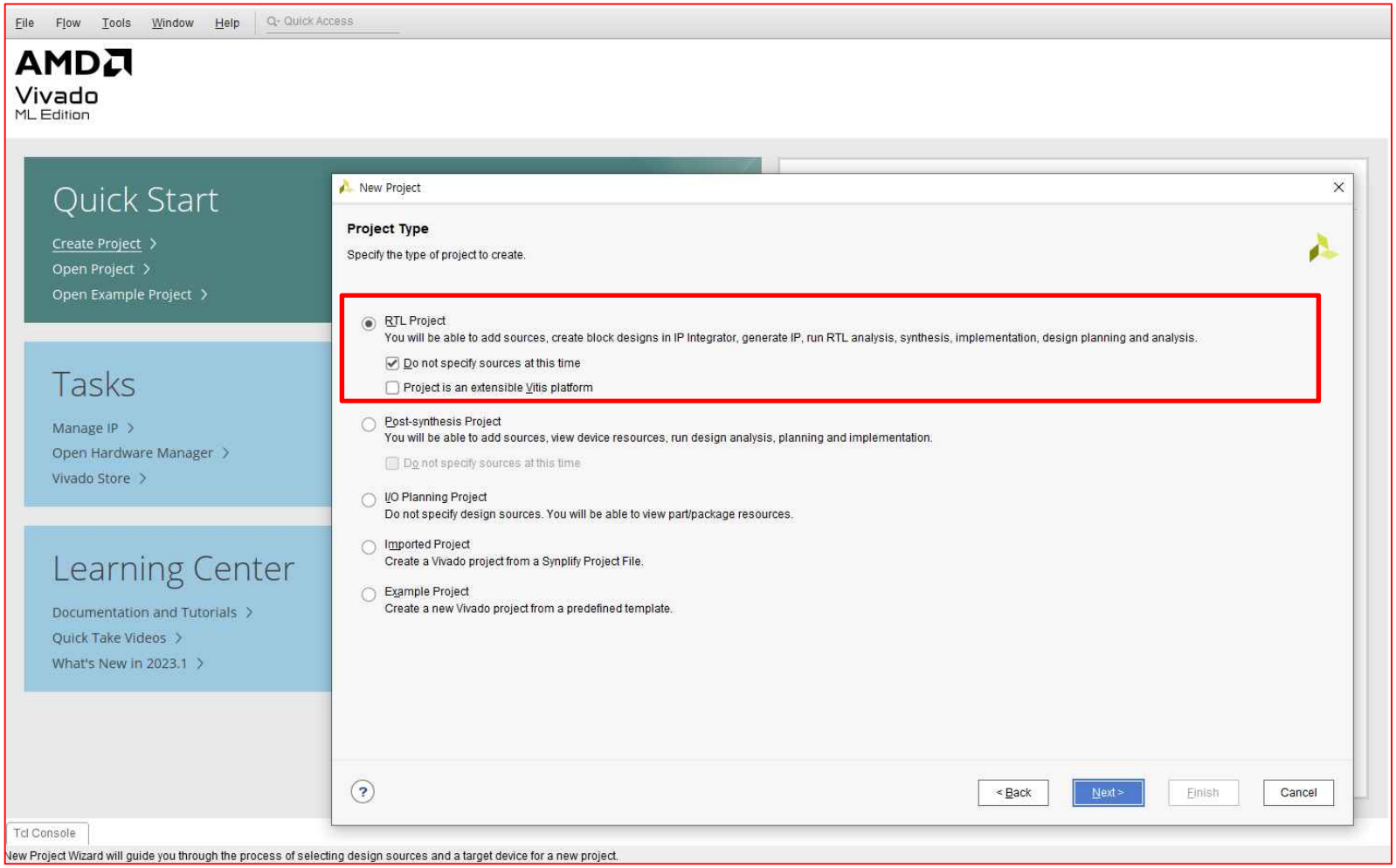
LED_Counter Implementation

- *LED_Counter Implementation* ➔ *Basic Template Implementation*
  - *Vivado* 실행 → [*Create Project*] 클릭 → [*Next*]
  - [*Project name*] 설정 ➔ [*BlazeTop*]



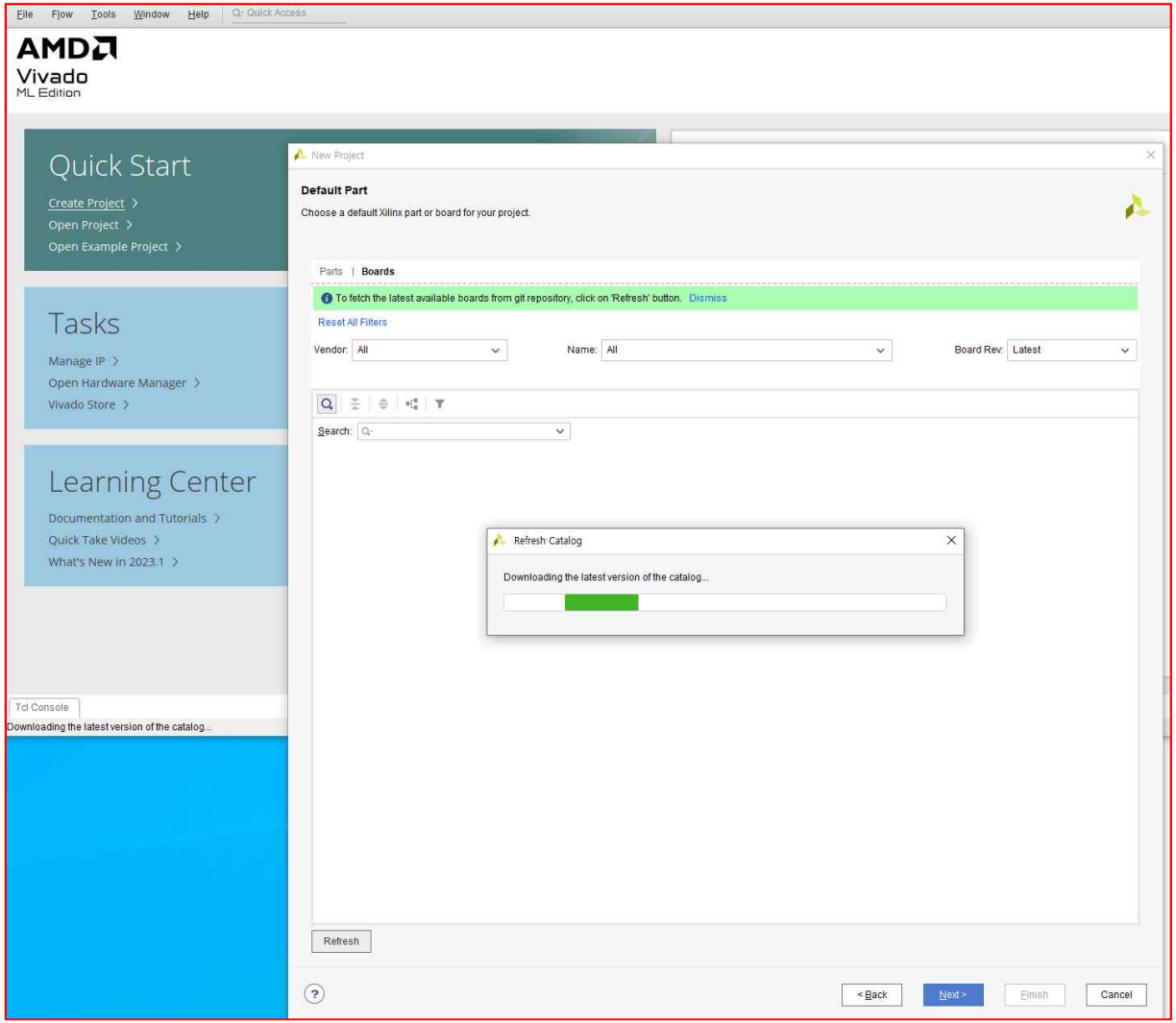
LED_Counter Implementation

- *LED_Counter Implementation → Basic Template Implementation*
  - *RTL Project → Next*



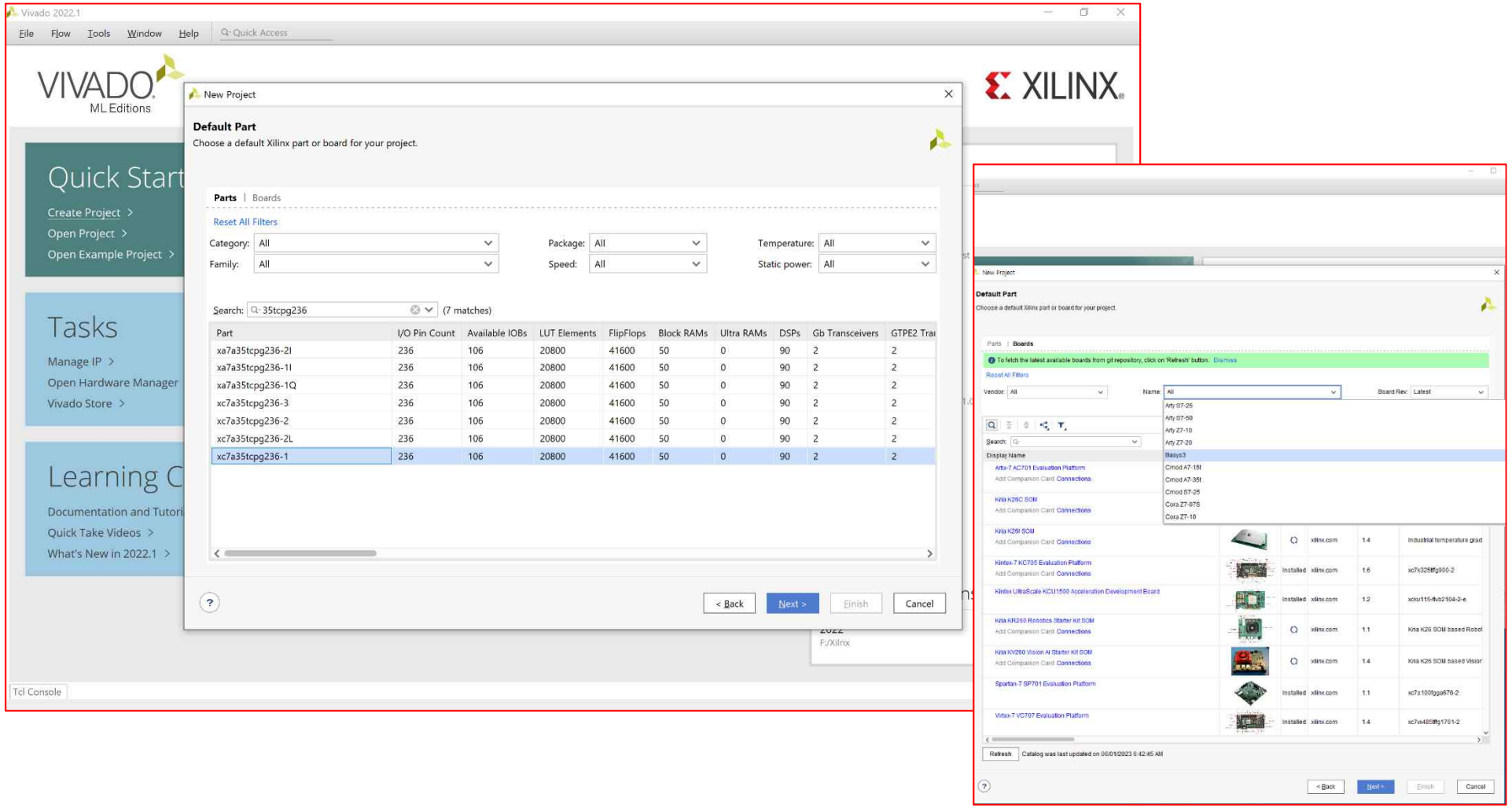
LED_Counter Implementation

- *LED_Counter Implementation ➔ Basic Template Implementation*
  - *Default Part ➔ Boards ➔ Basys3 ➔ Next*



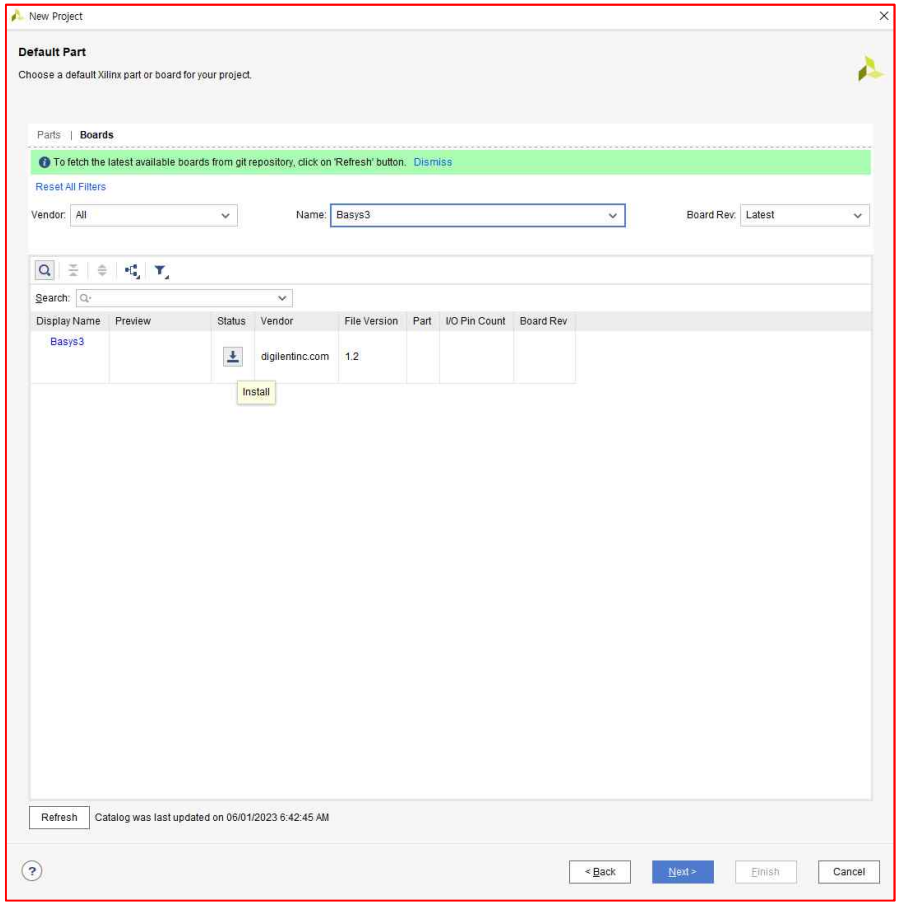
LED_Counter Implementation

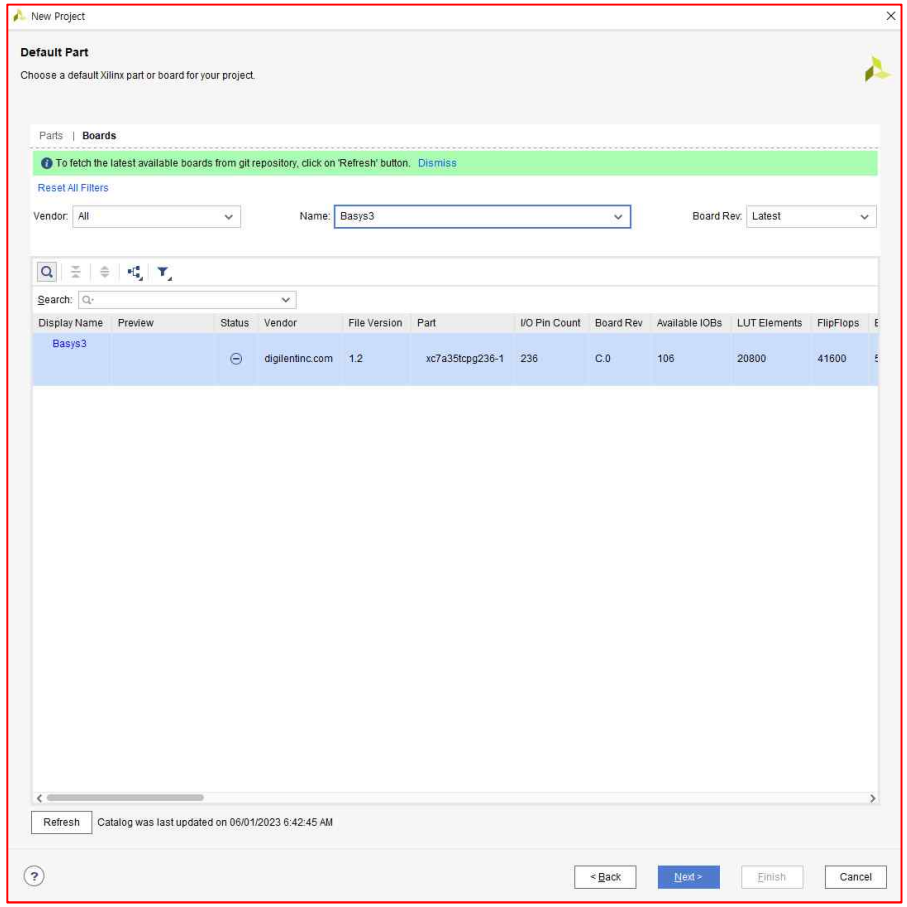
- *LED_Counter Implementation ➔ Basic Template Implementation*
  - *Default Part ➔ Parts ➔ xc7a35tcpg236-1 ➔ Next*
    - ✓ *[or] Default Part ➔ Boards ➔ Basys3 ➔ Next*



LED_Counter Implementation

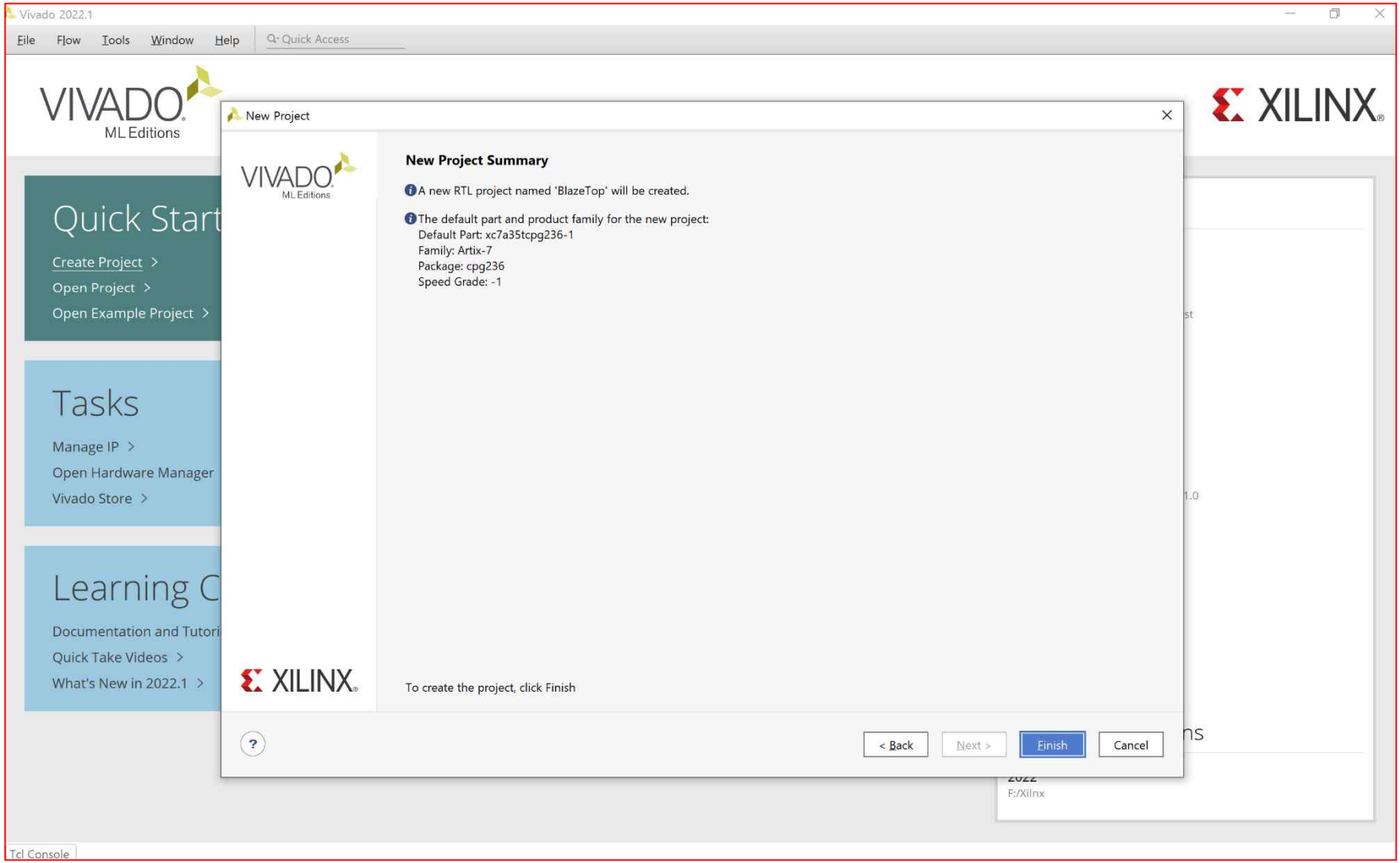
- *LED_Counter Implementation ➔ Basic Template Implementation*
  - *Default Part ➔ Parts ➔ xc7a35tcpg236-1 ➔ Next*
    - ✓ *[or] Default Part ➔ Boards ➔ Basys3 ➔ Next*





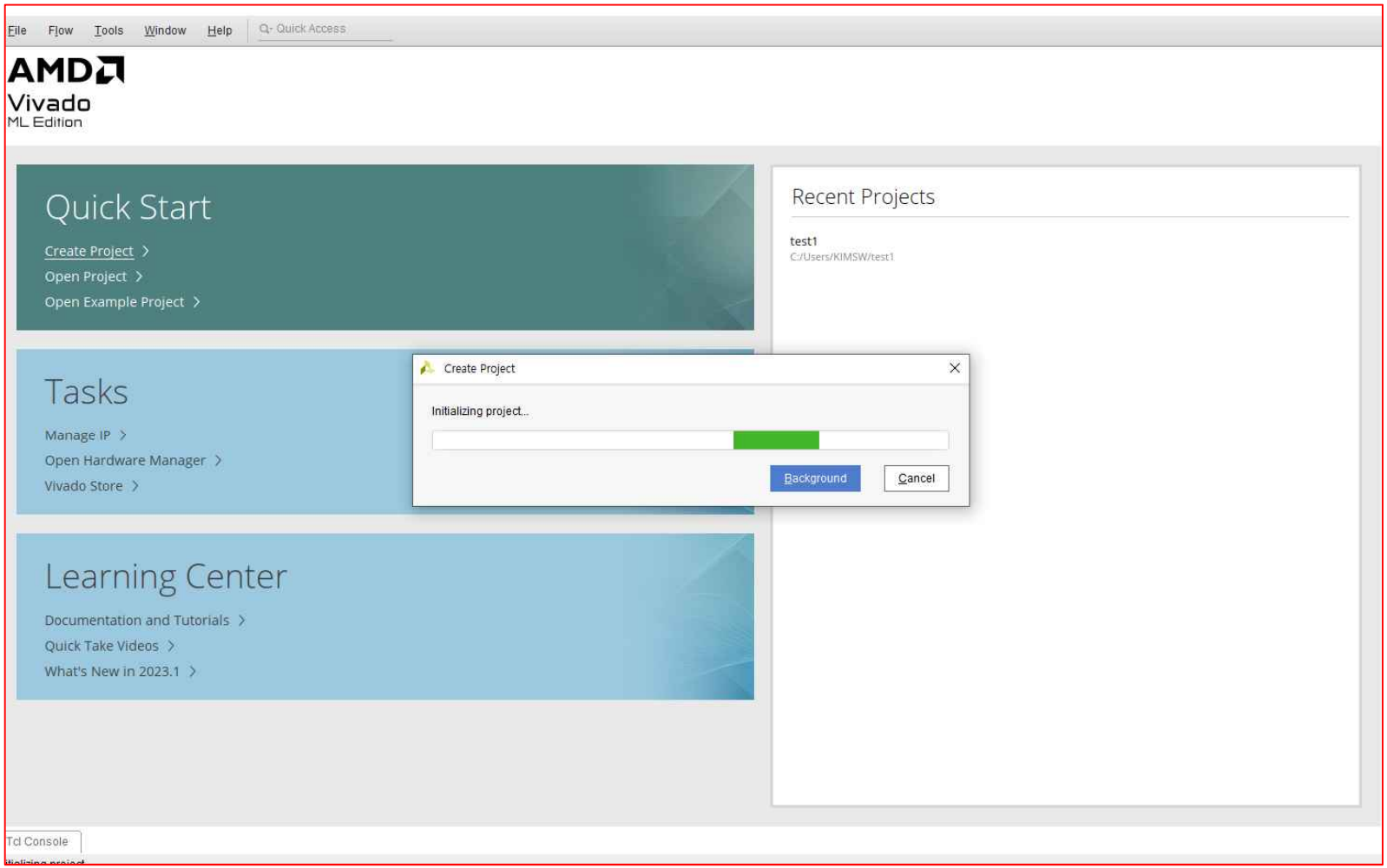
LED_Counter Implementation

- *LED_Counter Implementation ➔ Basic Template Implementation*
  - *New Project Summary ➔ Finish*



LED_Counter Implementation

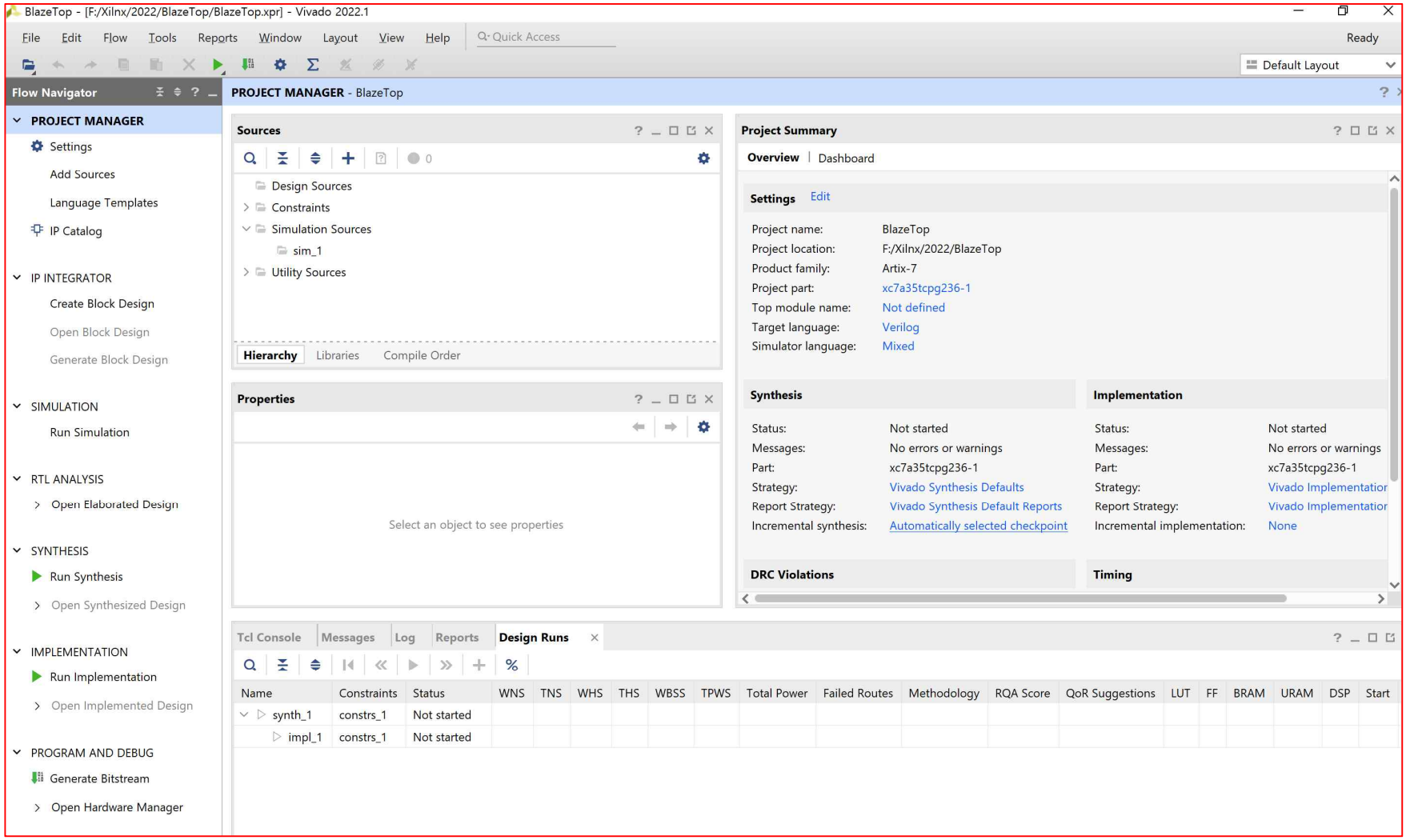
- ***LED_Counter Implementation***
  - ***Create Project*** 완료



LED_Counter Implementation

➢ *LED_Counter Implementation*

- *Create Project* 완료 ➔ ***PROJECT MANAGER***

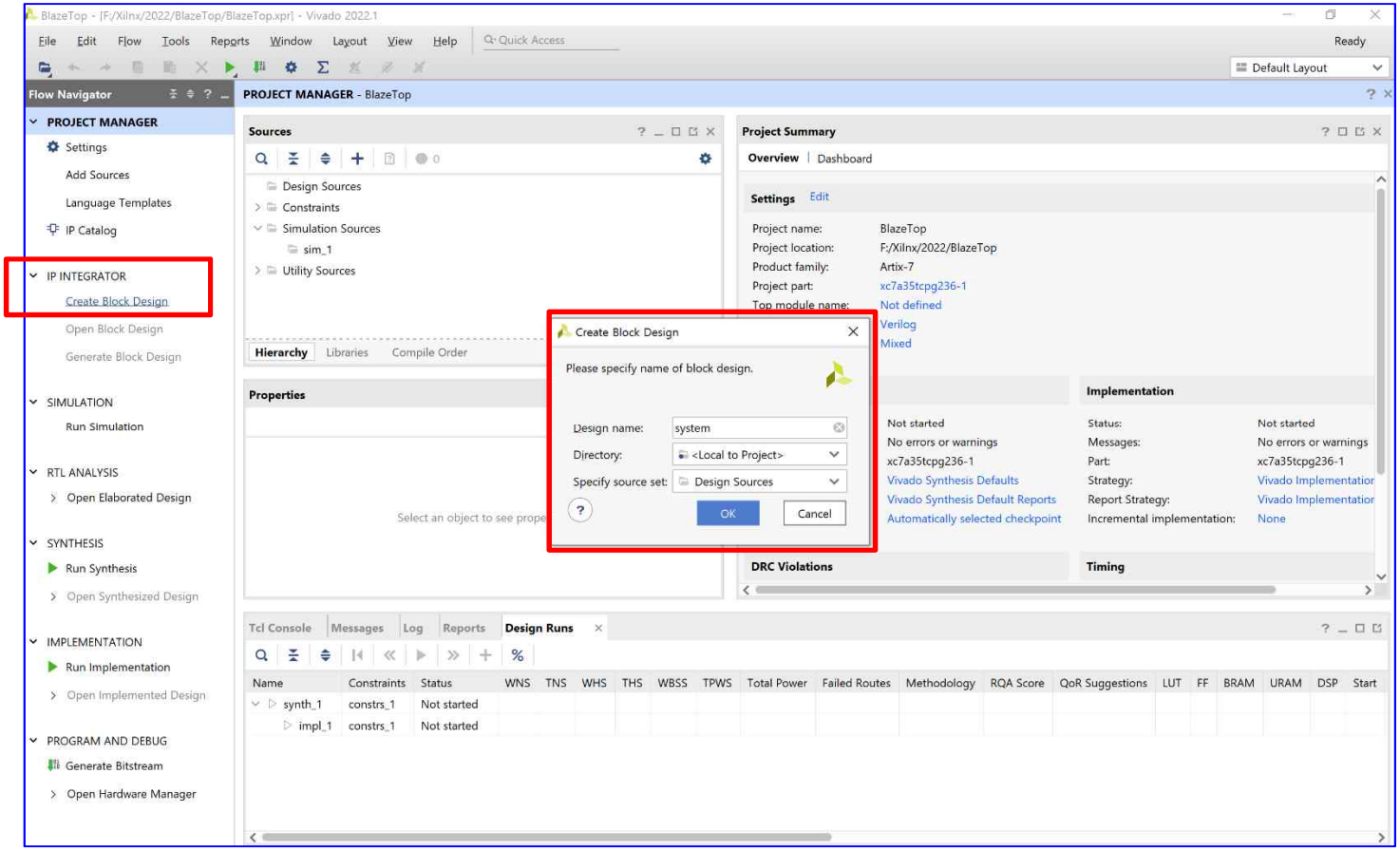


LED_Counter Implementation

- Create Project
- Create Block Design

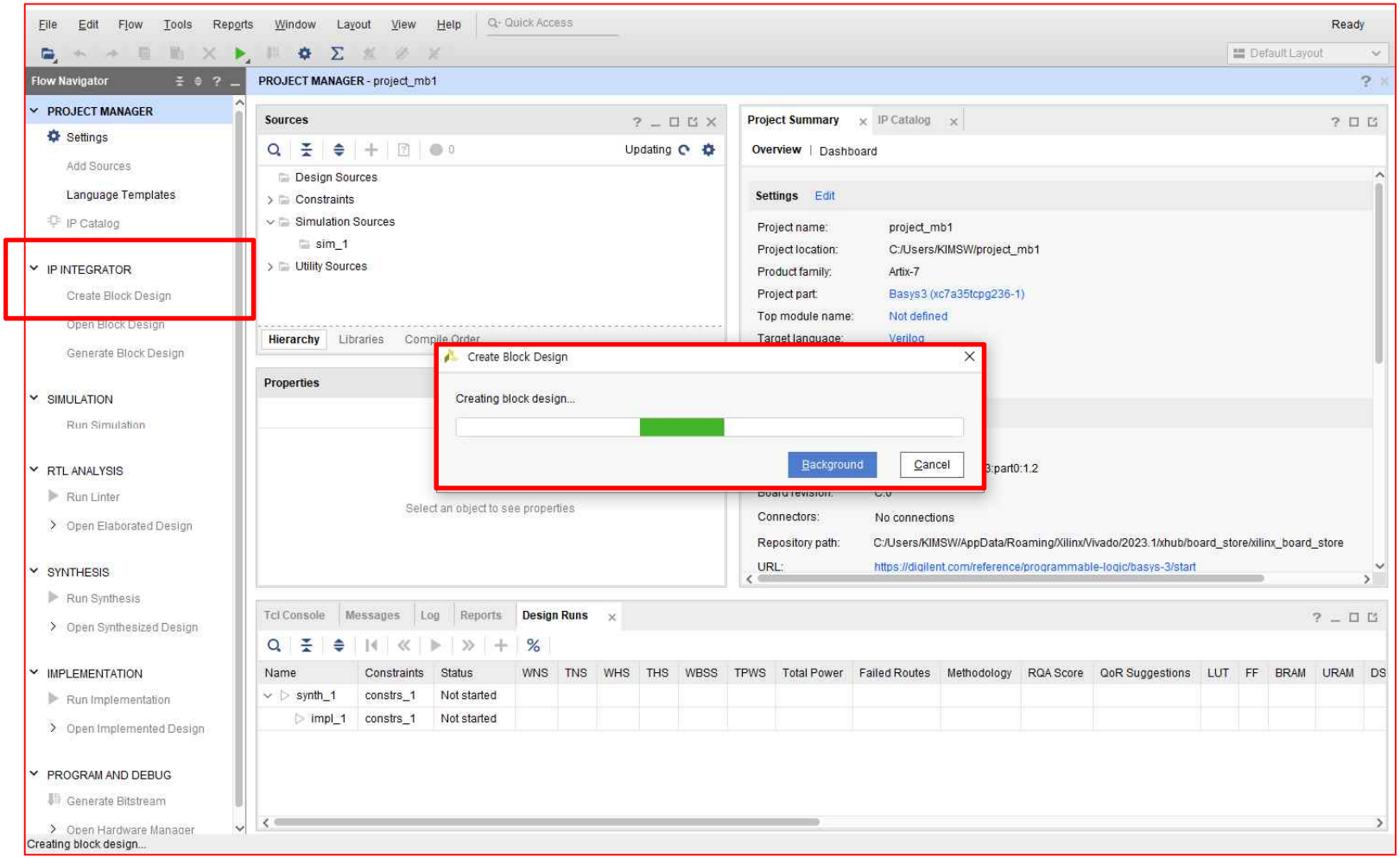
LED_Counter Implementation

- ***LED_Counter Implementation*** → ***Microblaze*** 생성
  - ***PROJECT MANAGER*** → ***IP INTEGRATOR*** → ***Create Block Design***
  - ***Microblaze*** : 사용자가 원하는 ***IP Block*** 을 선택 , 속성 지정 및 각 *Block* 들을 연결해서 사용
  - ***Design Name [(ex) system]*** 입력 → ***OK***



LED_Counter Implementation

- *LED_Counter Implementation → Microblaze 생성*
  - *IP INTEGRATOR → Create Block Design*

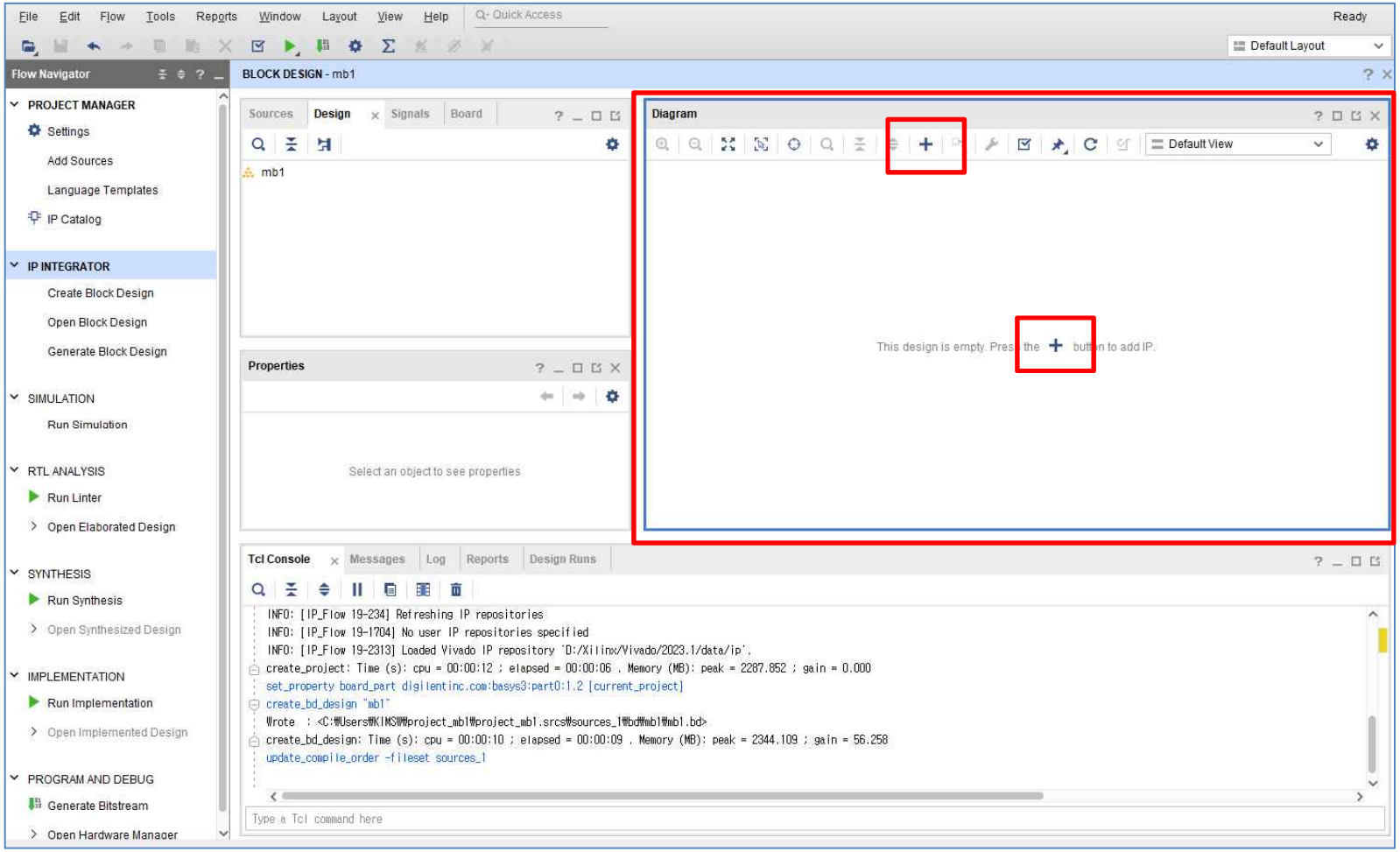


LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze

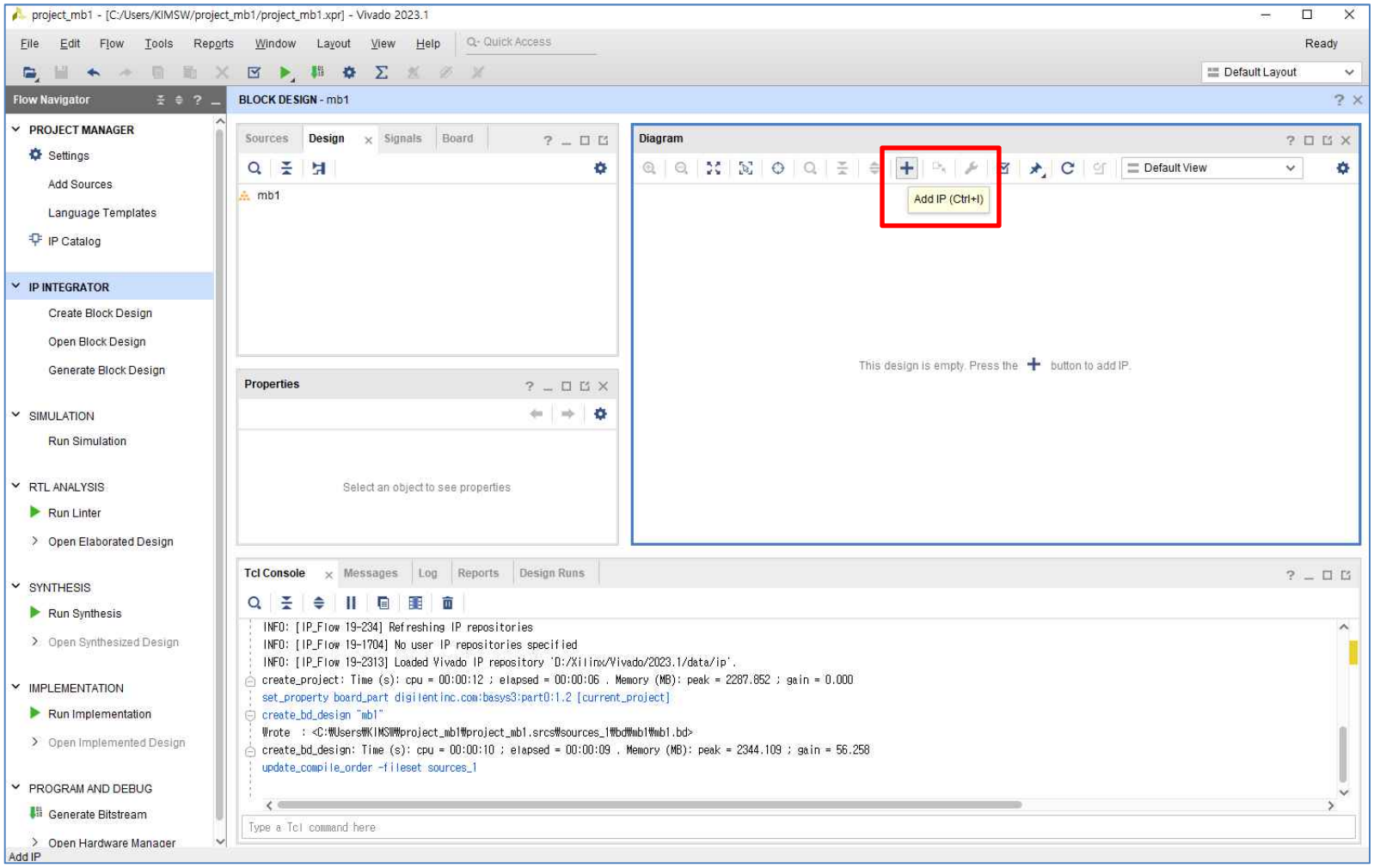
LED_Counter Implementation

- *LED_Counter Implementation* → *Microblaze* 생성
  - *Diagram Window* → *Add IP* → “ + ” 아이콘 클릭



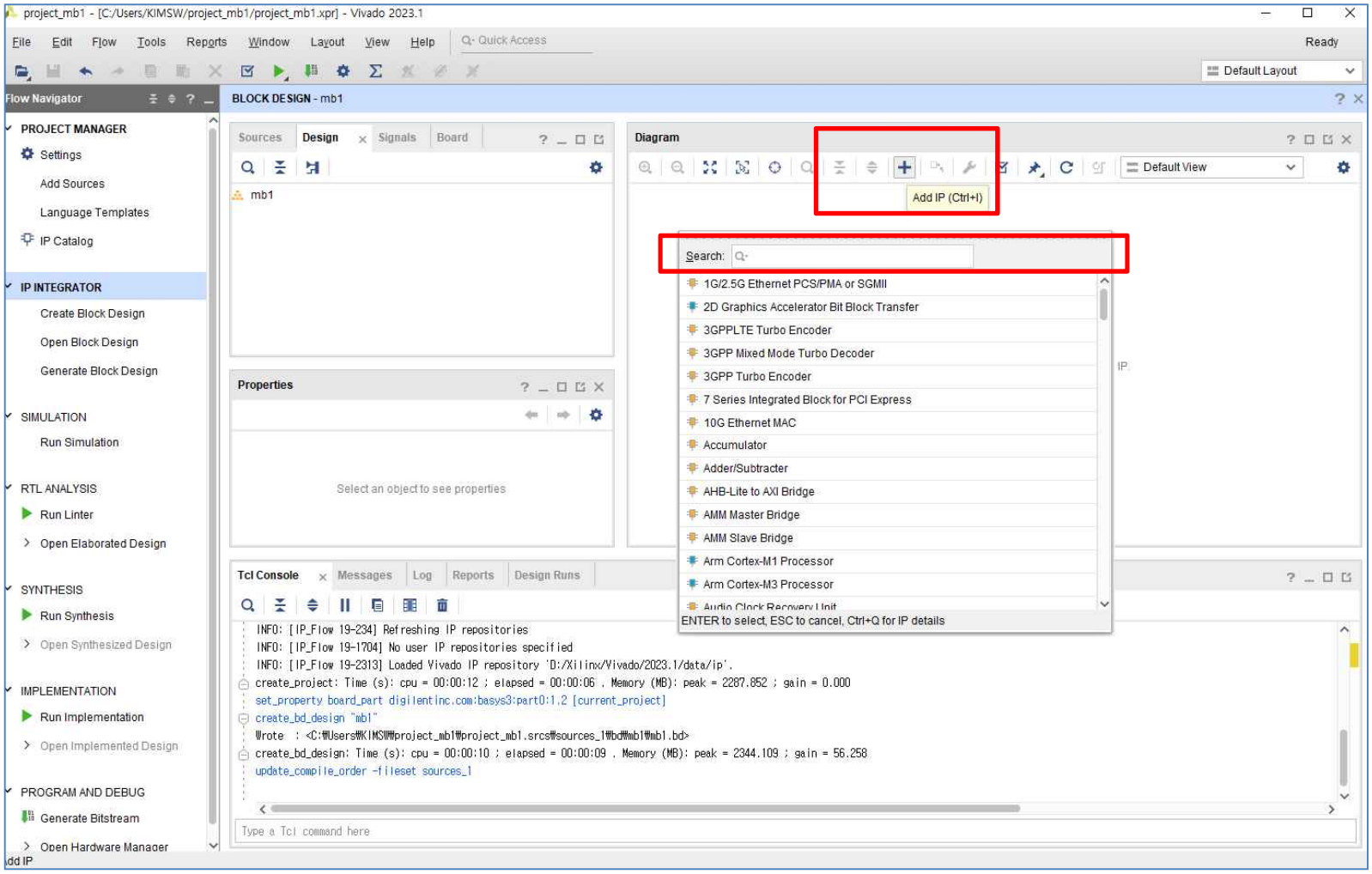
LED_Counter Implementation

- *LED_Counter Implementation → Microblaze* 생성
  - *Diagram Window → Add IP →* “+” 아이콘 클릭 → *“MicroBlaze”* 검색 및 *IP [MicroBlaze]* 추가(1/4)



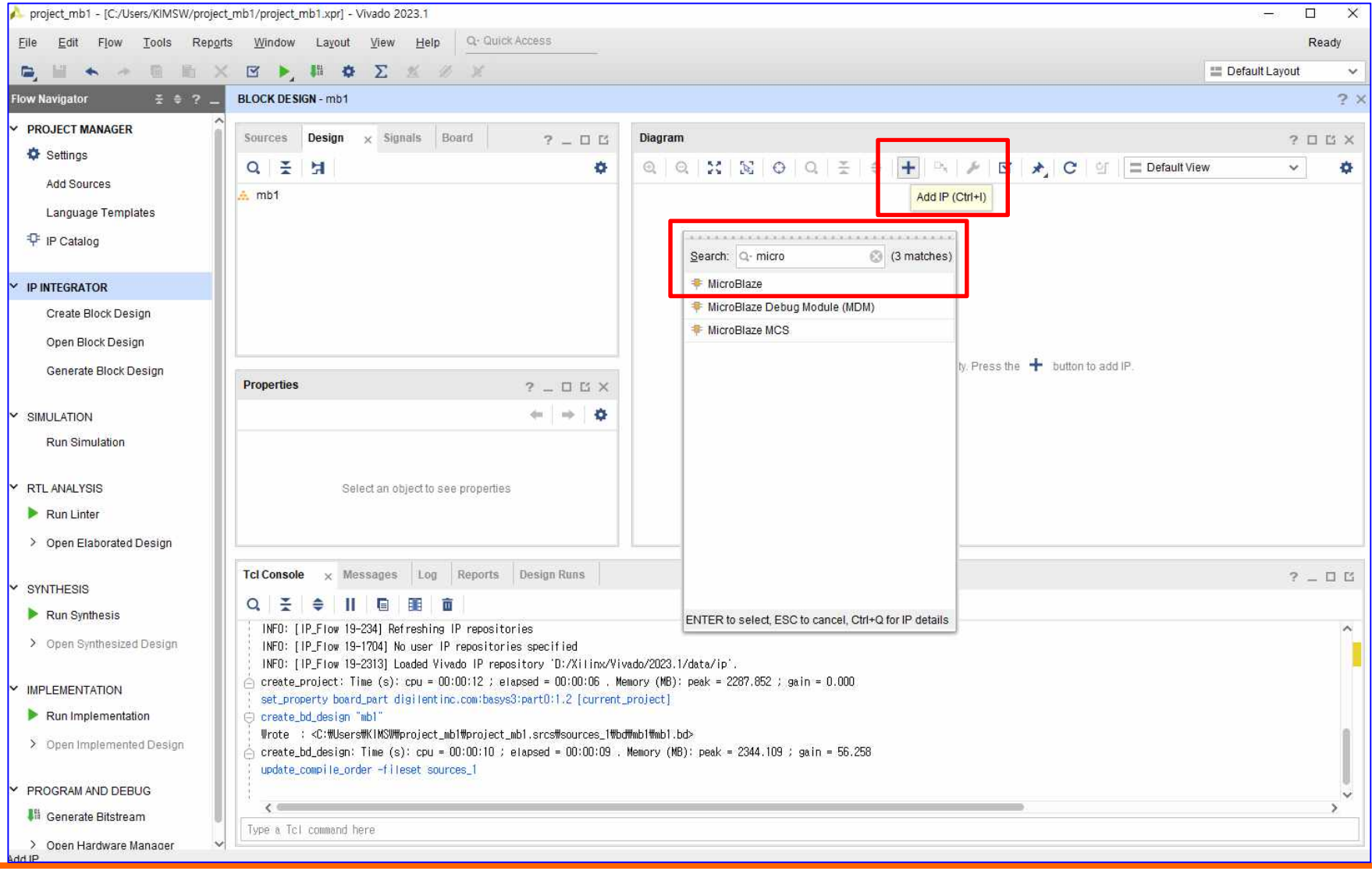
LED_Counter Implementation

- *LED_Counter Implementation → Microblaze 생성*
  - *Diagram Window → Add IP → "+" 아이콘 클릭 → "MicroBlaze" 검색 및 IP [MicroBlaze] 추가(2/4)*



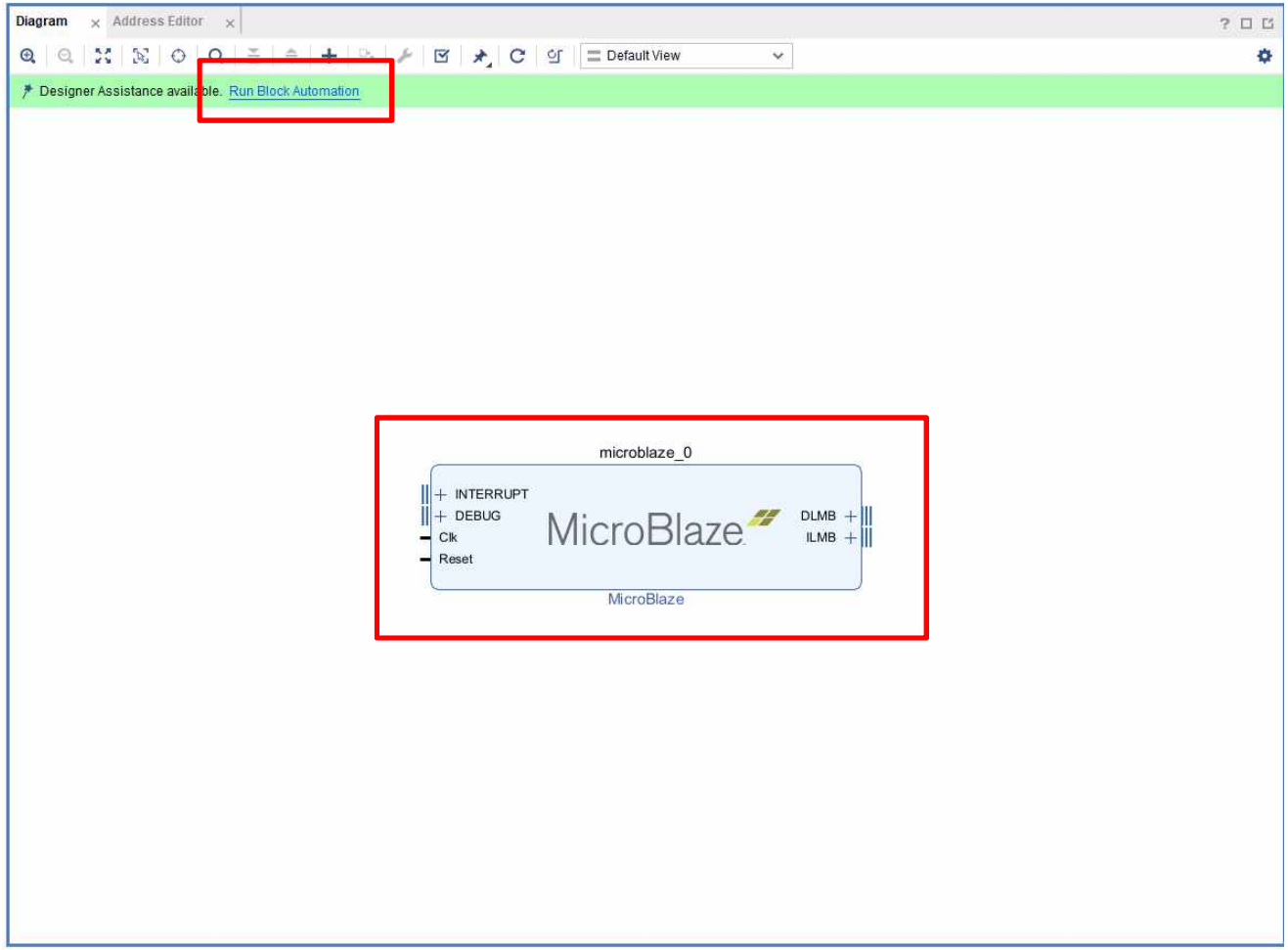
LED_Counter Implementation

- *LED_Counter Implementation → Microblaze 생성*
  - *Diagram Window → Add IP → "+" 아이콘 클릭 → "MicroBlaze" 검색 및 IP [MicroBlaze] 추가(3/4)*



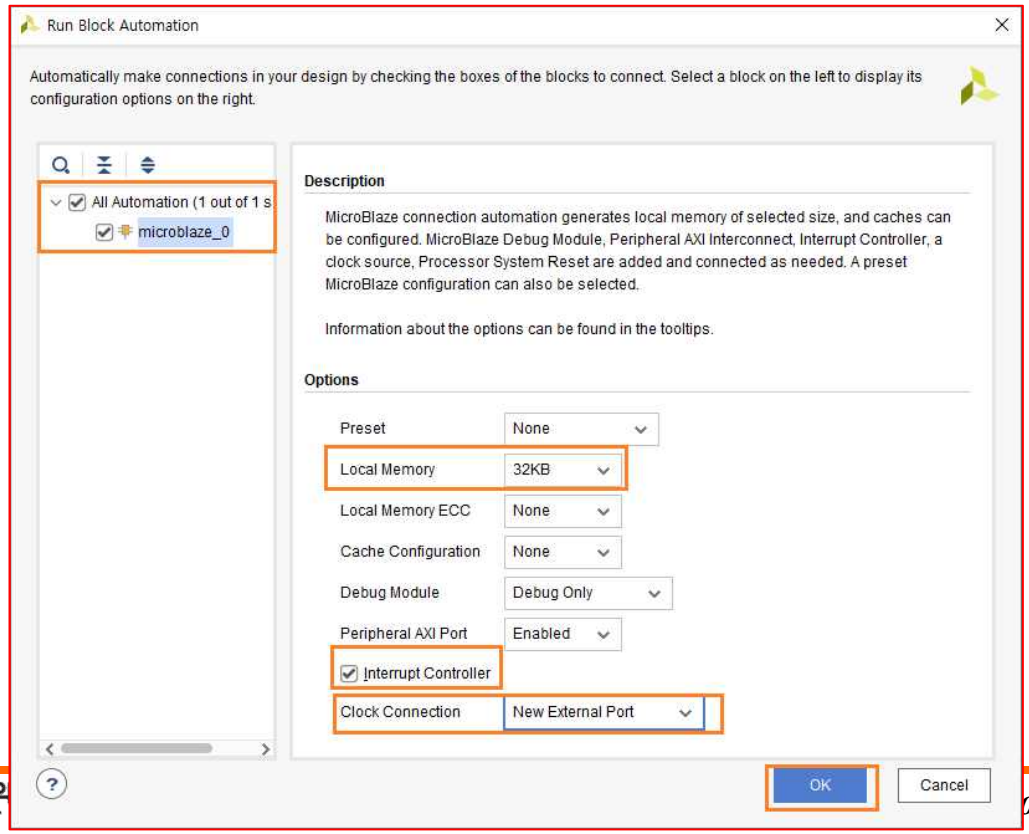
LED_Counter Implementation

- *LED_Counter Implementation → Microblaze 생성*
  - *Diagram Window → Add IP → "+" 아이콘 클릭 → "MicroBlaze" 검색 및 IP [MicroBlaze] 추가(4/4)*
  - *Run Block Automation → Options*



LED_Counter Implementation

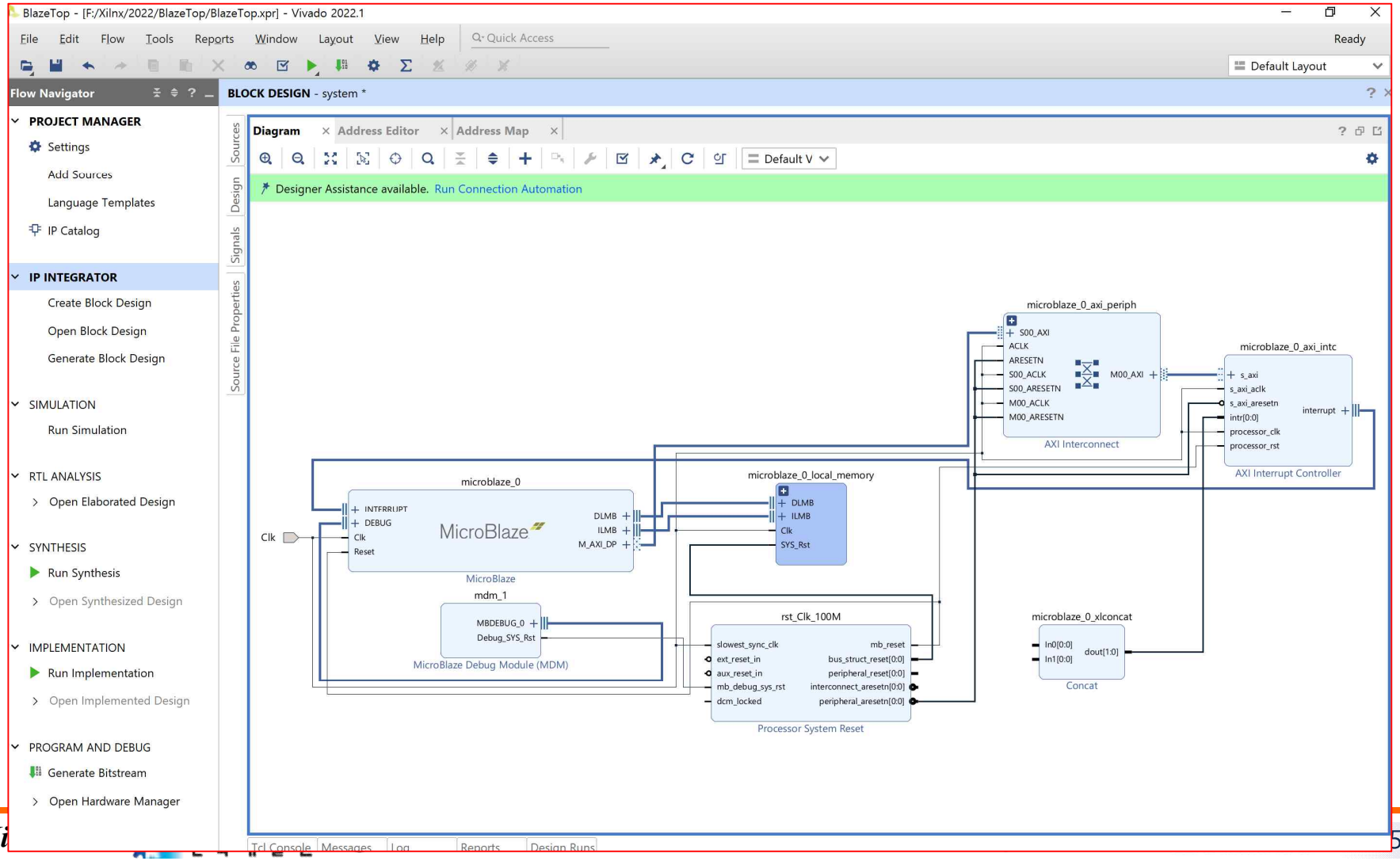
- *LED_Counter Implementation* → *Microblaze* 생성
  - *Run Block Automation* → *Options*
  - *Run Block Automation* 윈도에서 속성을 아래와 같이 설정하고 *OK* 를 클릭
  - *Local Memory : 32KB* [기본값은 *8KB* 이지만, *32KB*로 변경]
  - *Interrupt Controller : checked, Interrupt Controller*를 사용하기 위하여 선택
  - *Clock Connection : New External Port* [기본값은 *New Clock Wizard* → 보드에 있는 *100Mhz*]
  - *clock*을 그대로 사용하기 위하여 *New External Port*를 선택



LED_Counter Implementation

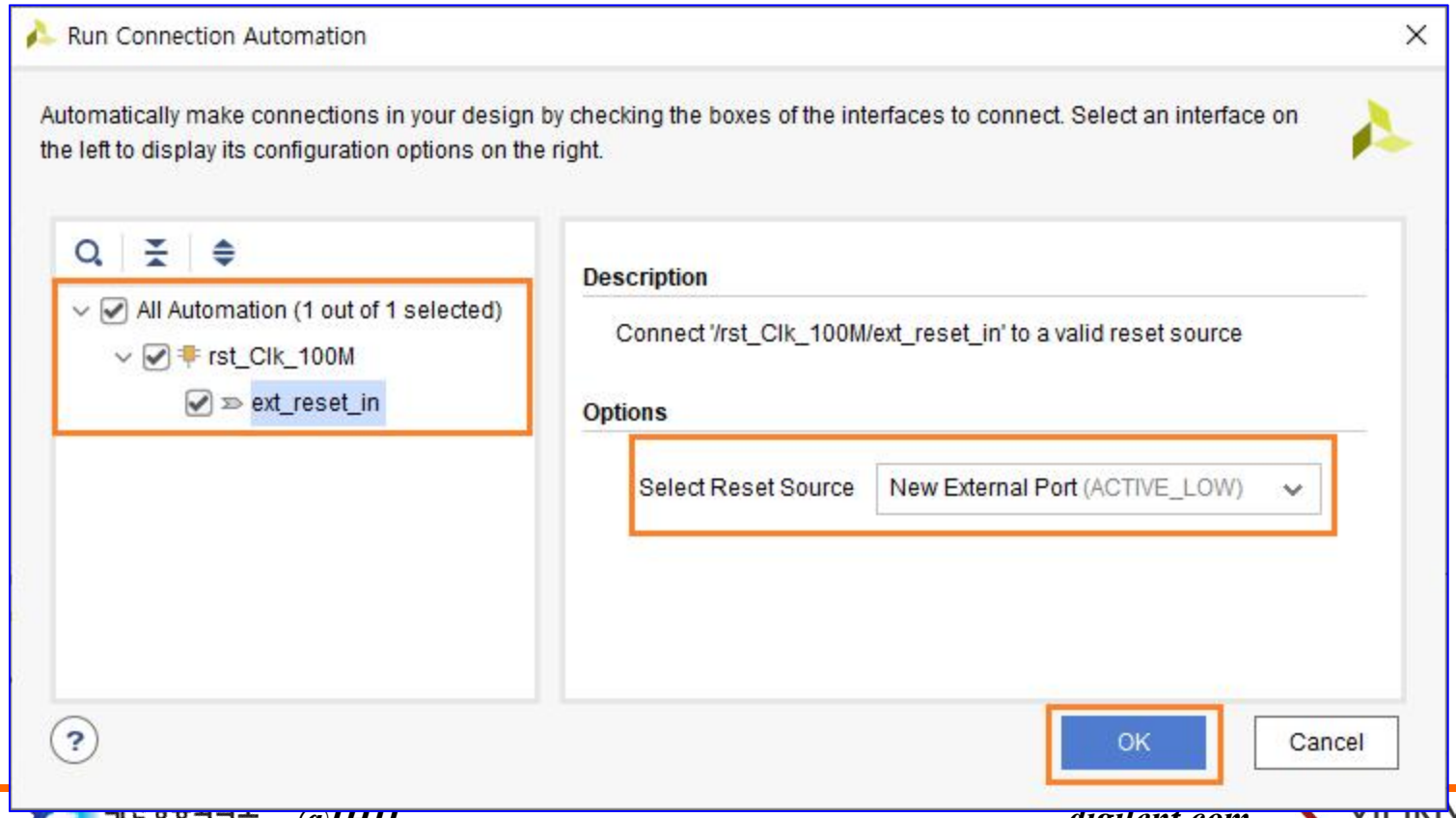
➢ LED_Counter Implementation → Microblaze 생성

- *Run Block Automation* → *Options*
- *Run Block Automation* 윈도에서 속성을 아래와 같이 설정하고 OK 를 클릭
- *Local Memory : 32KB* | 기본값은 8KB 이지만, 32KB로 변경 |
- *Interrupt Controller : checked, Interrupt Controller*를 사용하기 위하여 선택
- *Clock Connection : New External Port* | 기본값은 New Clock Wizard → 보드에 있는 100Mhz |
- *clock*을 그대로 사용하기 위하여 *New External Port*를 선택



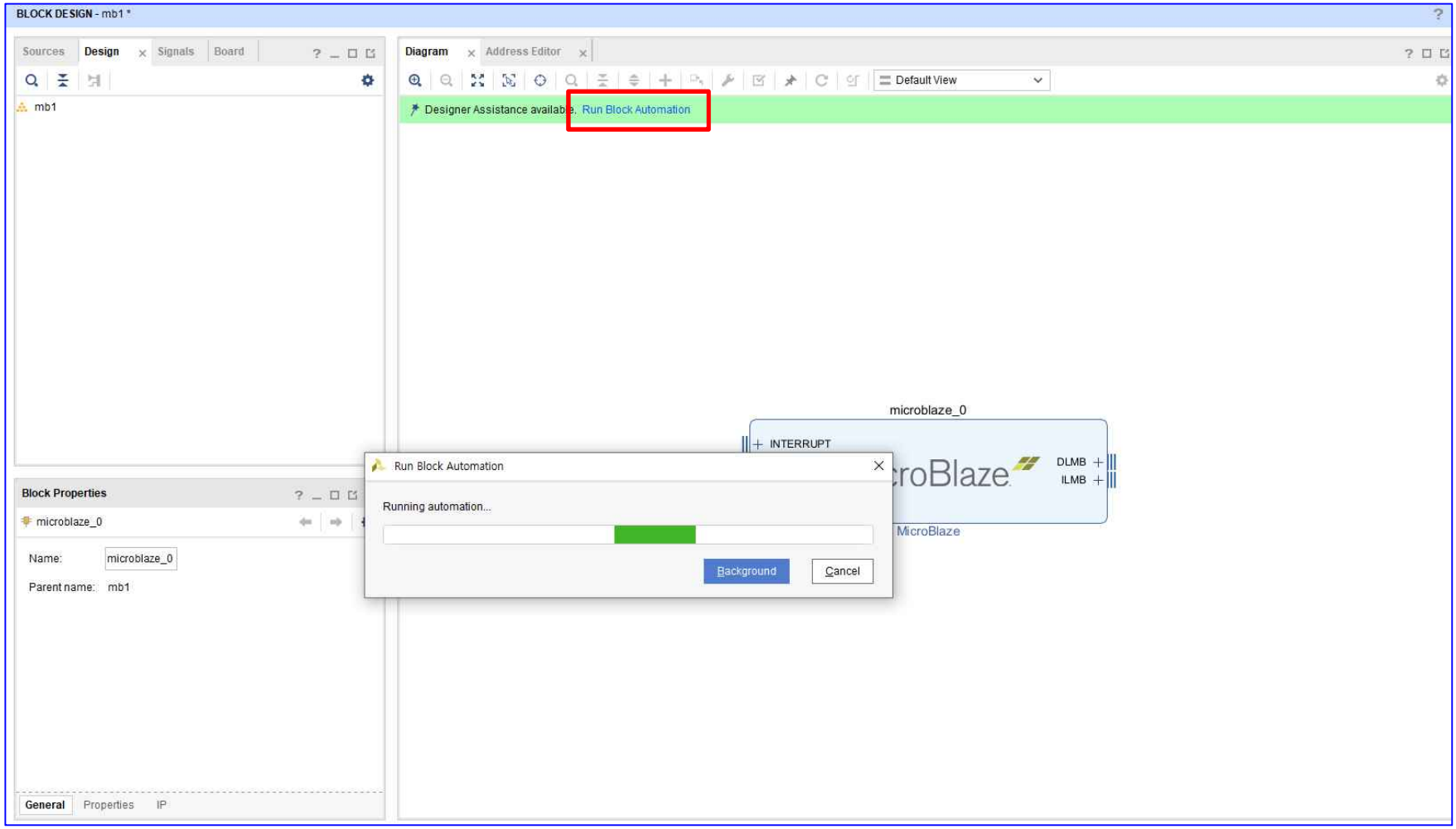
LED_Counter Implementation

- ***LED_Counter Implementation***
  - ***Run Connection Automation*** ➔ ***Run Connection Automation*** 윈도에서는 왼쪽에는 아직 연결되지 않은 Block을 나타내고 해당 핀을 선택하면 오른쪽에 속성을 설정할 수 있음
  - ***rst_Clk_100M Block***의 ***ext_rest_in*** 핀이 연결되지 않았음 ➔ ***ext_rest_in*** 핀을 선택하고 오른쪽에서 속성을 설정
  - 보드에 있는 ***RESET*** 핀은 ***Active Low*** 이므로, "***New External Port (Active_Low)***를 선택하고 ***OK***를 클릭



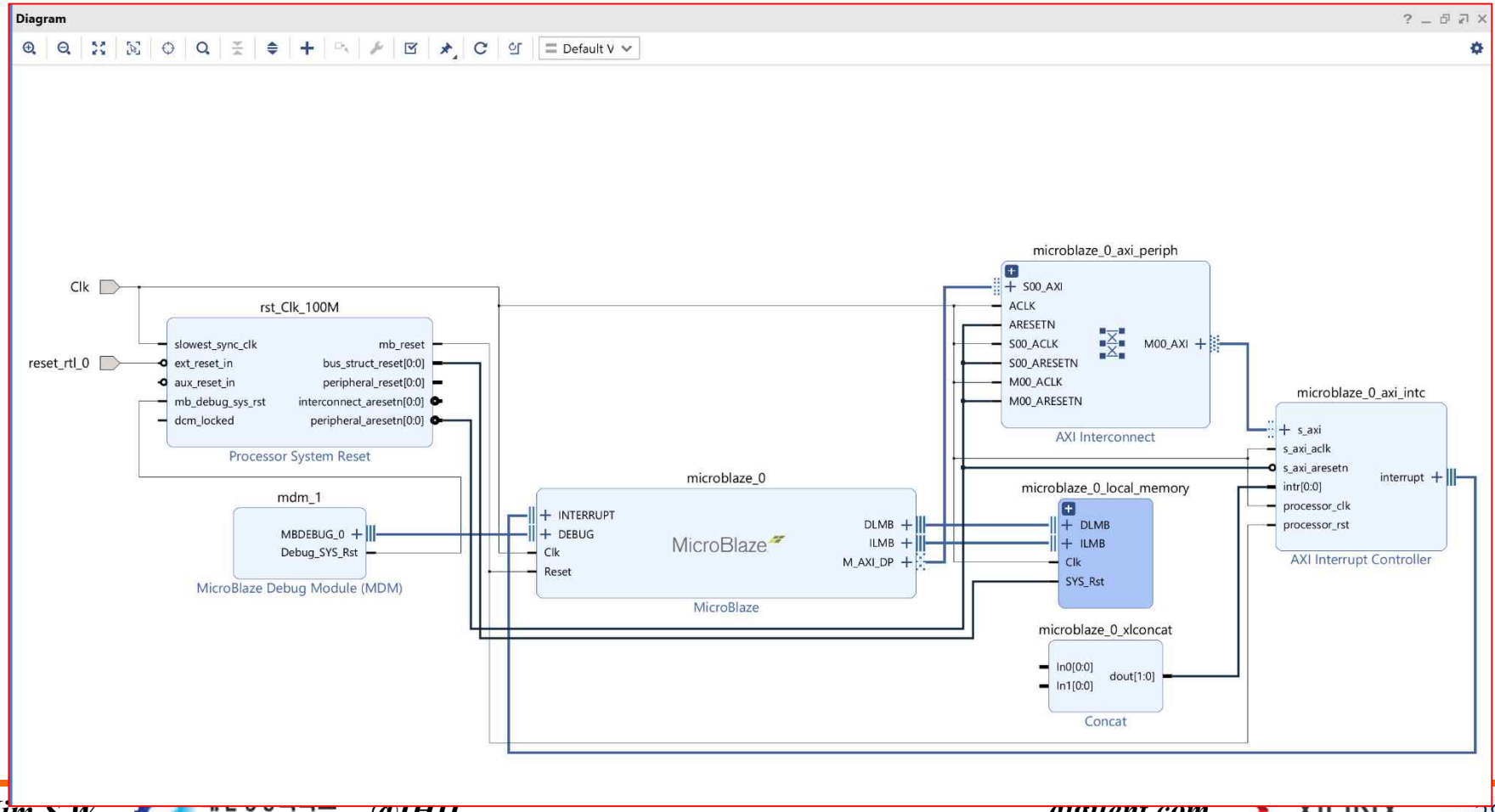
LED_Counter Implementation

- *LED_Counter Implementation*
  - *Run Connection Automation* ➔ *Block* 자동 연결
  - *IP* 추가 및 수정 후 ➔ *[Run Block Automation] & [Run Connection Automation]* 기능 적용



LED_Counter Implementation

- *LED_Counter Implementation*
  - *Run Connection Automation* ➔ *Block* 자동 연결
  - *[All Automation ➔ OK]* 생성된 *Block* 자동 연결
  - 생성 완료 ➔ *Clk, reset_rtl_0* 포트가 추가됨



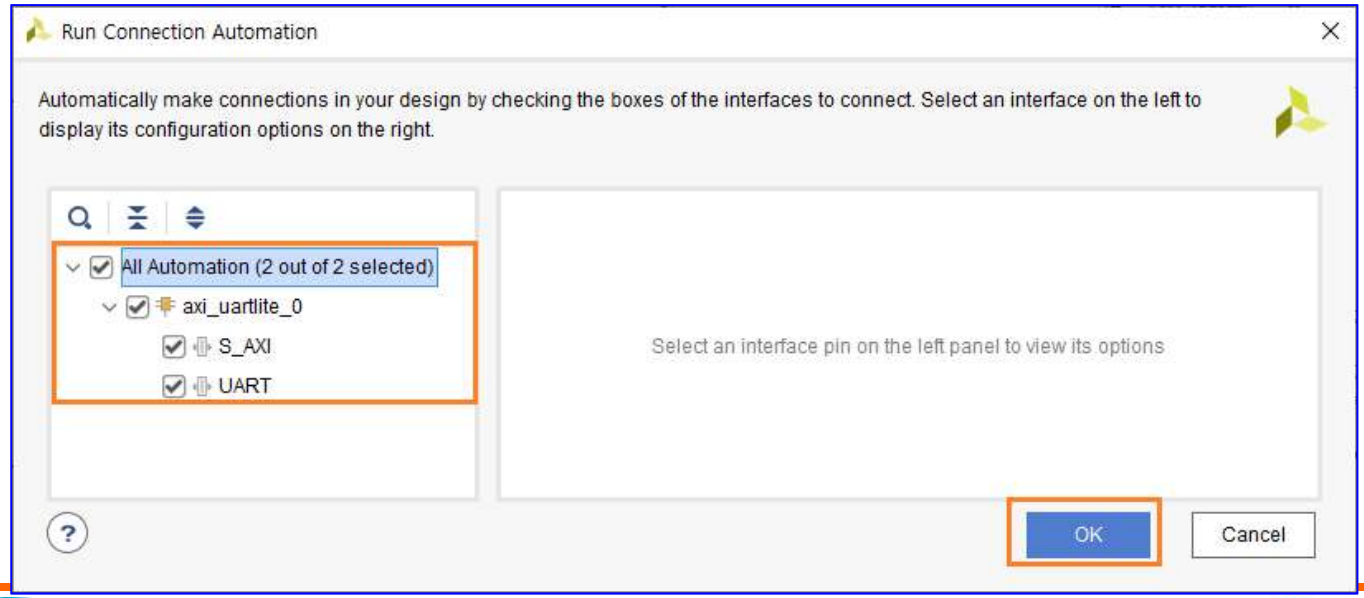
LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite

LED_Counter Implementation

- *LED_Counter Implementation*
  - *Add IP* 아이콘 (+)을 클릭해서 *UART Block*을 추가
  - *uart* 입력 후 나오는 *IP* 중에 "*AXI Uartlite*"를 *Double Click* 해서 추가

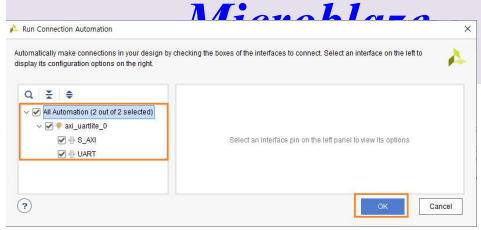
  - "*Run Connection Automation(2 out of 2selected)*"을 클릭해서 연결 ➔ *All Automation*을 선택해서 모든 *Block*을 선택하고 **OK** 클릭

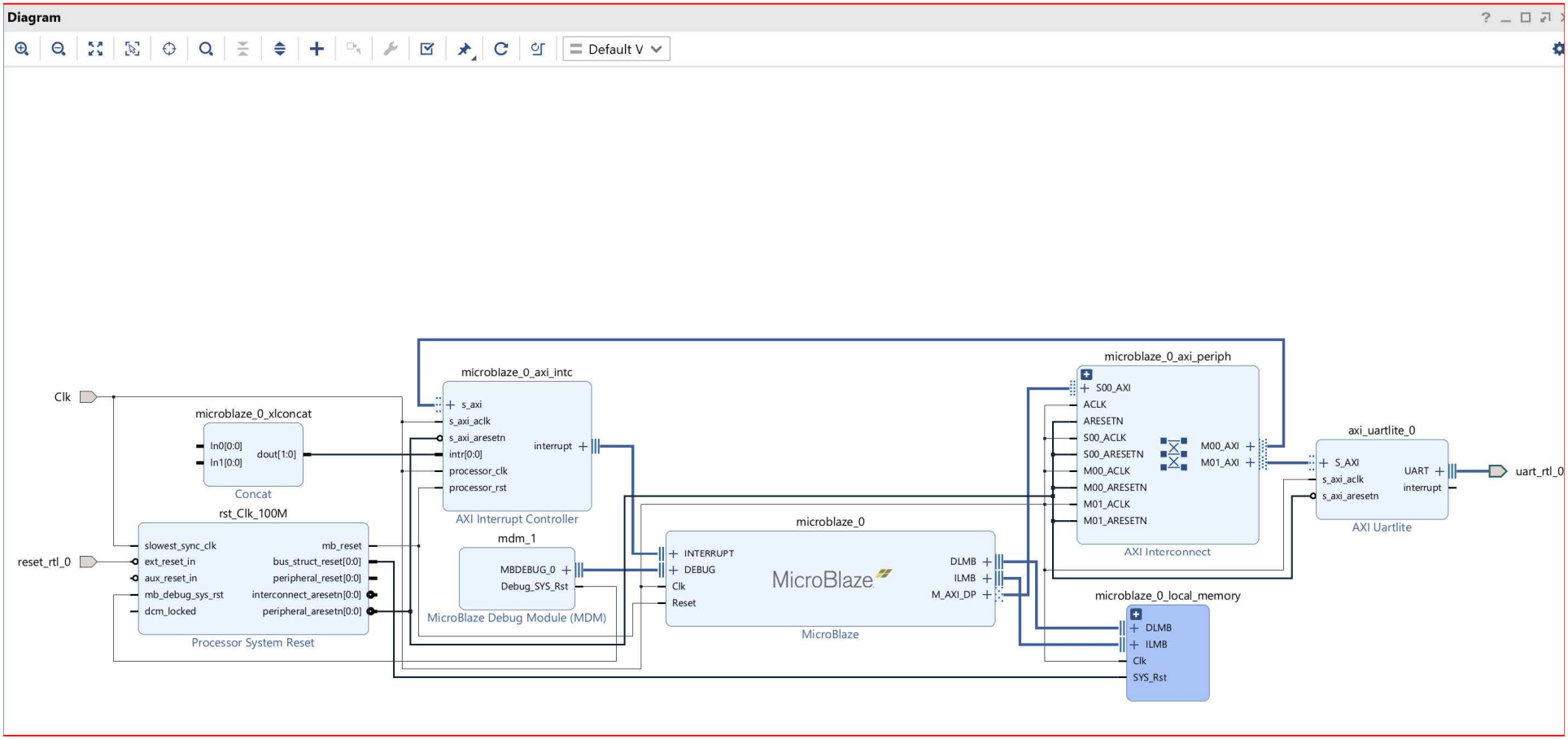


LED_Counter Implementation

➢ LED_Counter Implementation

- *Add IP* 아이콘 (+)을 클릭해서 *UART Block*을 추가
- *uart* 입력 후 나오는 *IP* 중에 "*AXI Uartlite*"를 *Double Click* 해서 추가
- "*Run Connection Automation(2 out of 2selected)*"을 클릭해서 연결 ➔ *All Automation*을 선택해서 모든 *Block*을 선택하고 **OK** 클릭





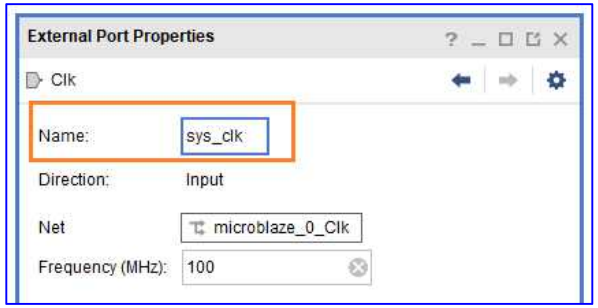
LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- **Change Port Name and Interrupt Pin Connection**

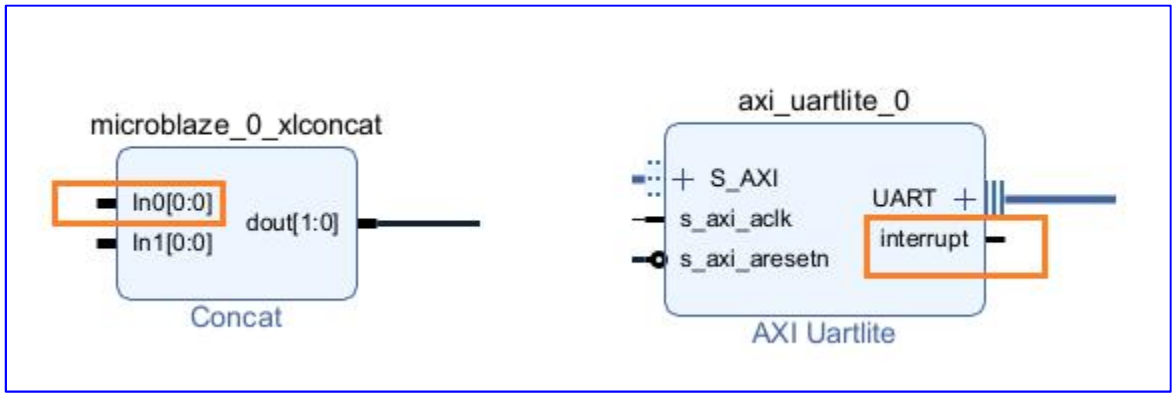
LED_Counter Implementation

➢ *LED_Counter Implementation*

- *Port* 이름을 아래와 같이 변경 ➔ 해당 포트를 선택하면 *Diagram* 오른쪽에 해당 핀의 속성이 나타남



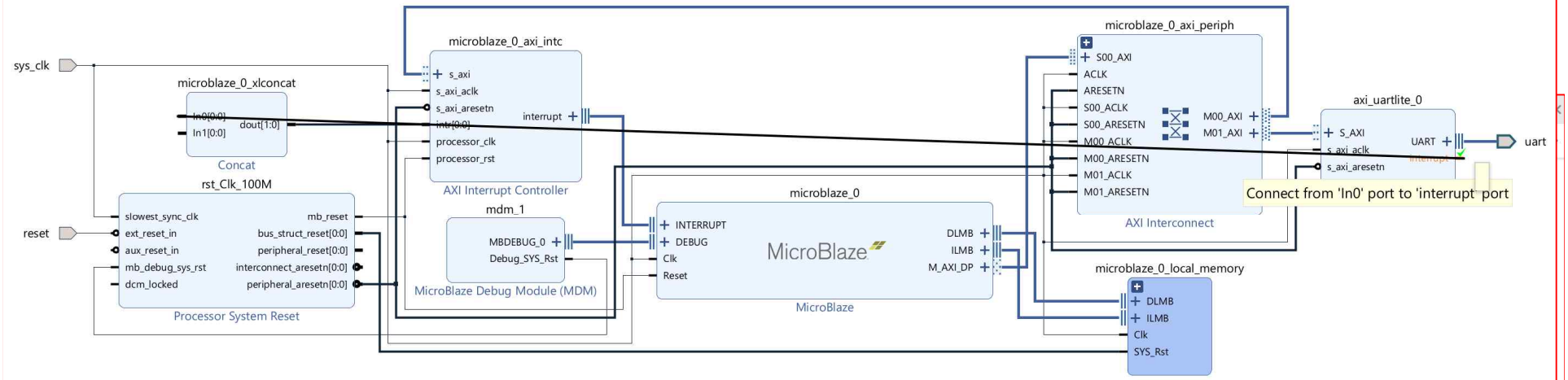
- **3개의 Port 이름을 변경 ➔ [*Clk* ➔ *sys_clk*], [*reset_rtl_0* ➔ *reset*], [*uart_rtl_0* ➔ *uart*]**
- *Interrupt* 핀은 사용자가 수동으로 연결해야 함 ➔ "*microblaze_0_xlconcat*" *block*의 *In0[0:0]* 핀과 "*axi_uartlite_0*" *block*의 *Interrupt* 핀을 연결 , 해당 핀으로 마우스를 이동으로 연필 모양으로 아이콘이 변경되며, 해당 핀을 클릭하고 드래그 해서 연결할 핀으로 이동하면 연결

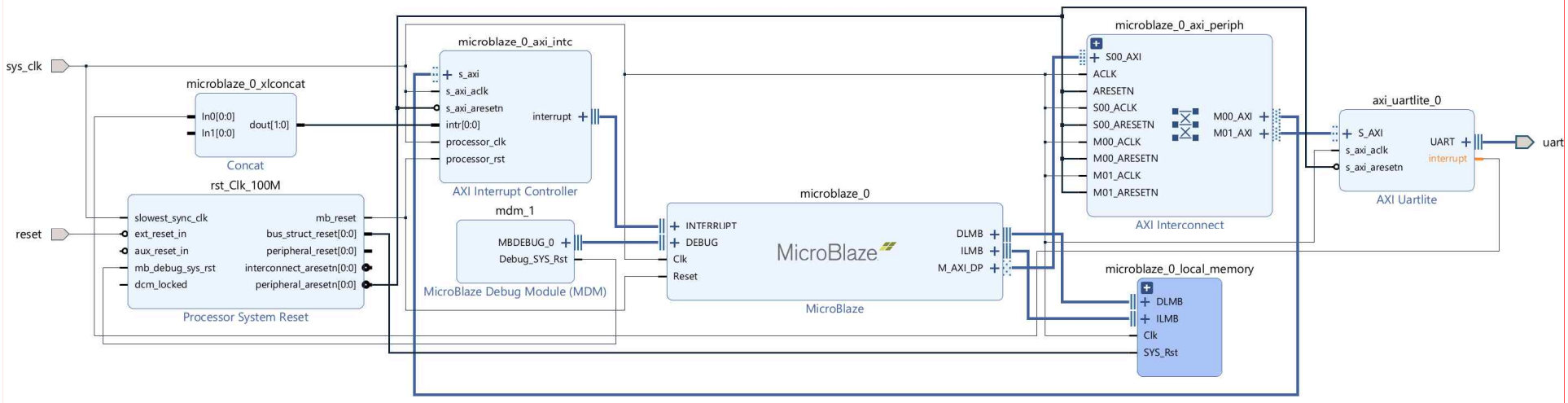


LED_Counter Implementation

LED_Counter Implementation

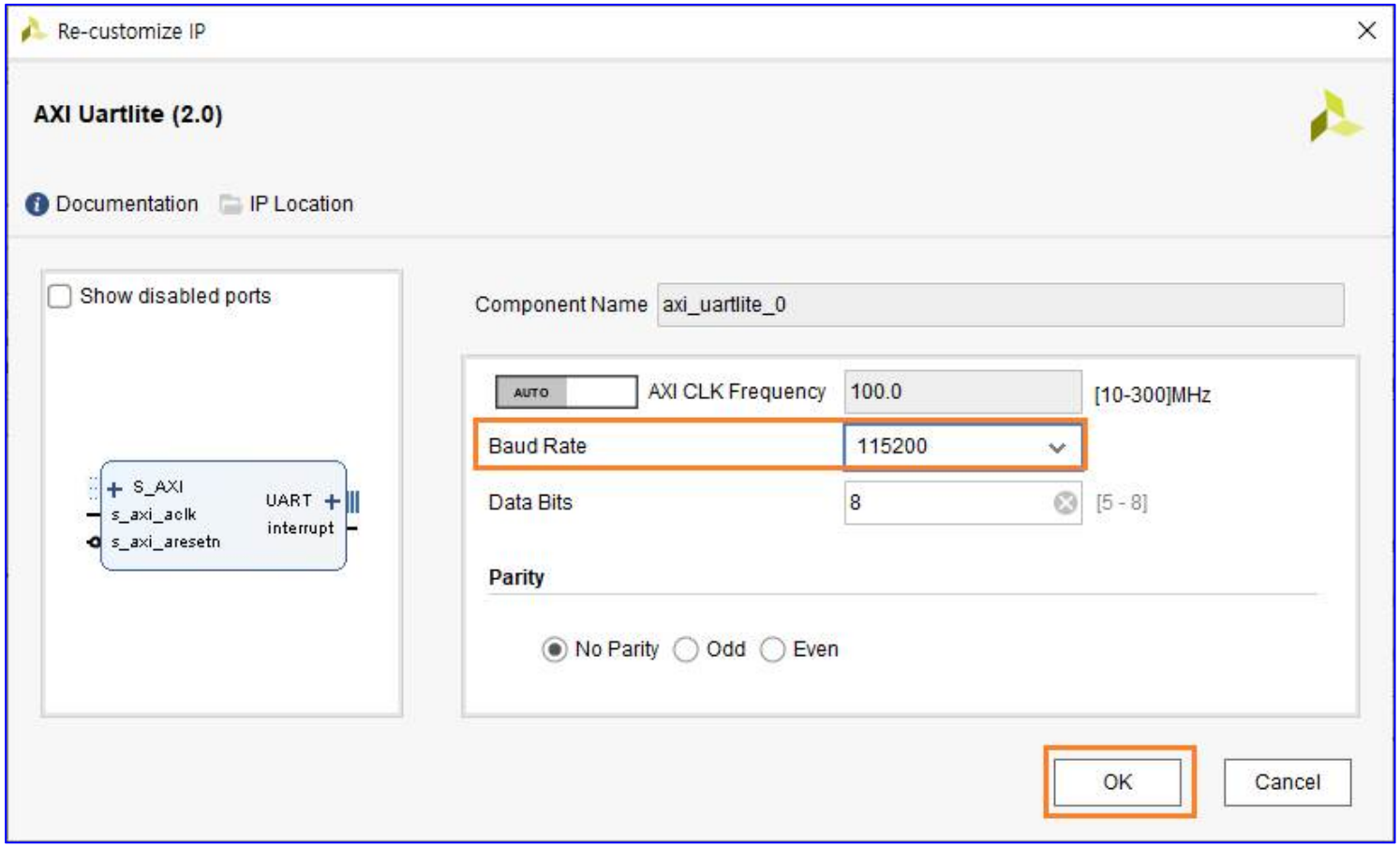
- *Port* 이름을 아래와 같이 변경 ➔ 해당 포트를 선택하면 *Diagram* 오른쪽에 해당 핀의 속성이 나타남
- 3개의 Port 이름을 변경 ➔ [*Clk* ➔*sys_clk*], [*reset_rtl_0* ➔*reset*], [*uart_rtl_0* ➔*uart*]
- *Interrupt* 핀은 사용자가 수동으로 연결해야 함 ➔ "*microblaze_0_xlconcat*" *block*의 *In0[0:0]* 핀과 "*axi_uartlite_0*" *block*의 *Interrupt* 핀을 연결 , 해당 핀으로 마우스를 이동으로 연필 모양으로 아이콘이 변경되며, 해당 핀을 클릭하고 드래그 해서 연결할 핀으로 이동하면 연결





LED_Counter Implementation

- *LED_Counter Implementation*
  - *uart block*의 *buadrate*를 수정
  - *axi_uartlite_0 block*을 더블 클릭하면 속성창이 나타남 ➔ *Baud Rate*을 *115200*으로 수정하고 *OK*를 클릭

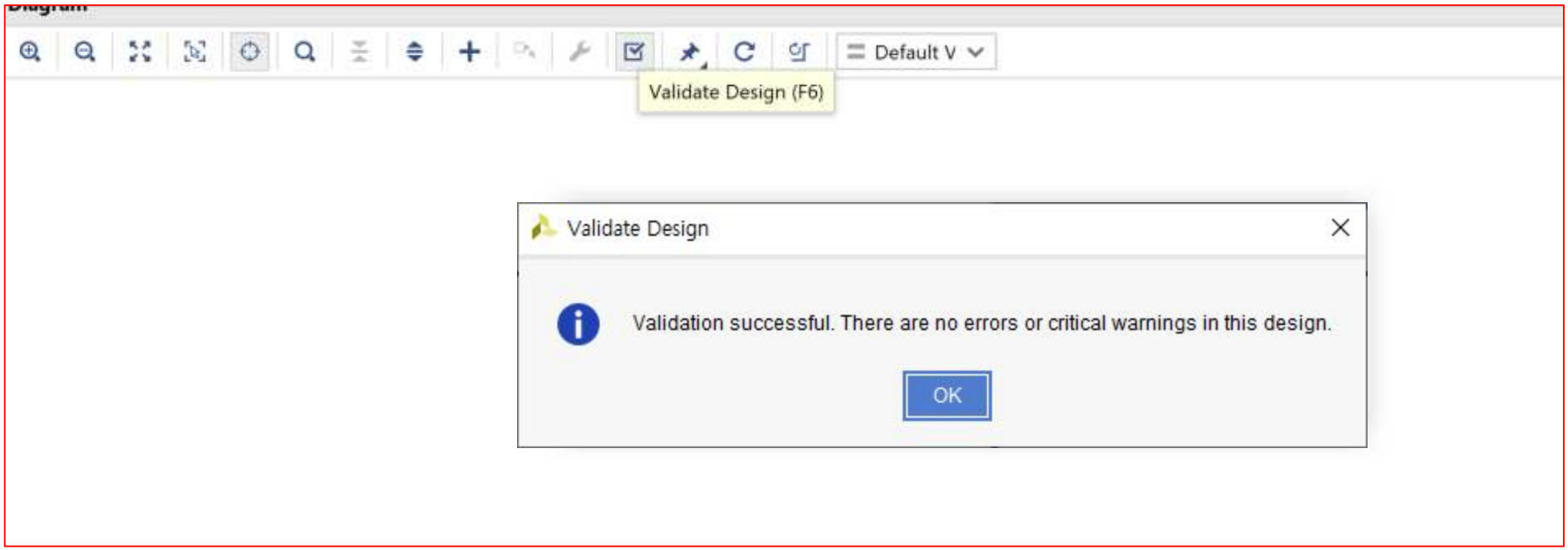


LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- **Validate Design and Regenerate Layout**

LED_Counter Implementation

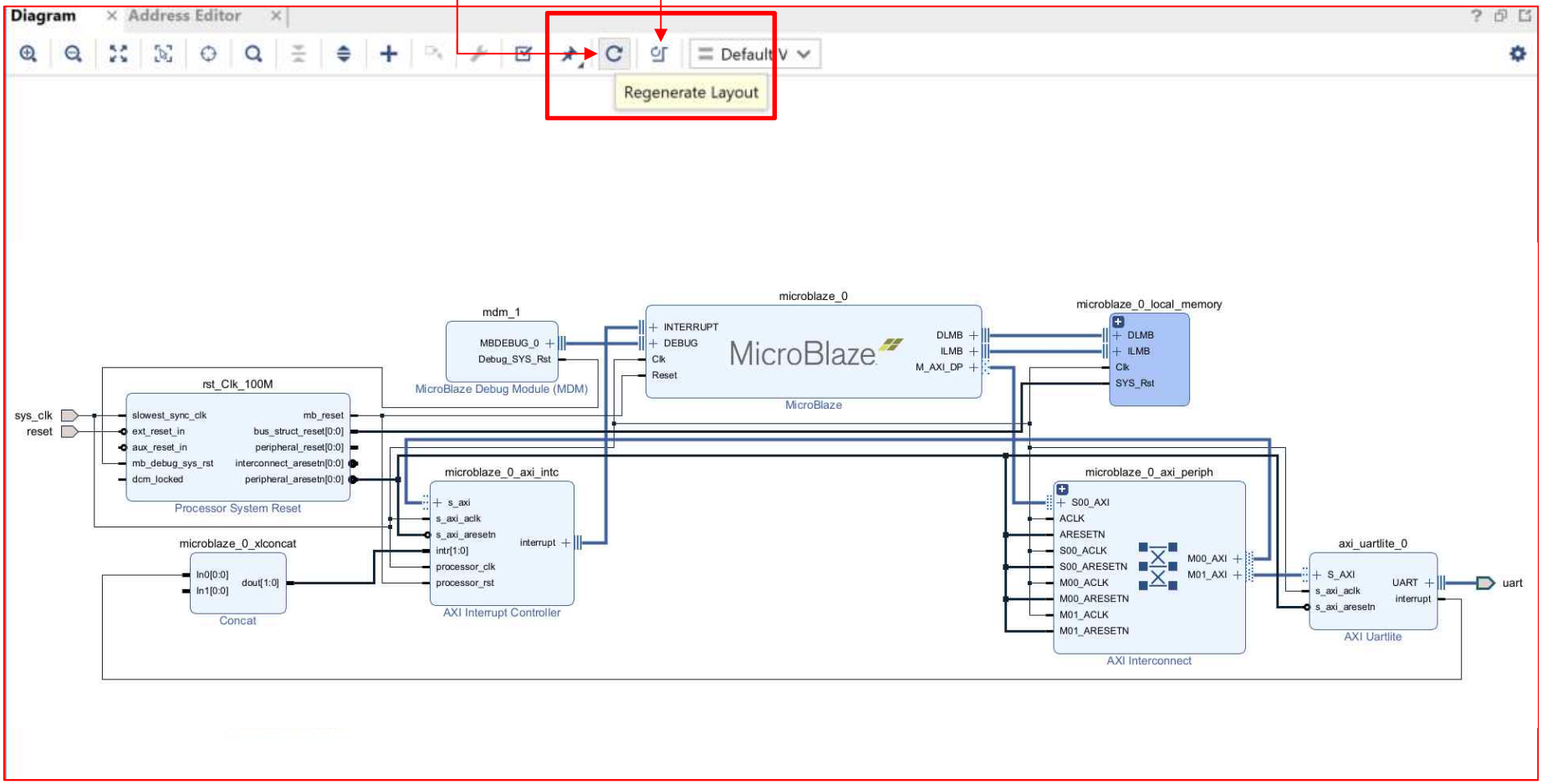
- *LED_Counter Implementation*
  - **Block Design**이 완료
  - ***Validate Design*** [단축키: F6]
  - **Block Design**이 완료되면 에러가 없는지 확인 ➔ 상단 아이콘 중에 **Validate Design ( )** 클릭
  - 에러가 없으면 아래와 같은 윈도가 나타남
  - 에러가 발생하면 관련 설정을 검토하여 에러를 수정하고 다시 에러를 확인



LED_Counter Implementation

➢ *LED_Counter Implementation*

- 배치 및 정리 ➔ ***Regenerate Layout [or Optimize Routing]***클릭

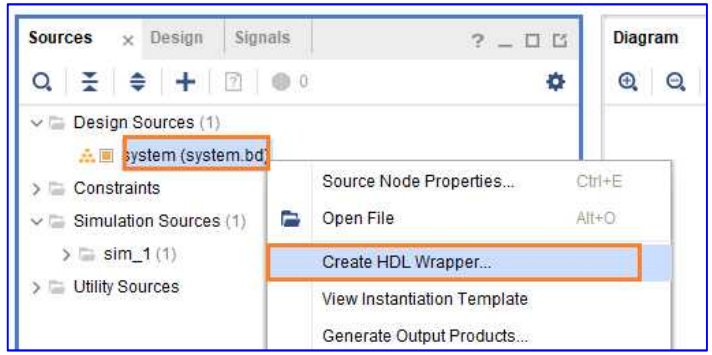


LED_Counter Implementation

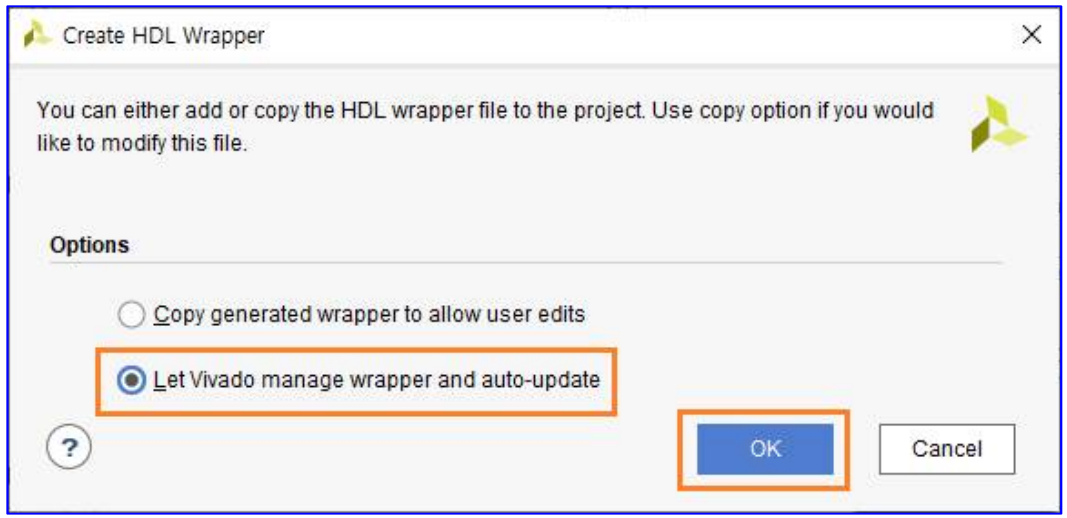
- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- **Create HDL Wrapper**

LED_Counter Implementation

- ***Create HDL Wrapper***
  - ***Block Design*** 완료
  - 코드로 변환 ➔ [***Design Sources – system*** ➔ 우클릭 ➔ ***Create HDL Wrapper...*** 를 클릭]



  - "***Let Vivado manage wrapper and auto-update***" 선택 ➔ OK를 클릭

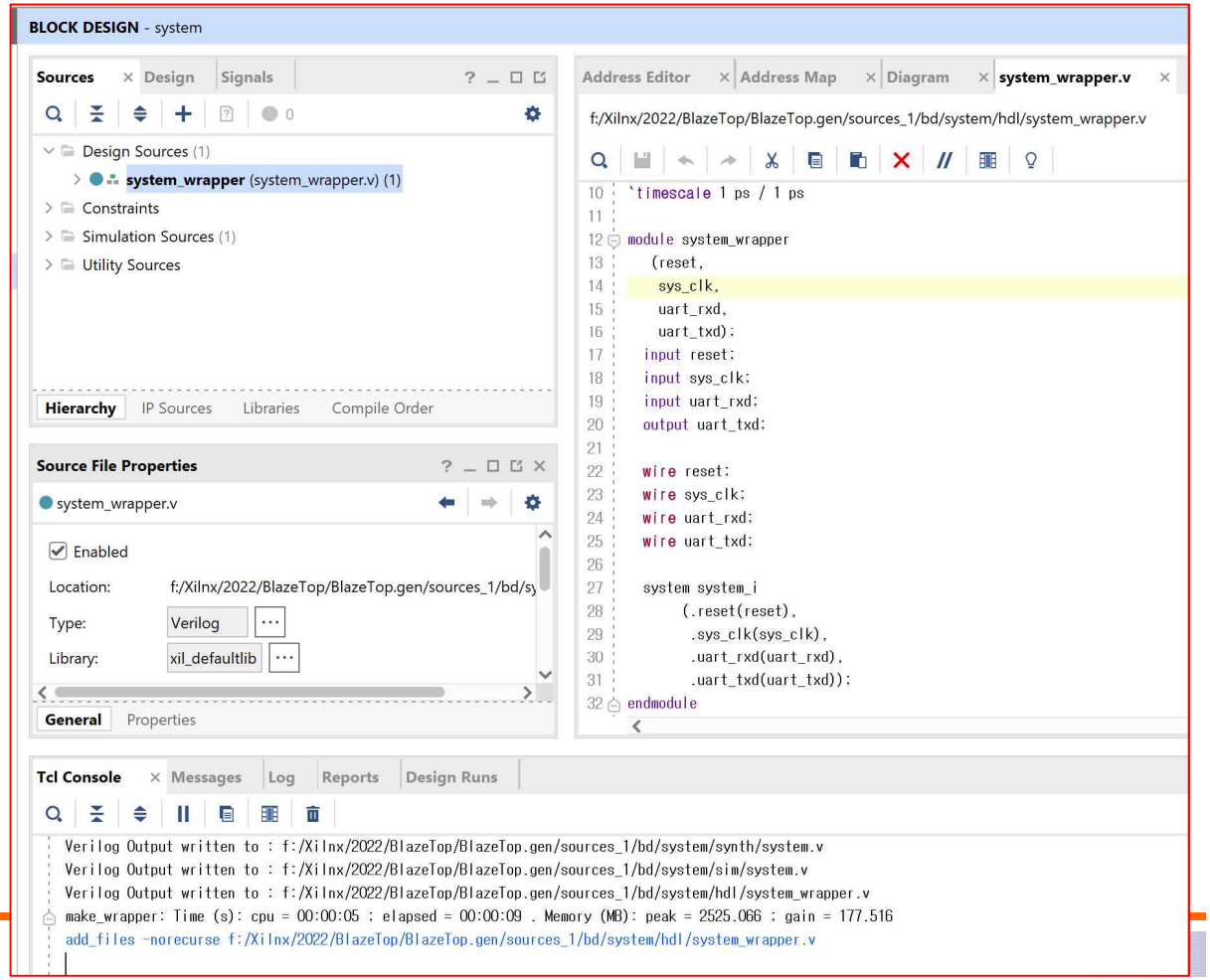


LED_Counter Implementation

➢ *Create HDL Wrapper*

- *system_wrapper.v* 파일이 생성

- *system_wrapper.v* 파일 ➔ 사용자가 추가한 포트 (*reset, sys_clk, uart_rxd, uart_txd*) 가 있고, 내부에 *Block Design*에서 구현한 *system* 모듈

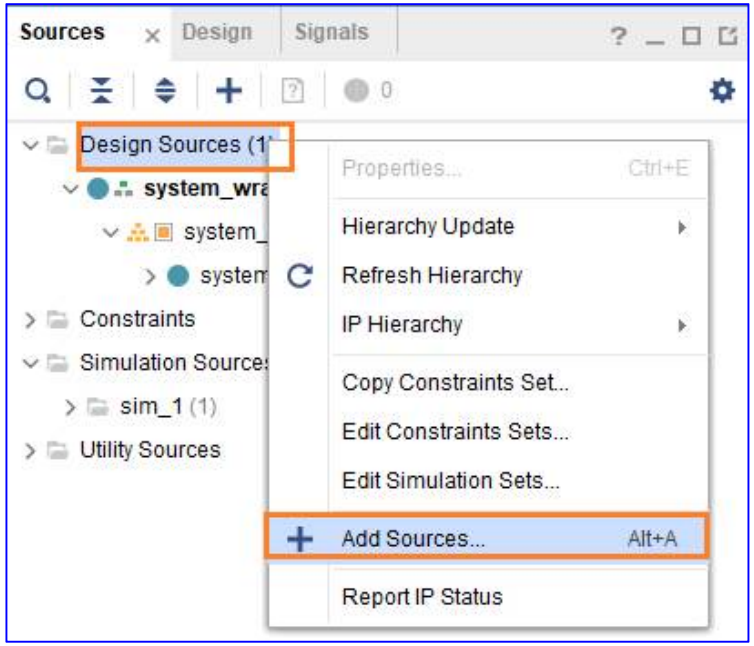


LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- Create HDL Wrapper
- User Logic Implementation → Led_On/Off_Logic → led_control

LED_Counter Implementation

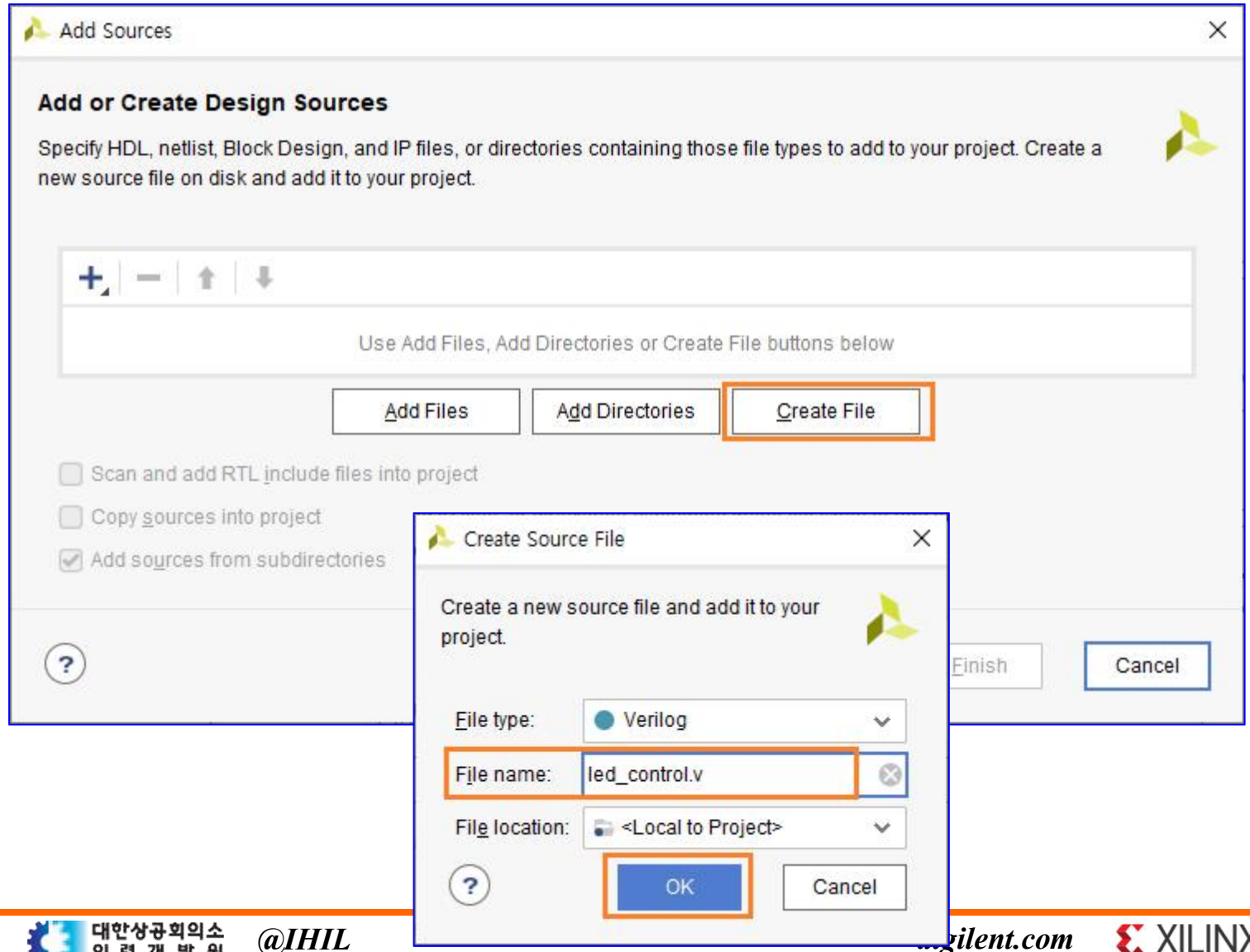
- *User Logic Implementation*
  - *Led*를 *On/Off* 하는 *Logic* 구현
  - *Design Sources* → 우클릭 → *Add Sources...* 를 클릭



- "Add or create design sources"을 선택하고 Next를 클릭

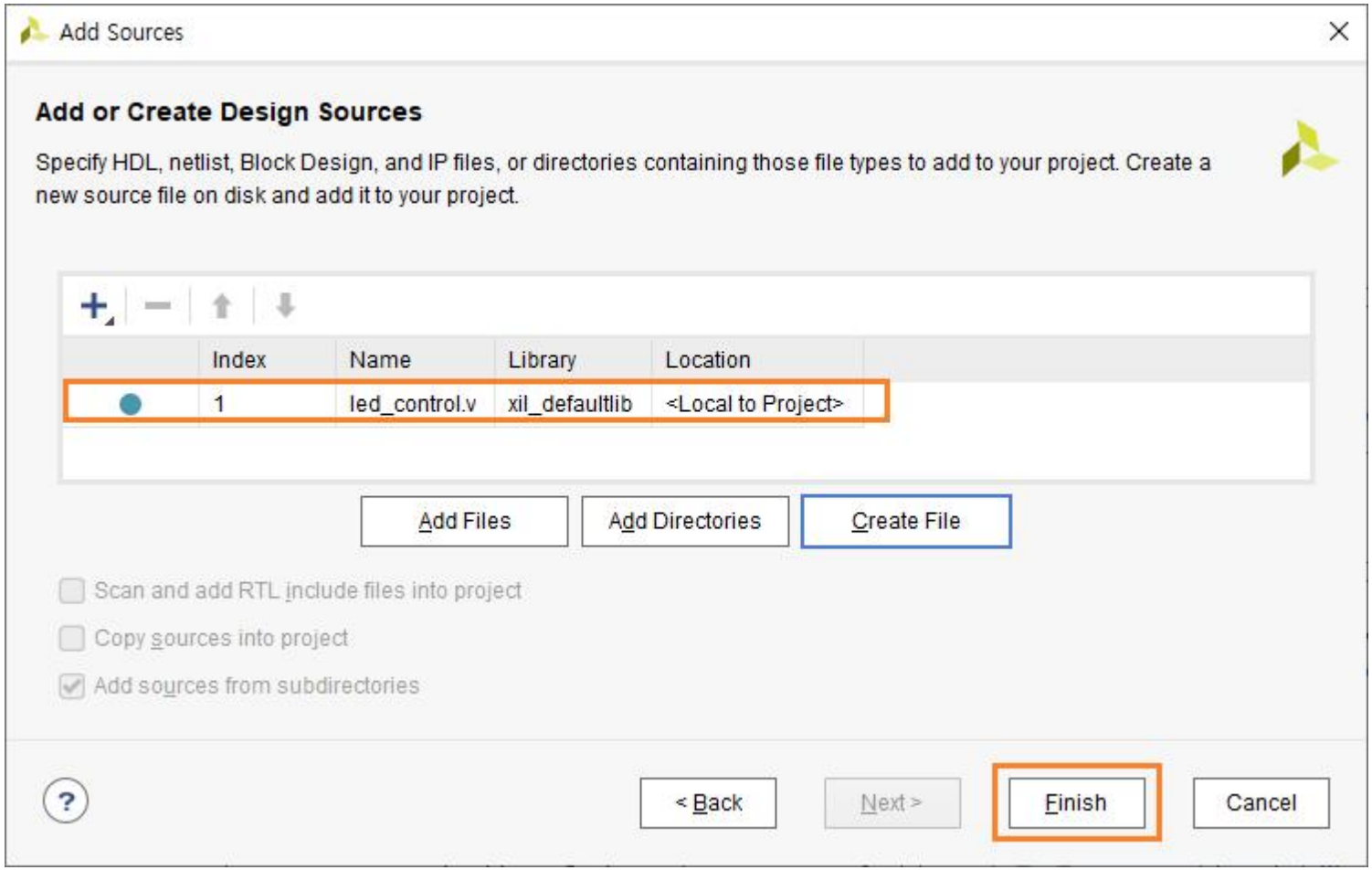
LED_Counter Implementation

- *User Logic Implementation*
  - *Create File* 클릭 ➔ *Create Source File* 윈도 ➔ ***File name : led_control.v*** 입력 ➔ OK 클릭



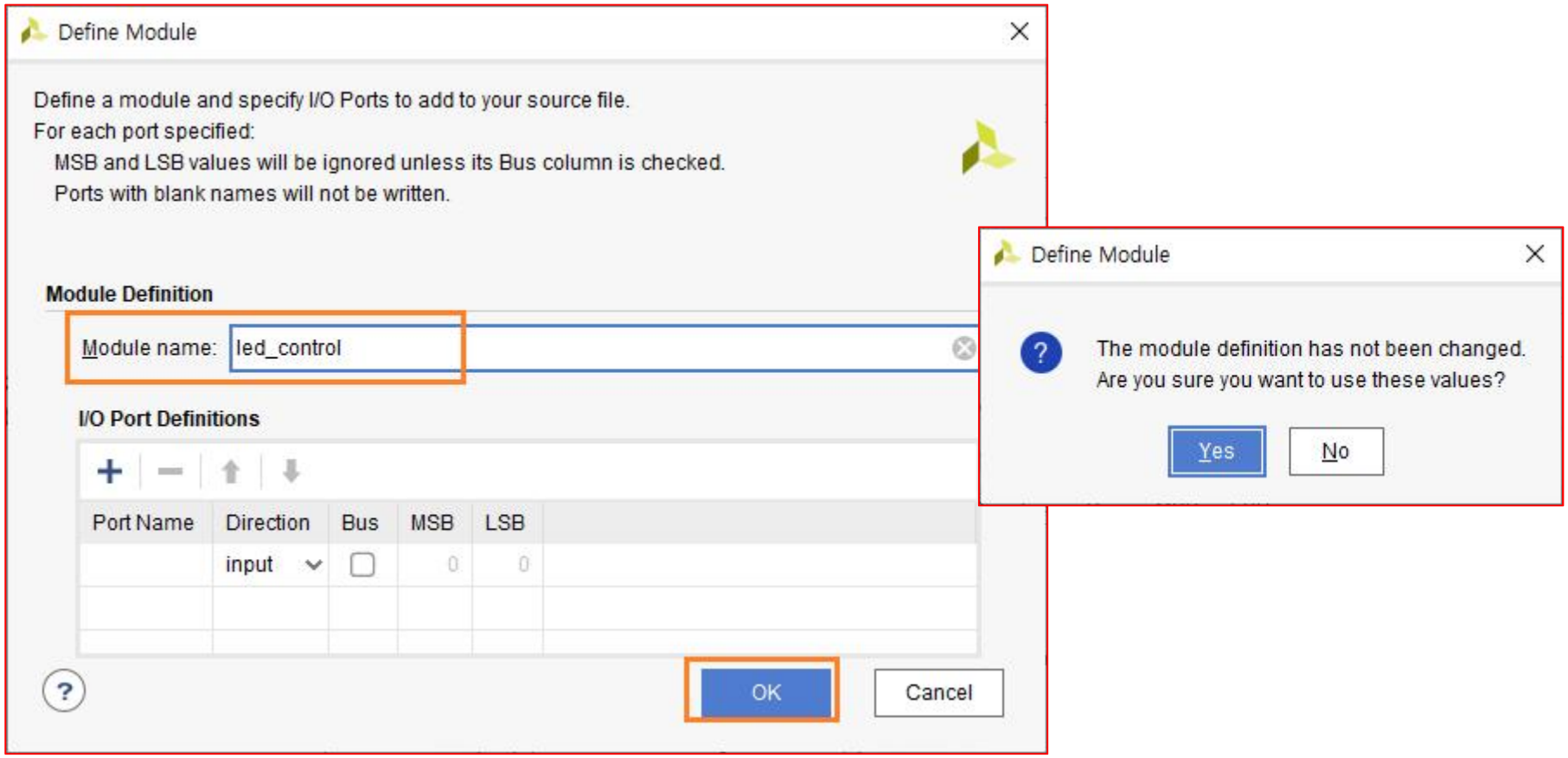
LED_Counter Implementation

- ***User Logic Implementation***
  - ***led_control.v*** 파일 추가 ➔ **Finish**를 클릭합니다. (이미 생성된 소스 파일을 프로젝트에 추가하려면 **Add Files** 버튼을 클릭하여 해당 파일을 추가)



LED_Counter Implementation

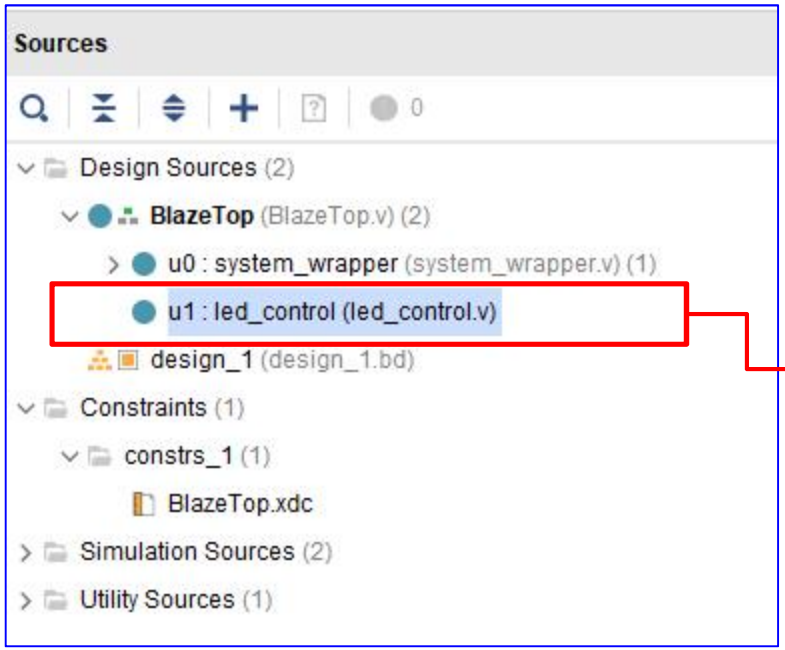
- ***User Logic Implementation***
  - ***Define Module*** 윈도에서는 모듈 이름과 핀들을 설정 ➔ 기본값을 사용, **OK** 클릭 ➔ **YES** 클릭



LED_Counter Implementation

➢ *User Logic Implementation*

- *led_control module* 이 추가된 *Design Source*



- *led_control.v* 파일을 열어 *Code* 추가
- *100Mhz Clock*을 이용하여 *0.5*초마다 *4*개의 *LED*를 순차적으로 **On/Off** 하는 코드

Project Summary | led_control.v

H:/Xilnx/2022/BlazeTop/BlazeTop.srcs/sources_1/new/led_control.v

```
1  `timescale 1ns / 1ps
2
3  module led_control(
4      reset,
5      clock,
6      led
7      );
8
9      input   reset;
10     input   clock;
11     output  [3:0] led ;
12
13     reg     [25:0]   cnt ;
14
15     always @ (posedge clock or negedge reset)
16     begin
17        if(~reset)  cnt <= 26'd0;
18        else        cnt <= (cnt==26'd50_000_000)  ? 26'd0 : cnt+1'b1;
19
20     end
21
22     reg  [3:0]   led;
23
24     always@(posedge clock or negedge reset)
25     begin
26        if(~reset)  led <= 4'd0;
27        else        led <= (cnt==26'd50_000_000)  ? led+1'b1 : led ;
28
29     end
30 endmodule
31
```

LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- Create HDL Wrapper
- User Logic Implementation → Led_On/Off_Logic → led_control
- **User Logic Implementation → Led_On/Off_Logic → BlazeTop**

LED_Counter Implementatio

- ***User Logic Implementation***
  - ***Design Source***에 ***led_counter, system_wrapper*** 모듈을 포함하는 ***Top*** 모듈을 추가
  - ***led_control*** 모듈을 추가할 때와 동일한 방법으로 ***BlazeTop.v*** 모듈을 추가하고, 아래와 같이 코드를 추가

Address Editor | Address Map | Diagram | system_wrapper.v

f:/Xilnx/2022/BlazeTop/BlazeTop.gen/sources_1/bd/system/hdl/system_wrapper.v

```
10  `timescale 1 ps / 1 ps
11
12  module system_wrapper
13     (reset,
14      sys_clk,
15      uart_rxd,
16      uart_txd);
17    input reset;
18    input sys_clk;
19    input uart_rxd;
20    output uart_txd;
21
22    wire reset;
23    wire sys_clk;
24    wire uart_rxd;
25    wire uart_txd;
26
27    system system_i
28         (.reset(reset),
29          .sys_clk(sys_clk),
30          .uart_rxd(uart_rxd),
31          .uart_txd(uart_txd));
32  endmodule
```

Project Summary | led_control.v

H:/Xilnx/2022/BlazeTop/BlazeTop.srcs/sources_1/new/led_control.v

```
1   `timescale 1ns / 1ps
2
3   module led_control(
4       reset,
5       clock,
6       led
7       );
8
9       input  reset;
10      input  clock;
11      output [3:0] led ;
12
13      reg    [25:0]   cnt ;
14
15      always @ (posedge clock or negedge reset)
16      begin
17        if(~reset) cnt <= 26'd0;
18        else       cnt <= (cnt==26'd50_000_000) ? 26'd0 : cnt+1'b1;
19
20      end
21
22      reg [3:0]  led;
23
24      always@(posedge clock or negedge reset)
25      begin
26        if(~reset) led <= 4'd0;
27        else       led <= (cnt==26'd50_000_000) ? led+1'b1 : led ;
28
29      end
30  endmodule
31
```

Project Summary | led_control.v | BlazeTop.v

H:/Xilnx/2022/BlazeTop/BlazeTop.srcs/sources_1/new/BlazeTop.v

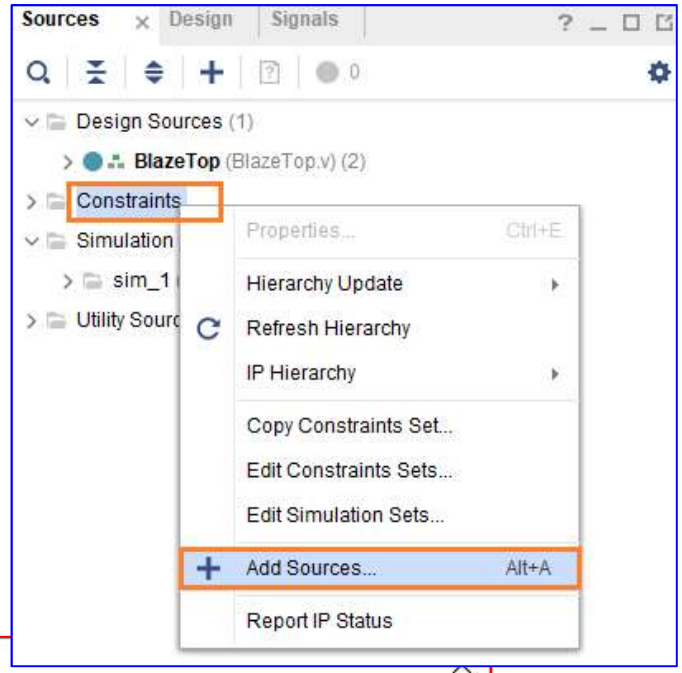
```
1   `timescale 1ns / 1ps
2
3   module BlazeTop(
4       reset,
5       sys_clk,
6       uart_rxd,
7       uart_txd,
8       led
9       );
10
11      input       reset ;
12      input       sys_clk ;
13      input       uart_rxd ;
14      output      uart_txd ;
15      output [3:0]  led ;
16
17      system_wrapper  u0(
18            .reset(reset),
19            .sys_clk(sys_clk),
20            .uart_rxd(uart_rxd),
21            .uart_txd(uart_txd)
22      );
23
24      wire    [3:0]  led;
25
26      led_control u1(
27            .reset(reset),
28            .clock(sys_clk),
29            .led(led)
30            );
31  endmodule
```

LED_Counter Implementation

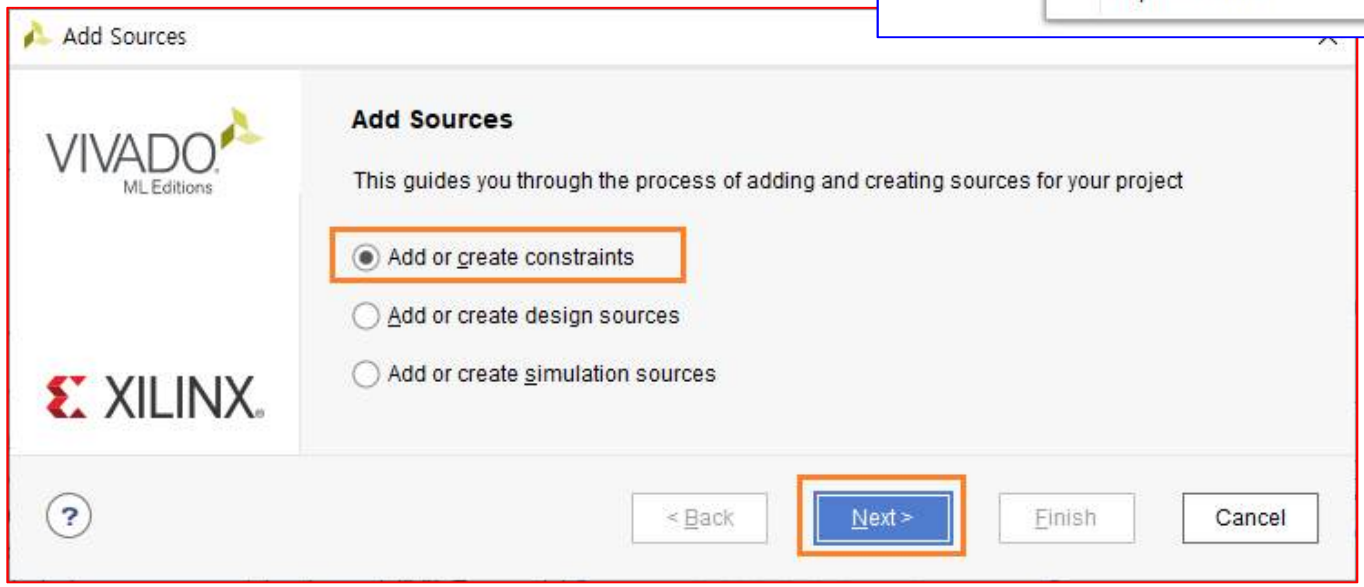
- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- Create HDL Wrapper
- User Logic Implementation → Led_On/Off_Logic → led_control
- User Logic Implementation → Led_On/Off_Logic → BlazeTop
- **XDC Implementation**

LED_Counter Implementation

- *XDC Implementation*
  - *Constrains* → 우클릭 → *Add Sources...*를 클릭



  - *"Add or Create constraints"*을 선택하고 **Next**를 클릭
  - *File name : BlazeTop.xdc*로 입력해서 파일을 추가



LED_Counter Implementation

- ***XDC Implementation***
  - ***BlazeTop.xdc*** 에 아래와 같이 코드를 추가(1/2)

BlazeTop.xdc

F:/Xilnx/2022/BlazeTop/BlazeTop.srcs/constrs_1/new/BlazeTop.xdc

```
1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the project
5
6  ## Clock signal
7  set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports sys_clk]
8  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports sys_clk]
9
10 ## Switches
11 #set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports {sw[0]}]
12 #set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports {sw[1]}]
13 #set_property -dict { PACKAGE_PIN W16   IOSTANDARD LVCMOS33 } [get_ports {sw[2]}]
14 #set_property -dict { PACKAGE_PIN W17   IOSTANDARD LVCMOS33 } [get_ports {sw[3]}]
15 #set_property -dict { PACKAGE_PIN W15   IOSTANDARD LVCMOS33 } [get_ports {sw[4]}]
16 #set_property -dict { PACKAGE_PIN V15   IOSTANDARD LVCMOS33 } [get_ports {sw[5]}]
17 #set_property -dict { PACKAGE_PIN W14   IOSTANDARD LVCMOS33 } [get_ports {sw[6]}]
18 #set_property -dict { PACKAGE_PIN W13   IOSTANDARD LVCMOS33 } [get_ports {sw[7]}]
19 #set_property -dict { PACKAGE_PIN V2    IOSTANDARD LVCMOS33 } [get_ports {sw[8]}]
20 #set_property -dict { PACKAGE_PIN T3    IOSTANDARD LVCMOS33 } [get_ports {sw[9]}]
21 #set_property -dict { PACKAGE_PIN T2    IOSTANDARD LVCMOS33 } [get_ports {sw[10]}]
22 #set_property -dict { PACKAGE_PIN R3    IOSTANDARD LVCMOS33 } [get_ports {sw[11]}]
23 #set_property -dict { PACKAGE_PIN W2    IOSTANDARD LVCMOS33 } [get_ports {sw[12]}]
24 #set_property -dict { PACKAGE_PIN U1    IOSTANDARD LVCMOS33 } [get_ports {sw[13]}]
25 #set_property -dict { PACKAGE_PIN T1    IOSTANDARD LVCMOS33 } [get_ports {sw[14]}]
26 set_property -dict { PACKAGE_PIN R2    IOSTANDARD LVCMOS33 } [get_ports {reset}]
27
```

LED_Counter Implementation

- *XDC Implementation*
  - *BlazeTop.xdc* 에 아래와 같이 코드를 추가(2/2)

```
28
29  ## LEDs
30  set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports {led[0]}]
31  set_property -dict { PACKAGE_PIN E19   IOSTANDARD LVCMOS33 } [get_ports {led[1]}]
32  set_property -dict { PACKAGE_PIN U19   IOSTANDARD LVCMOS33 } [get_ports {led[2]}]
33  set_property -dict { PACKAGE_PIN V19   IOSTANDARD LVCMOS33 } [get_ports {led[3]}]
34  #set_property -dict { PACKAGE_PIN W18   IOSTANDARD LVCMOS33 } [get_ports {led[4]}]
35  #set_property -dict { PACKAGE_PIN U15   IOSTANDARD LVCMOS33 } [get_ports {led[5]}]
36  #set_property -dict { PACKAGE_PIN U14   IOSTANDARD LVCMOS33 } [get_ports {led[6]}]
37  #set_property -dict { PACKAGE_PIN V14   IOSTANDARD LVCMOS33 } [get_ports {led[7]}]
38  #set_property -dict { PACKAGE_PIN V13   IOSTANDARD LVCMOS33 } [get_ports {led[8]}]
```

```
130
131 ##USB-RS232 Interface
132 set_property -dict { PACKAGE_PIN B18   IOSTANDARD LVCMOS33 } [get_ports uart_rxd]
133 set_property -dict { PACKAGE_PIN A18   IOSTANDARD LVCMOS33 } [get_ports uart_txd]
134
```

```
150
151 ## Configuration options, can be used for all designs
152 set_property CONFIG_VOLTAGE 3.3 [current_design]
153 set_property CFGBVS VCCO [current_design]
154
155 ## SPI configuration mode options for QSPI boot, can be used for all designs
156 set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
157 set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
158 set_property CONFIG_MODE SPIx4 [current_design]
159
```

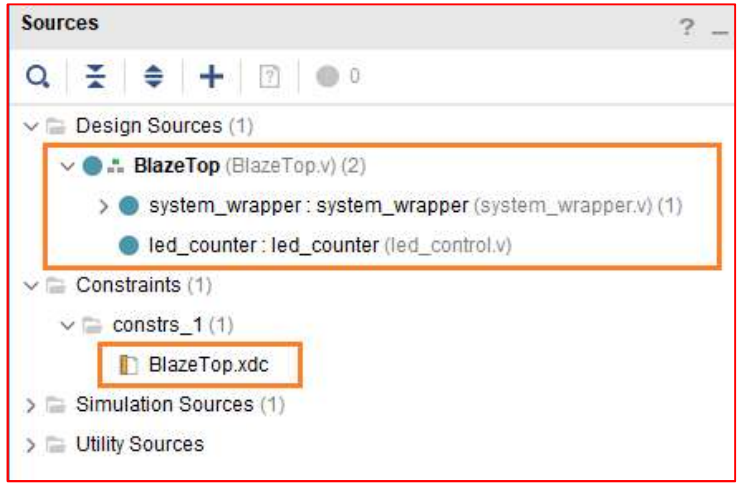
LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- Create HDL Wrapper
- User Logic Implementation → Led_On/Off_Logic → led_control
- User Logic Implementation → Led_On/Off_Logic → BlazeTop
- XDC Implementation
- **Generate Bitstream**

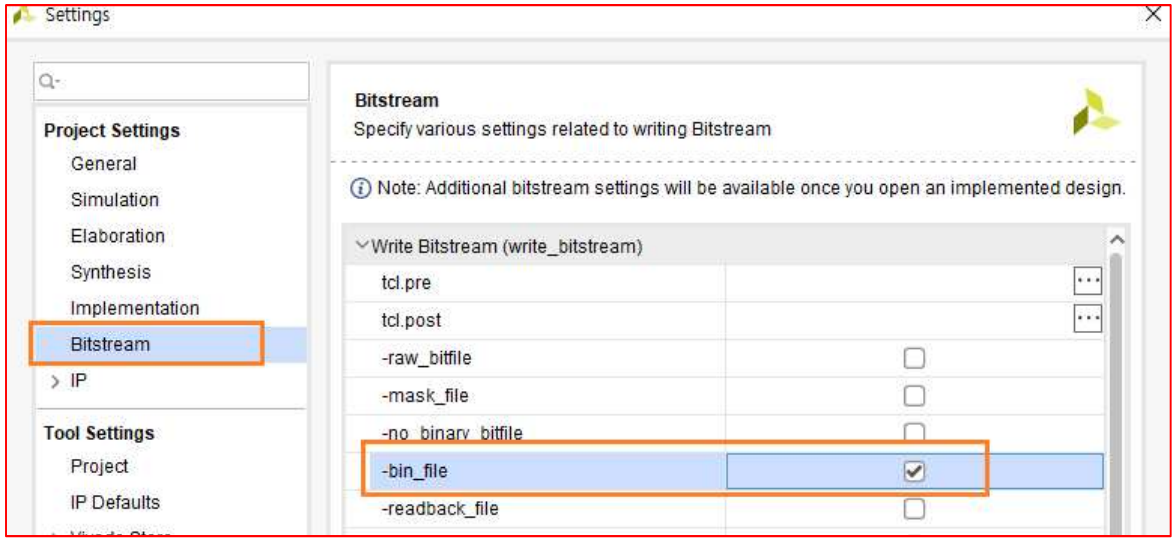
LED_Counter Implementation

➢ *Generate Bitstream*

- 구현된 *Source* 구조 ➔ *BlazeTop* 모듈이 Top 모듈로 설정



- *Project Settings* 에서 *Bitstream* 생성시 *bin*파일을 생성하도록 설정 ➔ *PROJECT MANAGER* ➔ *Settings* 을 클릭 ➔ *Settings* 윈도에서 *Bitstream*을 클릭하고 [*-bin_file*]항목을 check 하고 *Apply* ➔ *OK*를 클릭



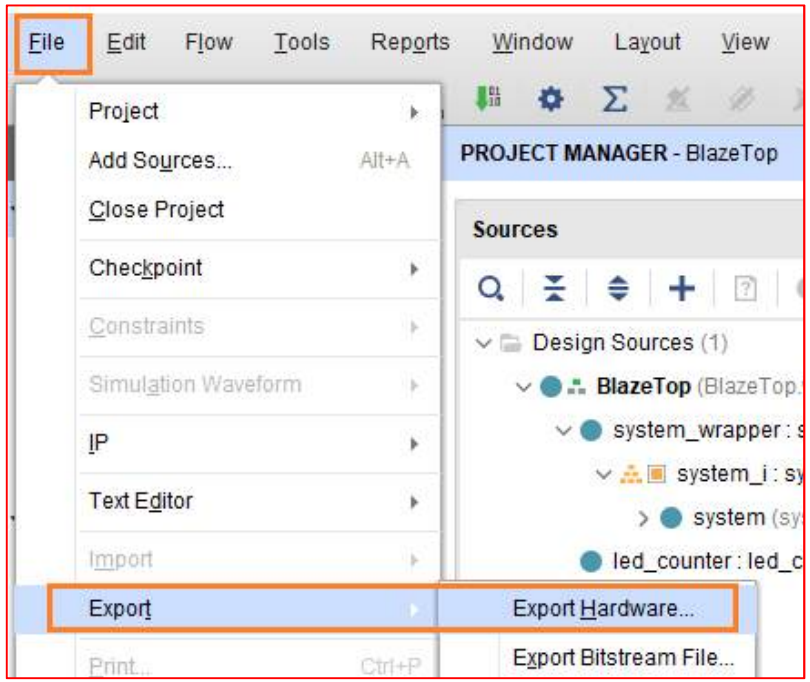
- *PROMGRAM AND DEBUG – Generate Bitstream*을 클릭해서 *Bitstream*을 생성
- 완료 되면 *Cancel* 버튼을 클릭

LED_Counter Implementation

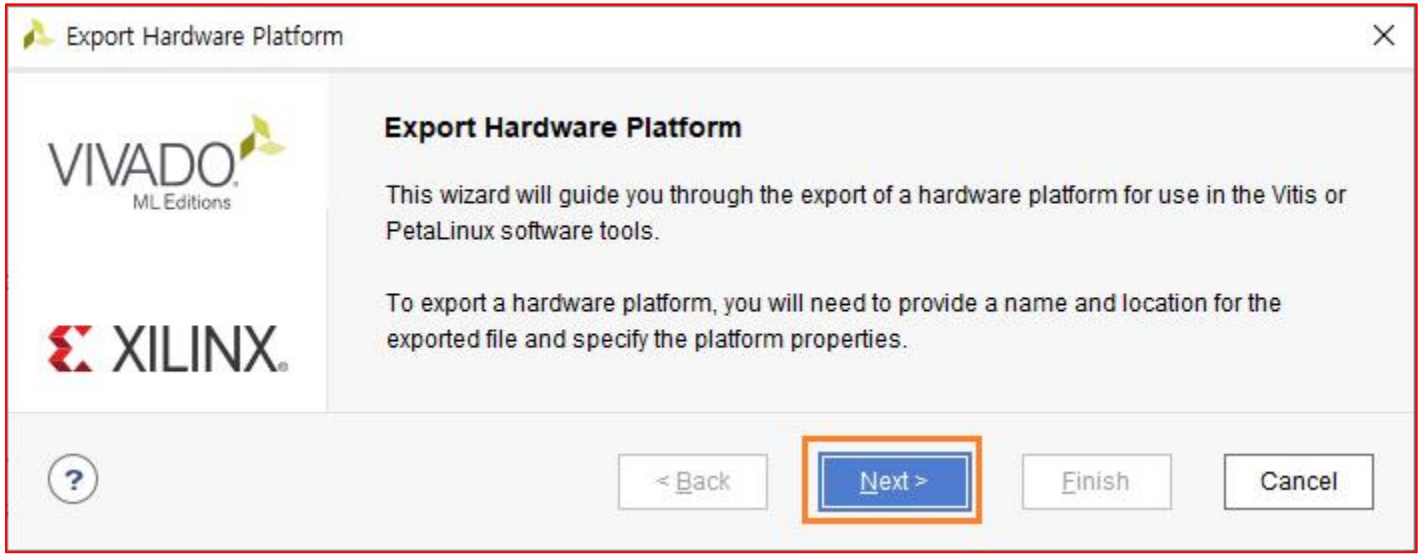
- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- Create HDL Wrapper
- User Logic Implementation → Led_On/Off_Logic → led_control
- User Logic Implementation → Led_On/Off_Logic → BlazeTop
- XDC Implementation
- Generate Bitstream
- **Export Hardware**

LED_Counter Implementation

- *Export Hardware*
  - 생성된 로직을 *Vitis (Embedded sw 개발 tool)*에서 사용하기 위해서는 *Export Hardware*
  - 메뉴에서 [*File –Export – Export Hardware*]를 클릭

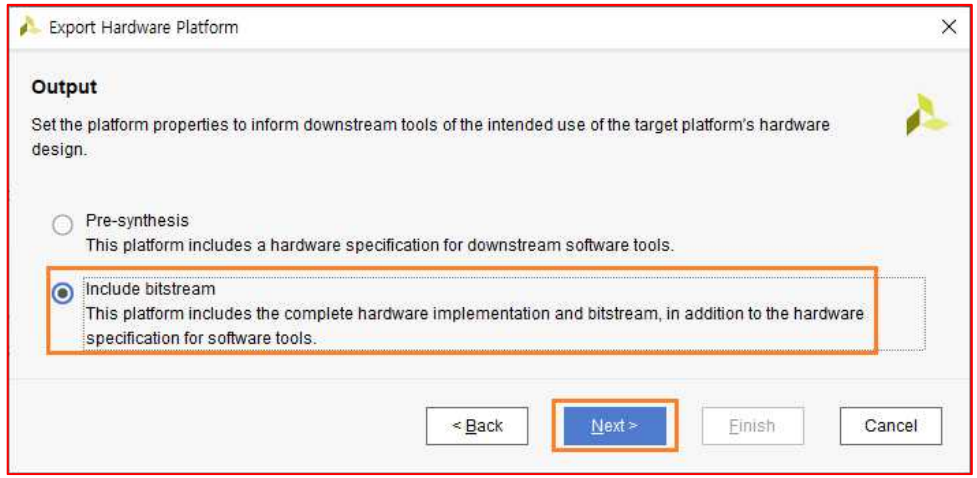


- **Next를 클릭**

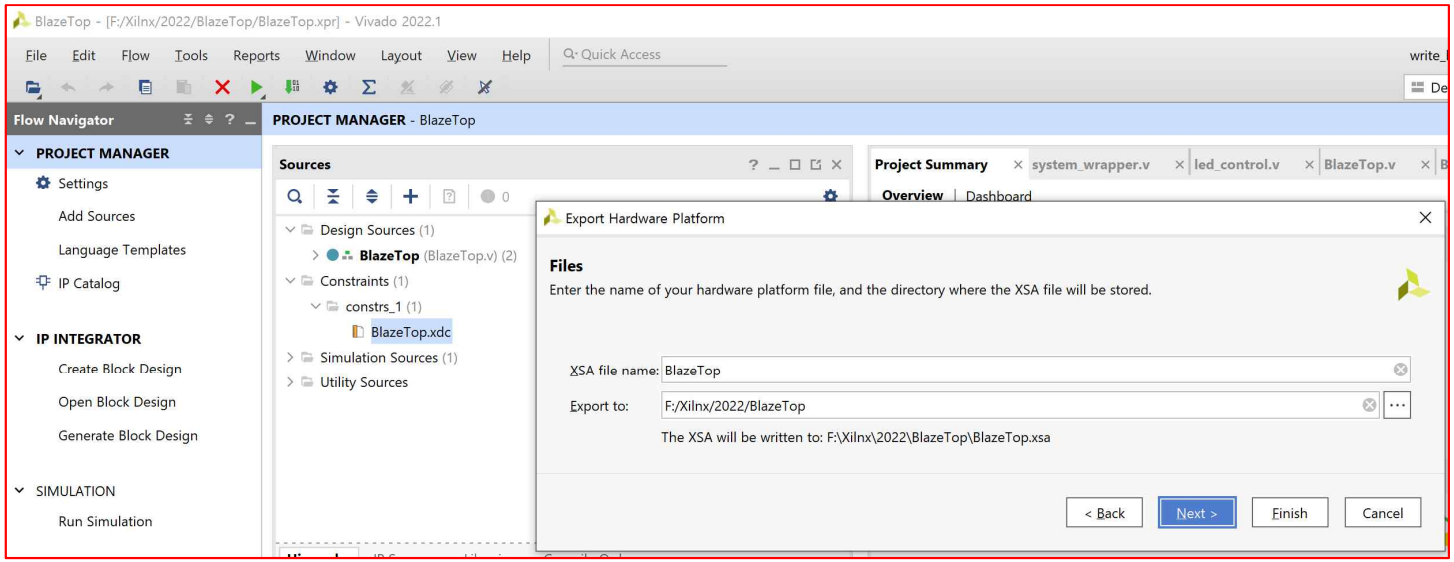


LED_Counter Implementation

- ***Export Hardware***
  - "*Include bitstream*"을 선택하고 Next를 클릭

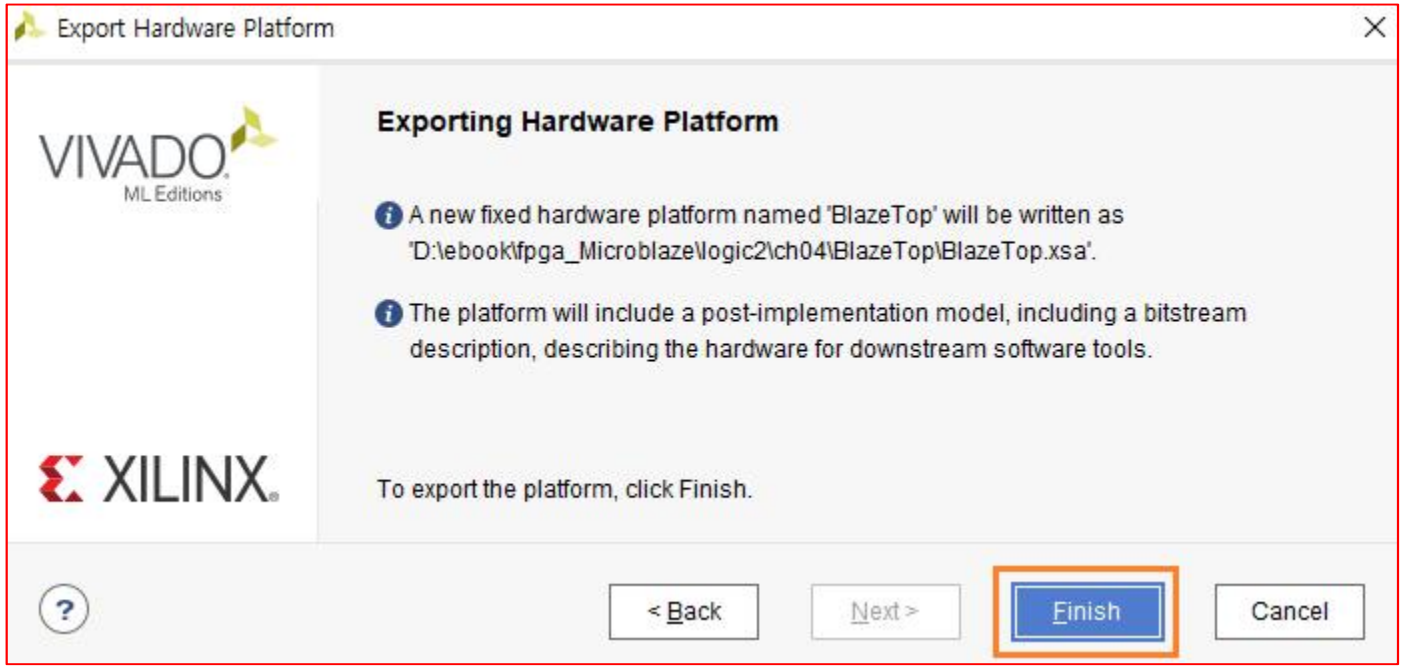


  - 생성되는 파일은 *xsa* 확장자를 가진 파일
  - 파일명과 경로를 확인하고 *Next*를 클릭



LED_Counter Implementation

- *Export Hardware*
  - *Finish*를 클릭하면 *XSA* 파일이 생성



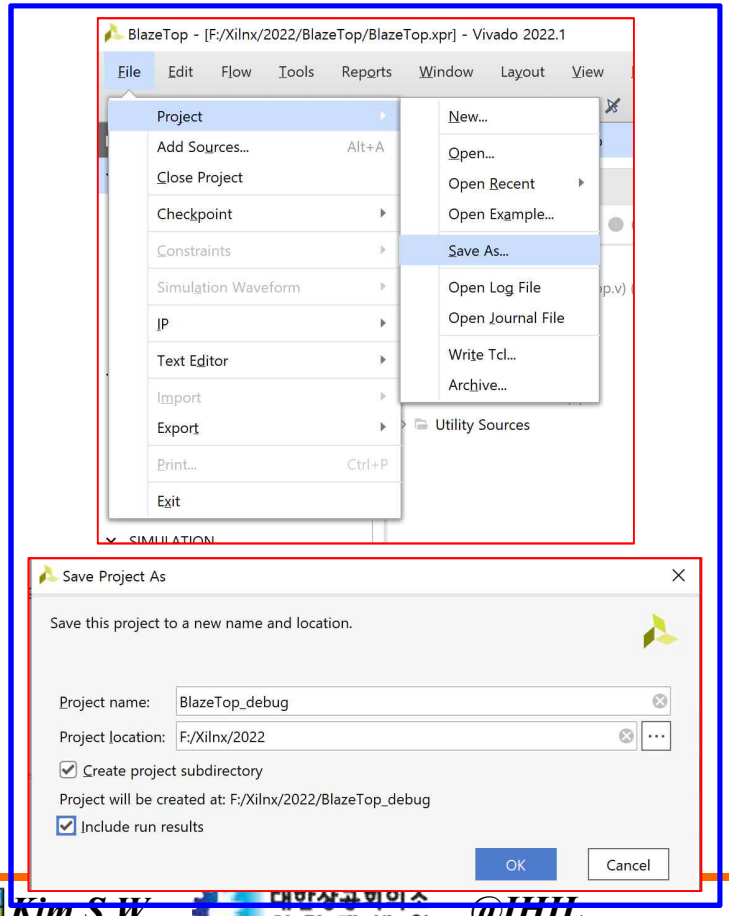
- 기본 *Template* 이 완료 ➔ 다른 응용에서 기본 *Template* 폴더를 복사하여 이름을 변경하여 사용
- [*Next*] *UART*를 통하여 *Hello world Message*를 출력하고, *LED*를 *On/Off* 하는 것을 구현
- [참고] *Vivado*는 폴더의 경로를 바꾸어서 사용해도 별 문제가 되지 않지만, *Vitis*는 프로젝트를 생성하고 폴더(경로나 이름)를 변경하면 인식을 못하는 문제 ➔ 이때, 프로젝트 환경 파일을 수동으로 수정해서 사용하면 되지만 번거로우므로 , *Vitis*를 진행하기 전에 기본 *Template*를 복사해서 사용하면 편리

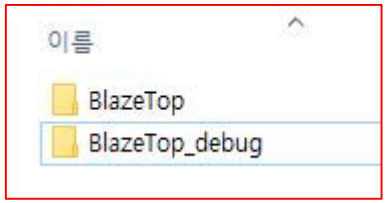
LED_Counter Implementation

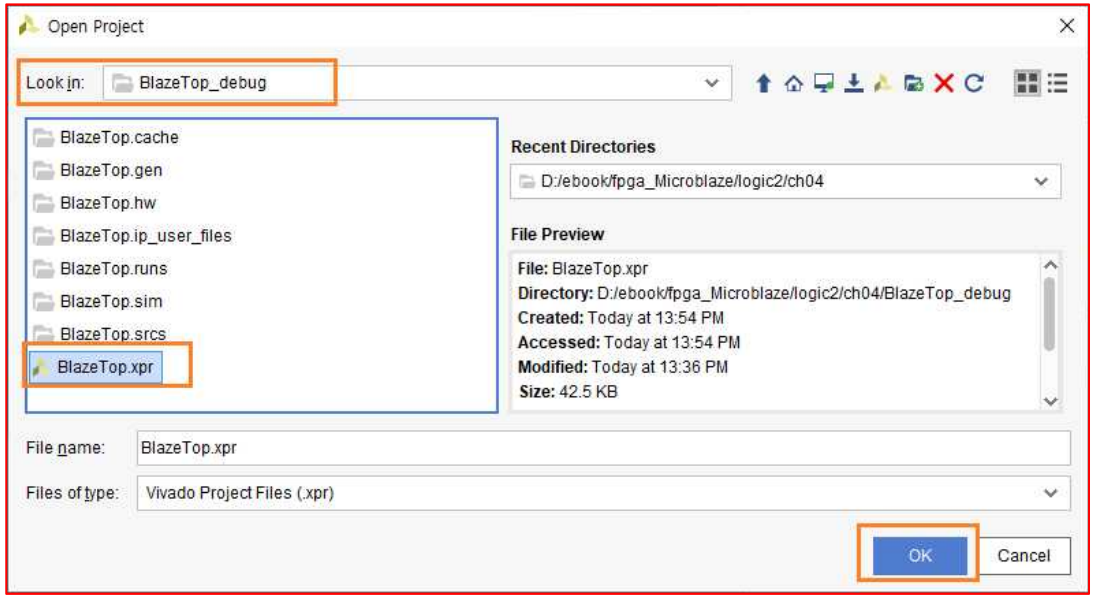
- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- Create HDL Wrapper
- User Logic Implementation → Led_On/Off_Logic → led_control
- User Logic Implementation → Led_On/Off_Logic → BlazeTop
- XDC Implementation
- Generate Bitstream
- Export Hardware
- **SW Implementation → Launch Vitis IDE & Create Application Project**

LED_Counter Implementation

- ***SW Implementation (Skip 가능)***
  - ***Vitis***를 이용하여 ***SW***를 구현 ➔ ***Hello world Implementation*** 및 ***LED On/Off***
  - 먼저 앞에서 생성했던 폴더를 복사하여 이름을 [***BlazeTop_debug***]로 변경
  - **BlazeTop** 폴더는 앞에서 생성한 기본 **Template** 폴더 ***BlazeTop_debug***는 ***BlazeTop*** 폴더를 복사한 후 이름을 변경
  - **Vivado**에서 **BlazeTop Project**를 닫고[***File – Close Project***], ***Open Project***를 클릭 ➔ ***BlazeTop_debug*** 폴더에 있는 ***BlazeTop.xpr*** 파일을 ***Open***



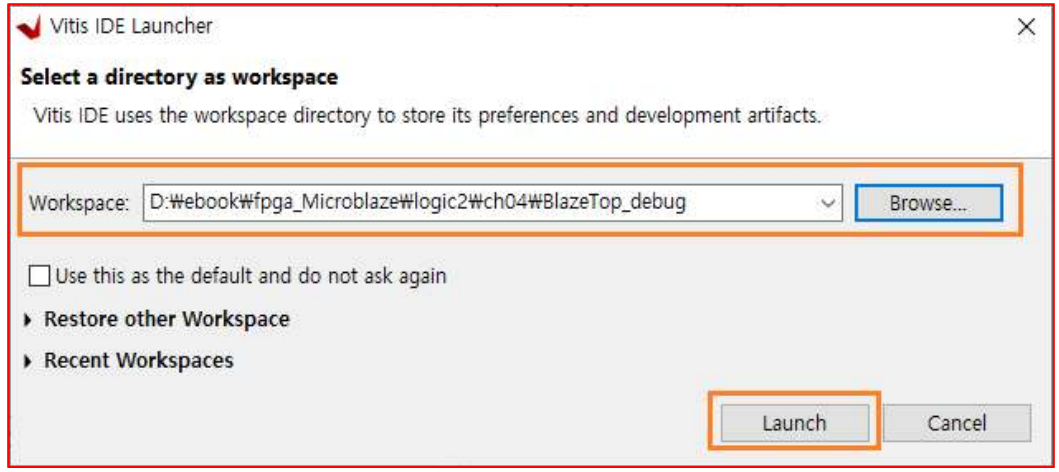




LED_Counter Implementation

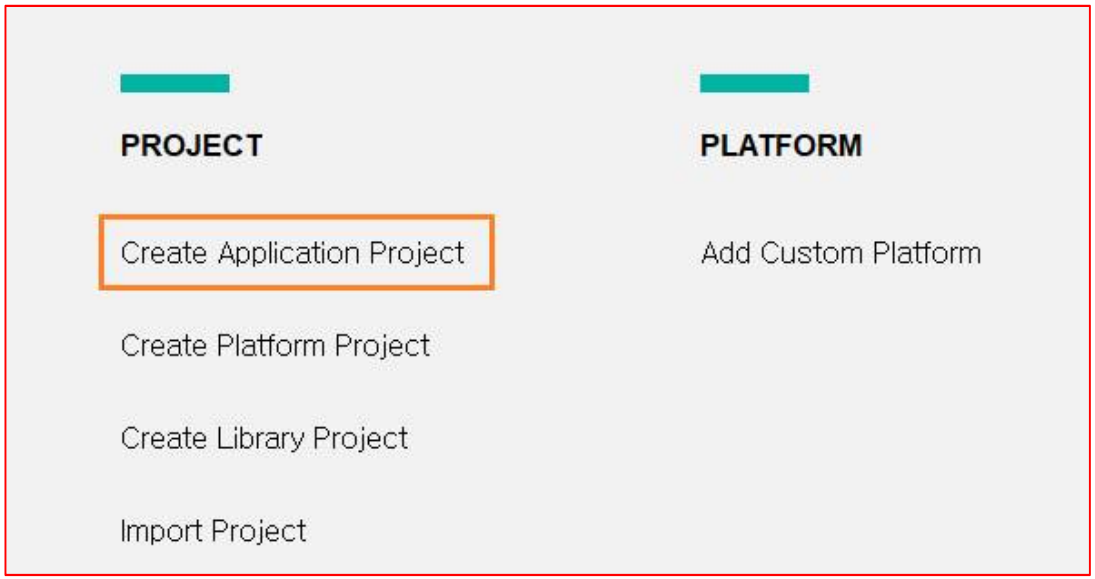
➢ *SW Implementation*

- *XSA* 파일까지 생성하였기 때문에 바로 *Vitis*에서 진행 가능 ➔ *Tools → Launch Vitis IDE*를 클릭하면 *Vitis*가 실행



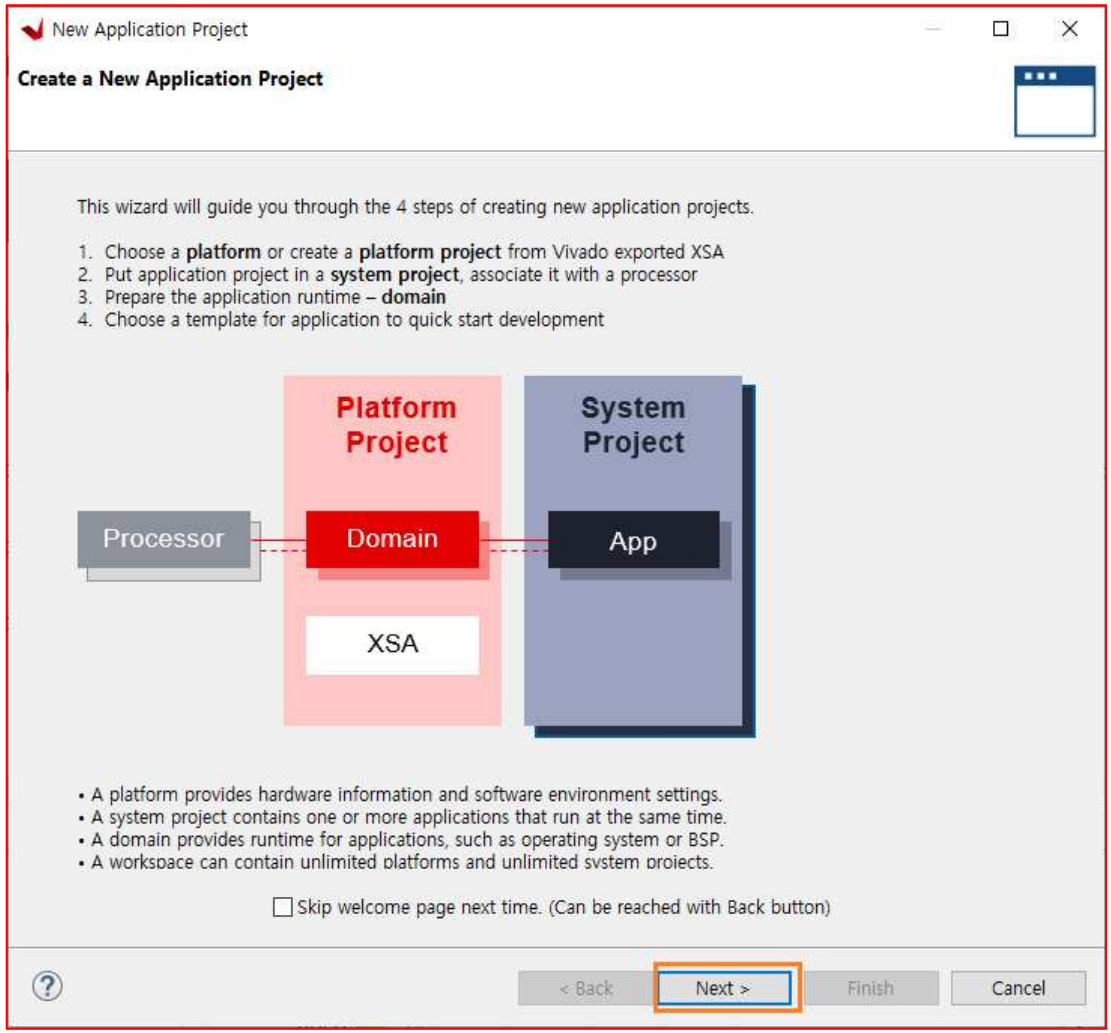
- *Workspace*가 *BlazeTop_debug* 를 확인하고 *Launch*를 클릭 *Workspace* 위치가 다르면 *Browse...* 클릭해서 해당 위치를 변경)

- 새로운 프로젝트를 생성하기 위하여 *Create Application Project* ➔ *Click*



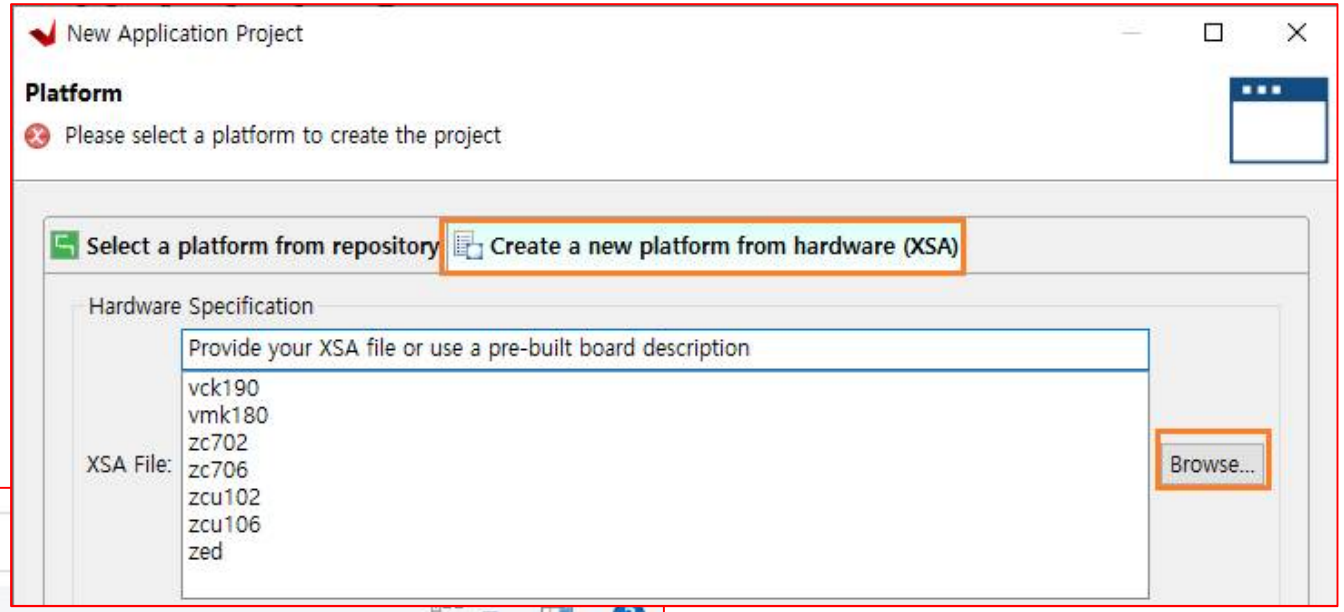
LED_Counter Implementation

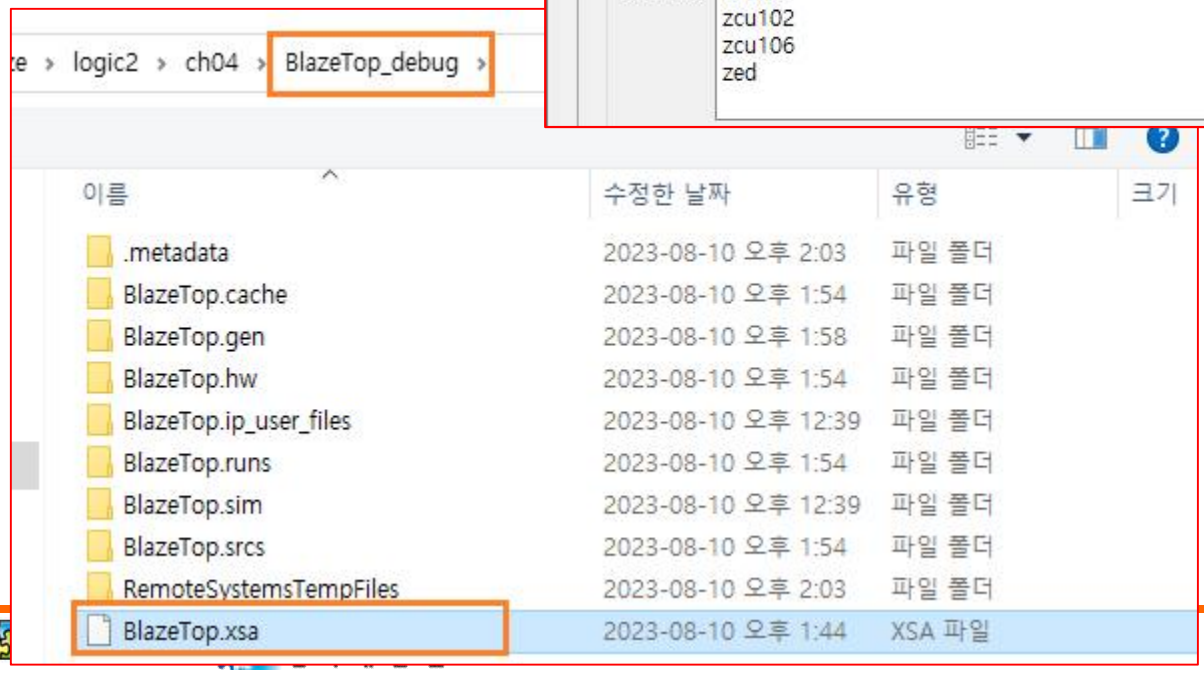
- *SW Implementation*
  - *Create a New Application Project* ➔ *Next* 클릭



LED_Counter Implementation

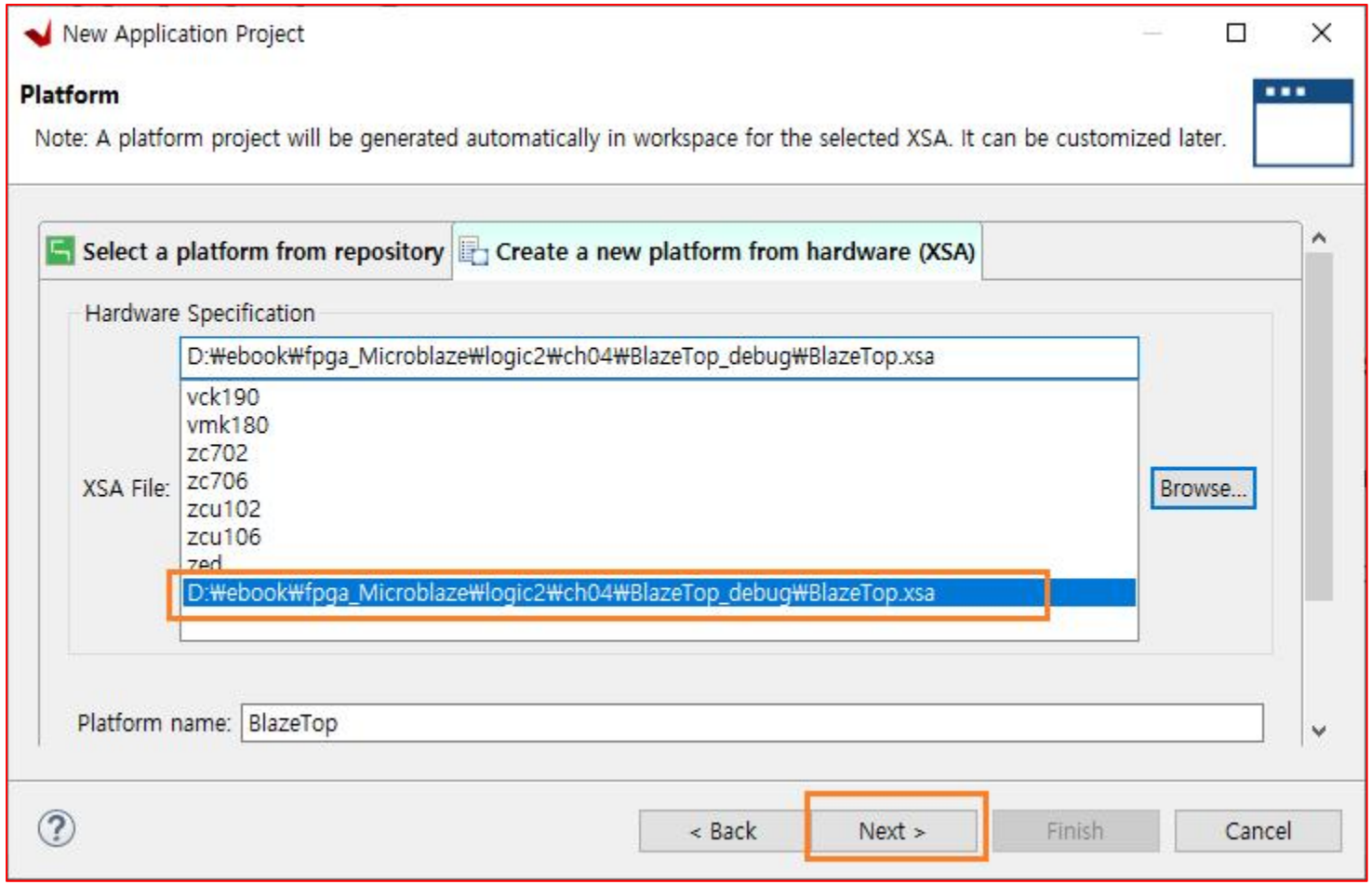
- ***SW Implementation***
  - ***Create a new platform from hardware (XSA)*** 탭 선택 ➔ ***Browe...*** 클릭 ➔ *Vivado*에서 생성한 ****.xsa*** 파일 ***Open***





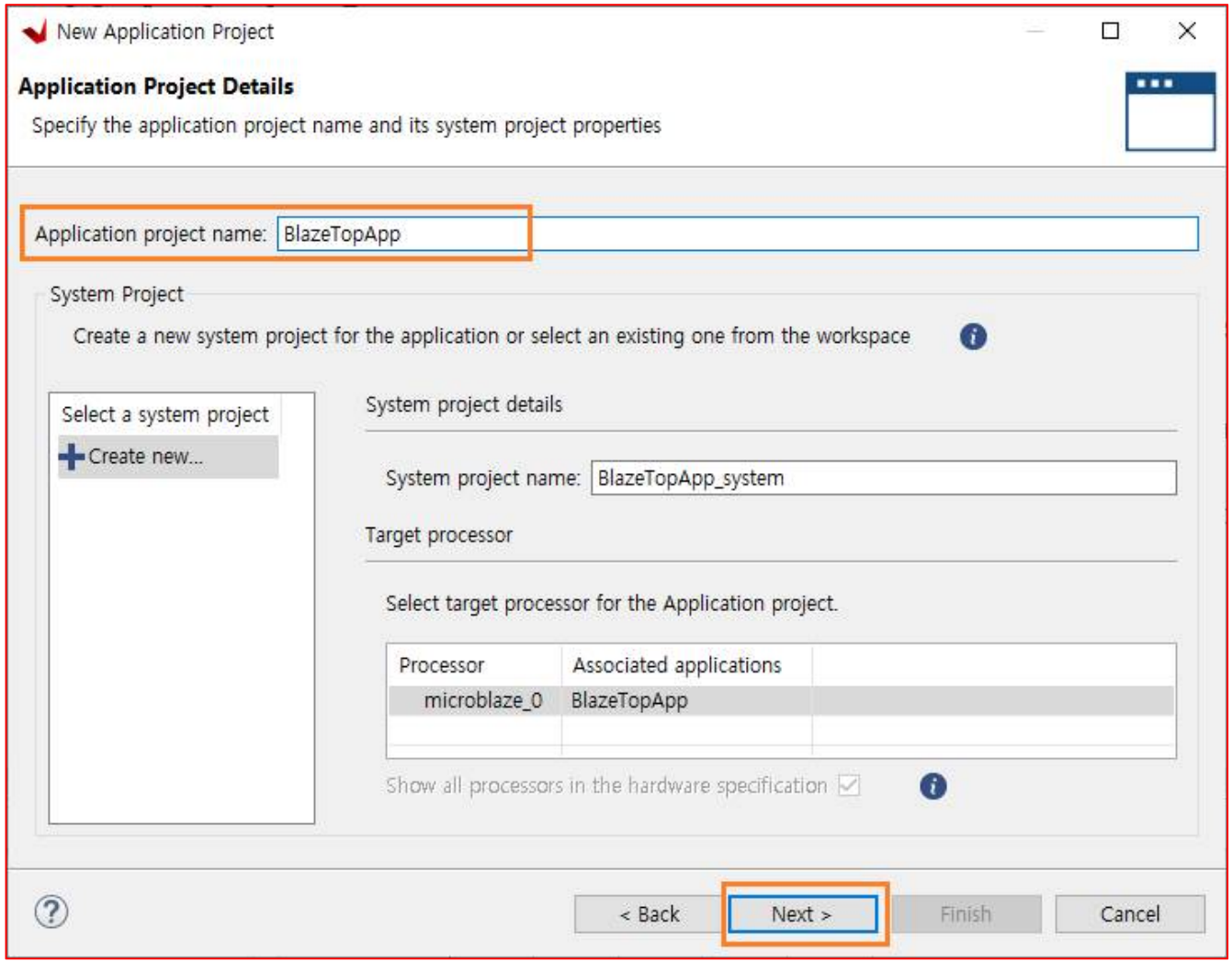
LED_Counter Implementation

- ***SW Implementation***
  - ***XSA*** 파일을 설정하면 **Next** 버튼이 활성화
  - ***Platform*** ➔ ***Create a new platform from hardware(XSA)*** ➔ ***Next***를 클릭



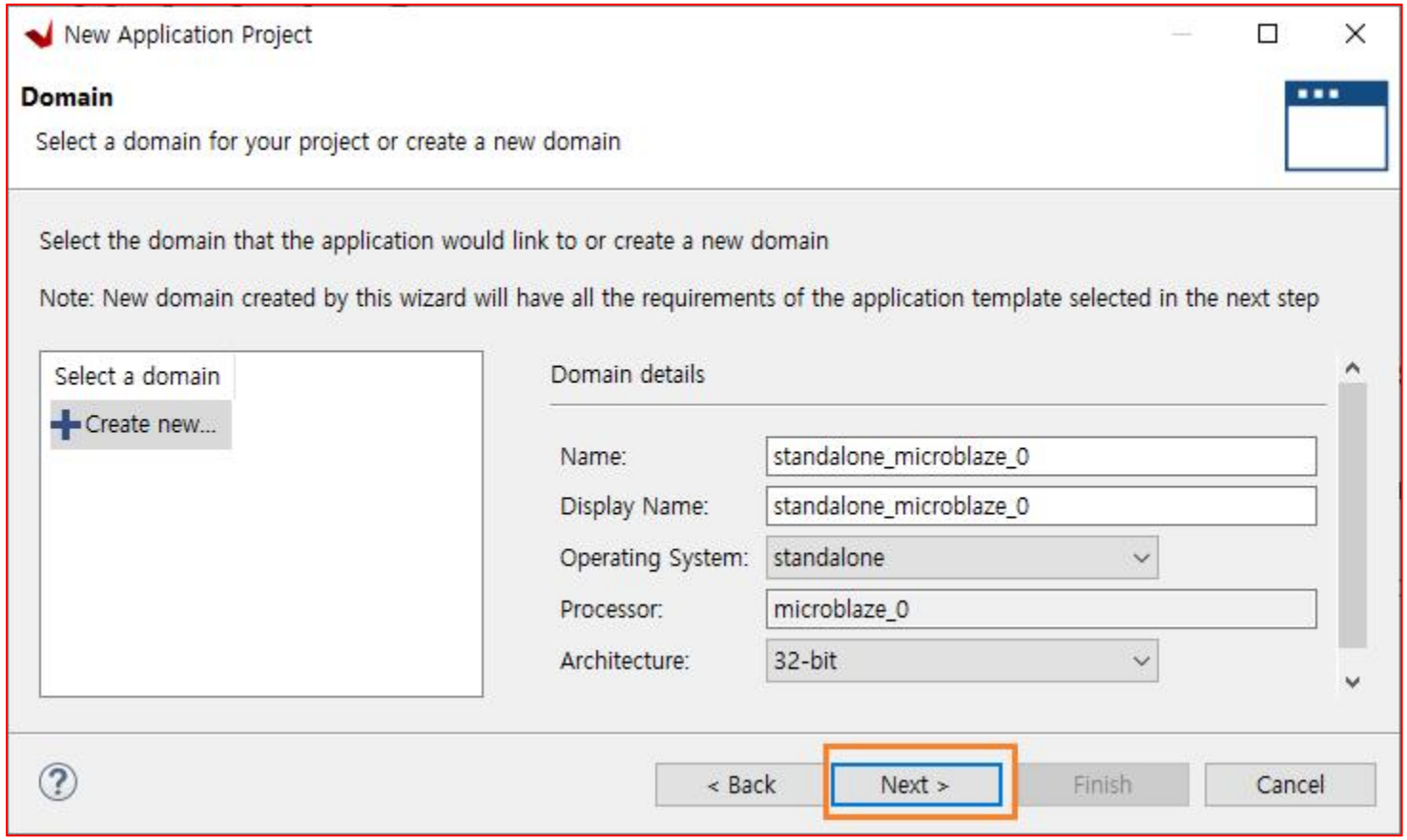
LED_Counter Implementation

- ***SW Implementation***
  - ***Application Project Details*** ➔ *Application project name (**BlazeTopApp**)*을 설정 ➔ ***Next***를 클릭



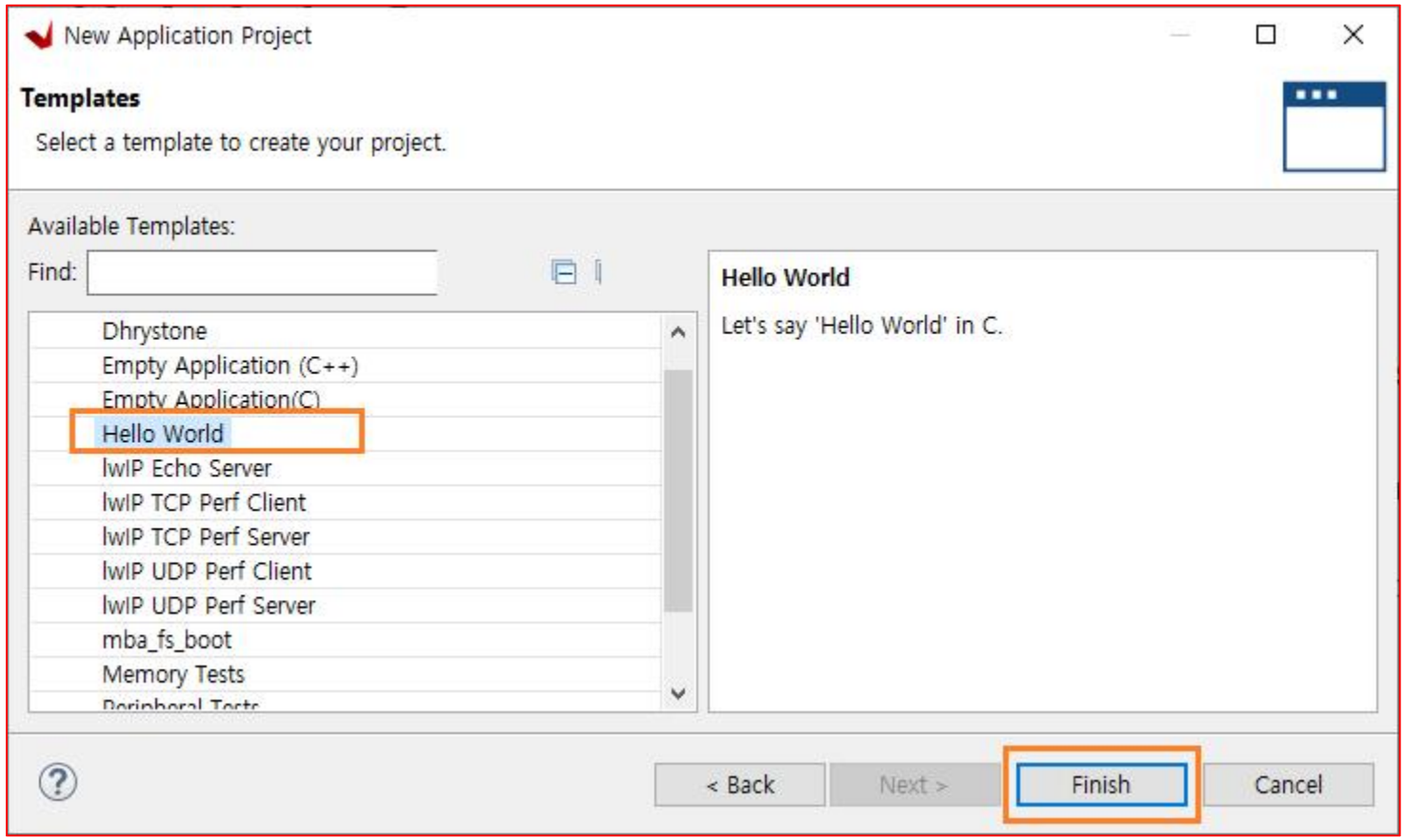
LED_Counter Implementation

- ***SW Implementation***
  - ***Domain ➔ [standalone_microblaze_0] ➔ Next*** 클릭



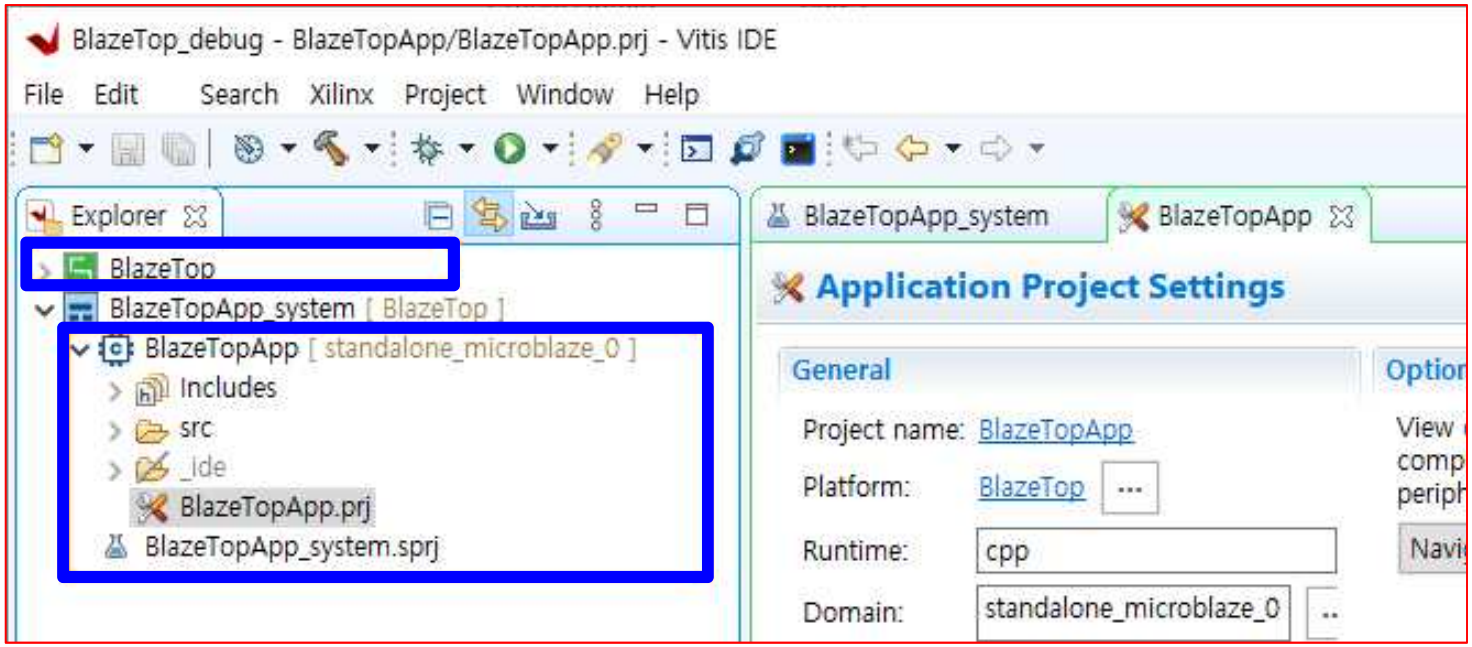
LED_Counter Implementation

- ***SW Implementation***
  - ***Templates*** 에서 ***'Hello World"***를 선택하고 ***Finish***를 클릭 ➔ 프로젝트 생성



LED_Counter Implementation

- ***SW Implementation***
  - ***Vitis*에는 2개의 *Project***
    - ✓ 하나는 **Vivado**에서 생성 후 **XSA** 파일로 변환한 프로젝트 파일
    - ✓ 나머지 하나는 **Vitis**에서 생성한 **SW**용 프로젝트
    - ✓ 즉, ***BlazeTop*이 *Vivado*에서 생성한 프로젝트, *BlazeTopApp*가 *Vitis*에서 생성한 프로젝트**



- ***Vivado*에서 *HW*관련된 내용들이 수정되어 xsa 파일이 수정되면, *BlazeTop*을 *Update* → *Re-Build*를 해야 함**
- ***SW*가 수정되면, *BlazeTopApp*를 *Re-Build* 해야 함**

LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- Create HDL Wrapper
- User Logic Implementation → Led_On/Off_Logic → led_control
- User Logic Implementation → Led_On/Off_Logic → BlazeTop
- XDC Implementation
- Generate Bitstream
- Export Hardware
- SW Implementation → Launch Vitis IDE & Create Application Project
- **SW Implementation → [helloworld.c] Code Implementation**

LED_Counter Implementation

- ***SW Implementation***
  - ***BlazeTopApp*** ➔ [***src***] 폴더 ➔ ***helloworld.c*** 의 내용 수정 ➔ 1초마다 메시지를 표시하는 간단한 프로그램

```c
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"


int main()
{
    init_platform();

    print("Hello World\n\r");
    print("Successfully ran Hello World application");
    cleanup_platform();
    return 0;
}
```

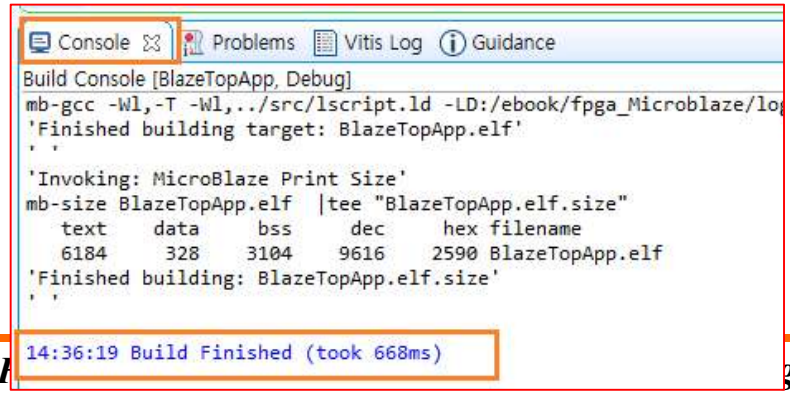
```c
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "sleep.h"

int main()
{
    int cnt = 0;

    init_platform();

    print("Hello World ..\r\n");
    while(1)
    {
        xil_printf("count : %d \r\n", cnt++);
        sleep(1);
    }
    cleanup_platform();
    return 0;
}
```

  - ***BlazeTopApp*** ➔ 우클릭 ➔ ***Build Project***를➔ ***Click*** ➔ ***Error*** 없이 ***Build***

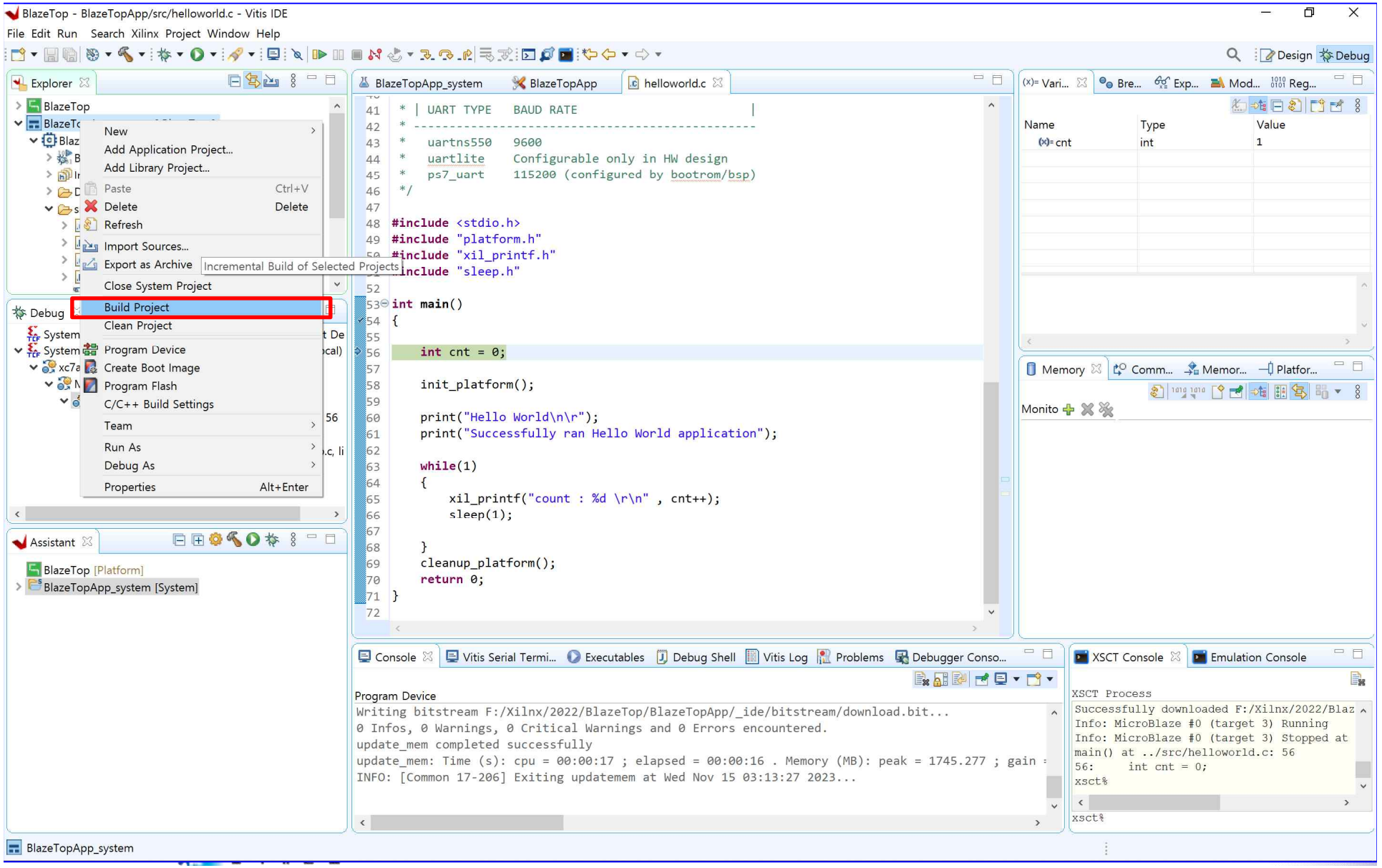


LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- Create HDL Wrapper
- User Logic Implementation → Led_On/Off_Logic → led_control
- User Logic Implementation → Led_On/Off_Logic → BlazeTop
- XDC Implementation
- Generate Bitstream
- Export Hardware
- SW Implementation → Launch Vitis IDE & Create Application Project
- SW Implementation → [helloworld.c] Code Implementation
- **SW Implementation → Build Project , Debug and Program Device**

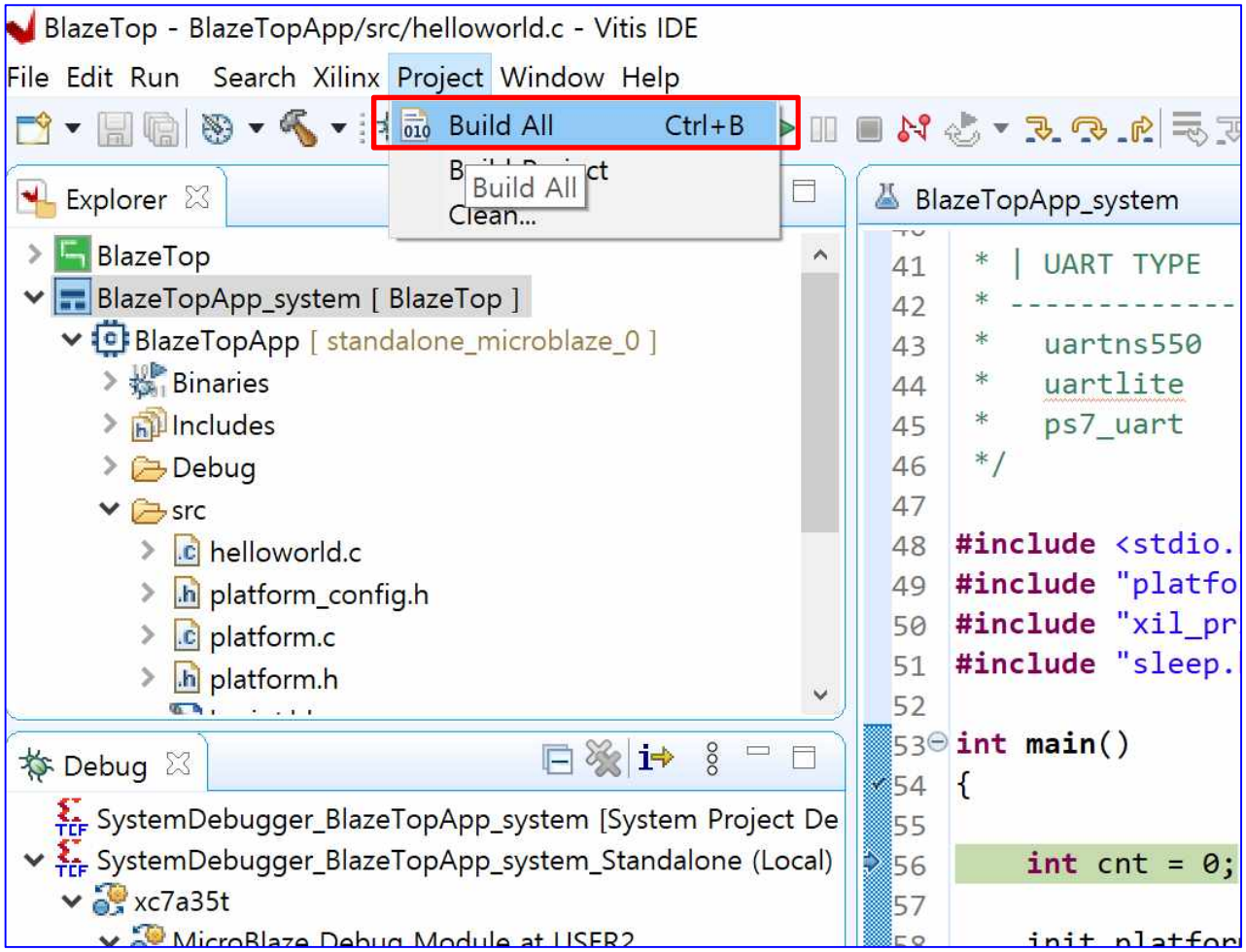
LED_Counter Implementation

- SW Implementation
  - Build Project or Build All(1/2)



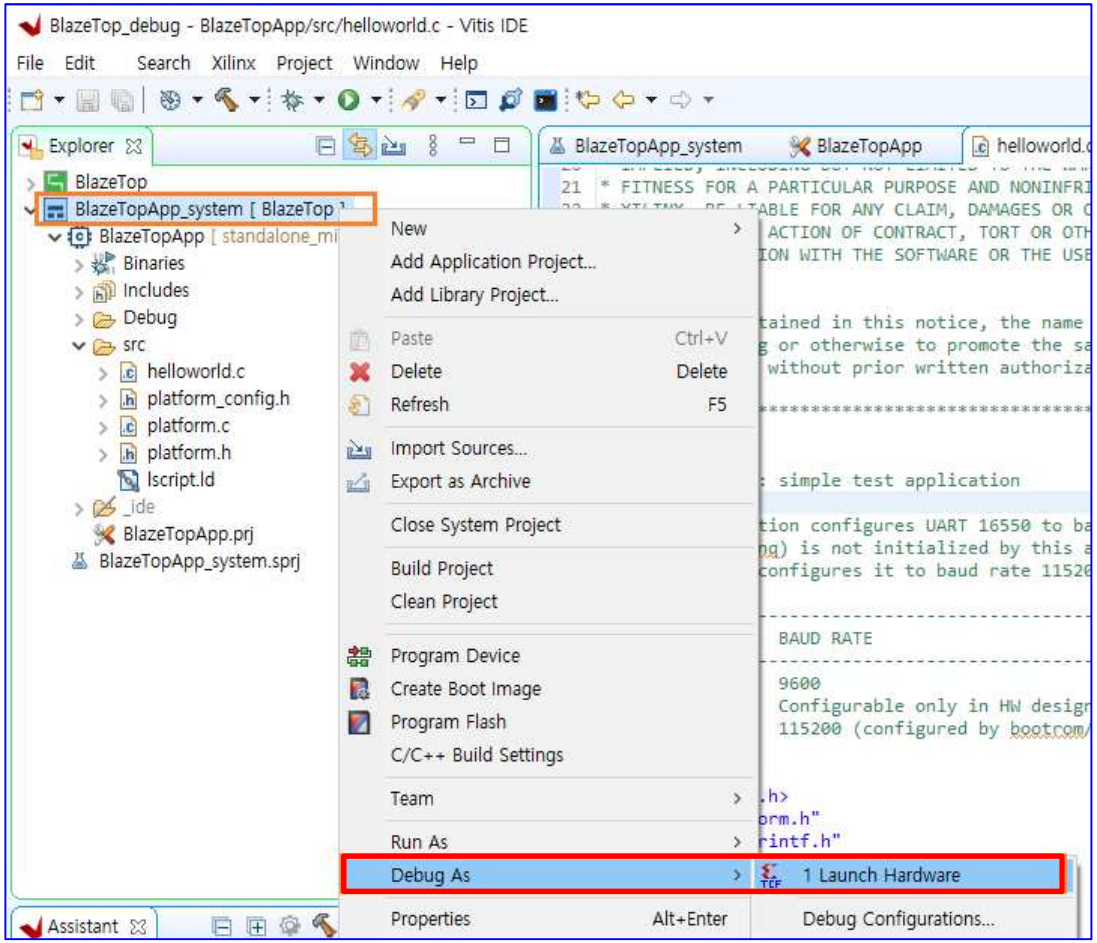
LED_Counter Implementation

- ***SW Implementation***
  - ***Build Project or Build All(2/2)***



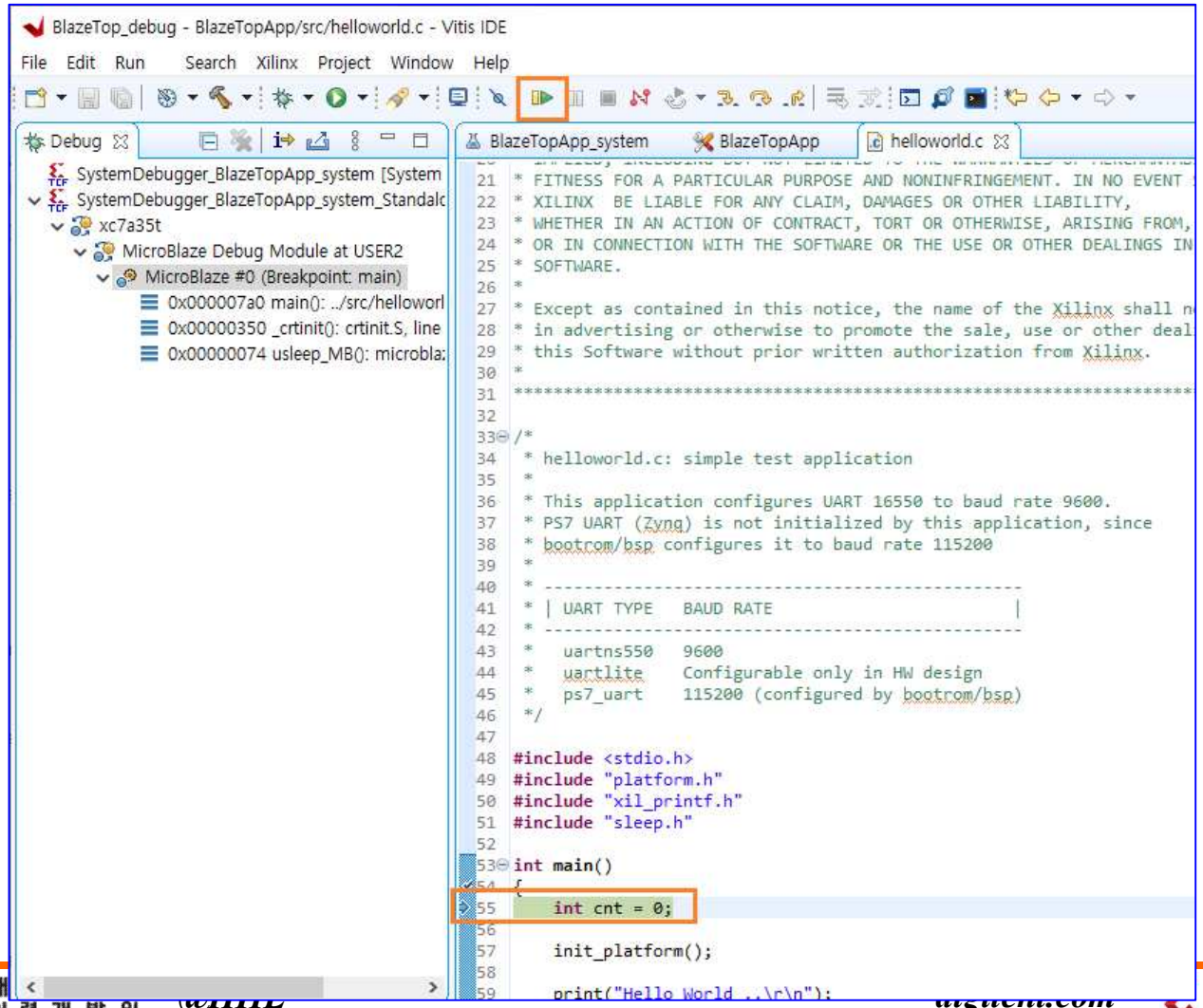
LED_Counter Implementation

- ***SW Implementation ➔ Program Download and Check Result***
  - 결과 확인하기 위하여 보드에 다운로드
  - ***Basys3*** 보드와 ***PC***를 ***USB***로 연결 ➔ 전원 인가 ➔ 보드가 인식
  - ***BlazeTopApp_system*** ➔우클릭 ➔ ***Debug As*** ➔ ***1 Launch Hardware*** 클릭



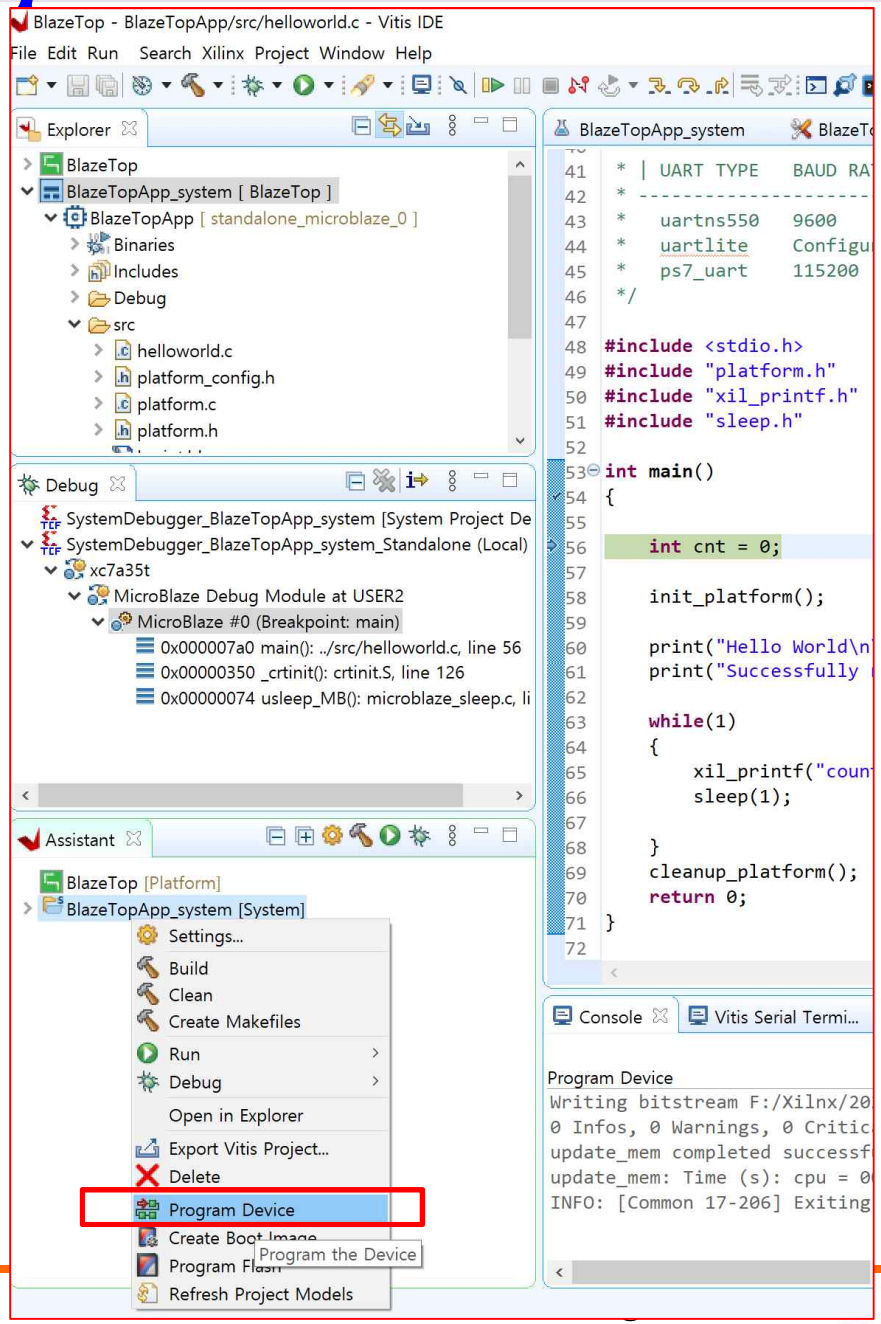
LED_Counter Implementation

- ***SW Implementation ➔ Program Download***
  - 프로그램이 다운로드 ➔ 우측 하단에 다운로드 과정이 %로 표시 ➔ 모두 다운로드가 되면 디버깅 모드가 되고 **main()**함수의 시작부분에 커서가 위치



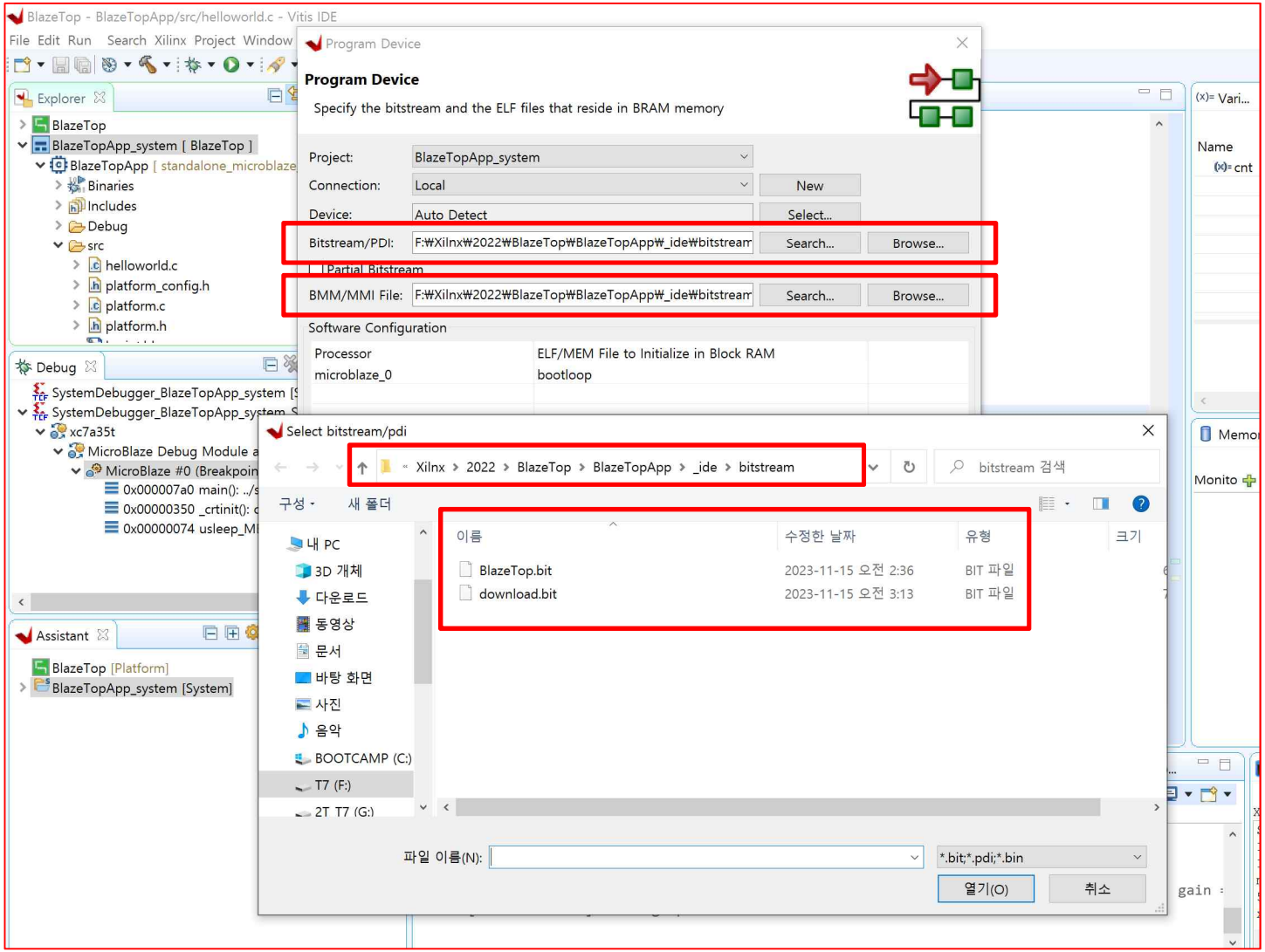
LED_Counter Implementation

- ***SW Implementation***
  - ***Program Device(1/4)***



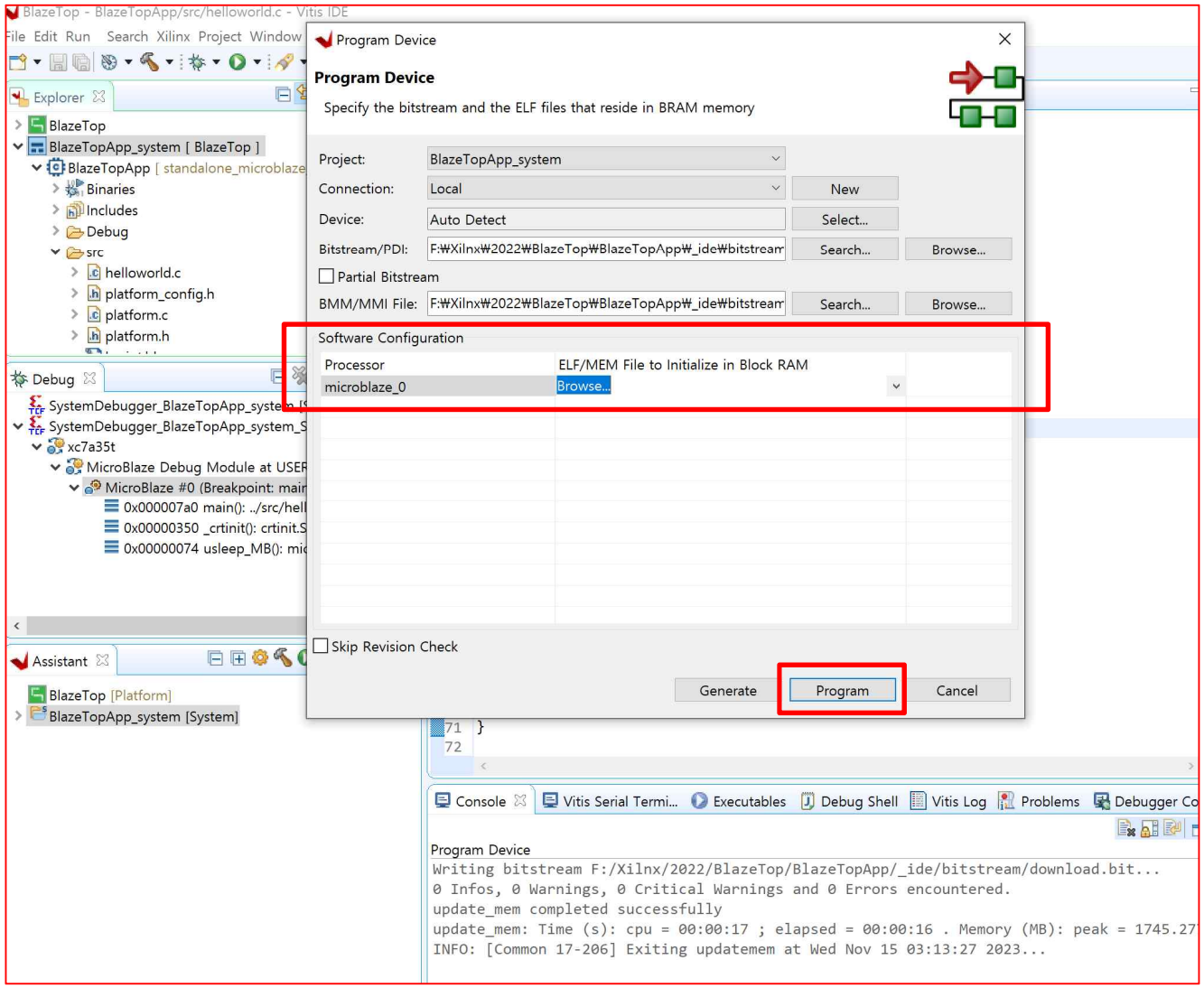
LED_Counter Implementation

- SW Implementation
  - Program Device(2/4)



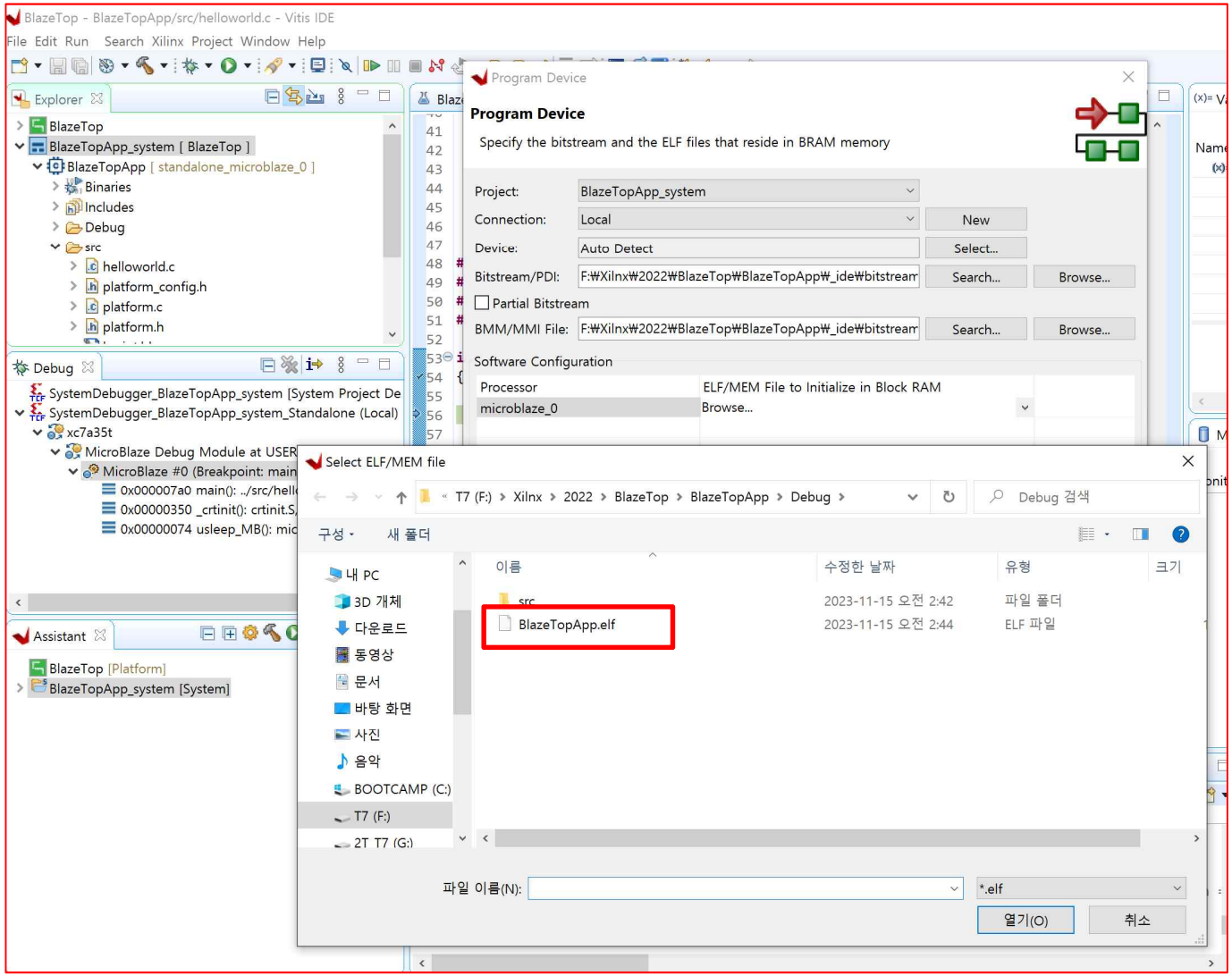
LED_Counter Implementation

- SW Implementation
  - Program Device(3/4)



LED_Counter Implementation

- ***SW Implementation***
  - ***Program Device(4/4)***

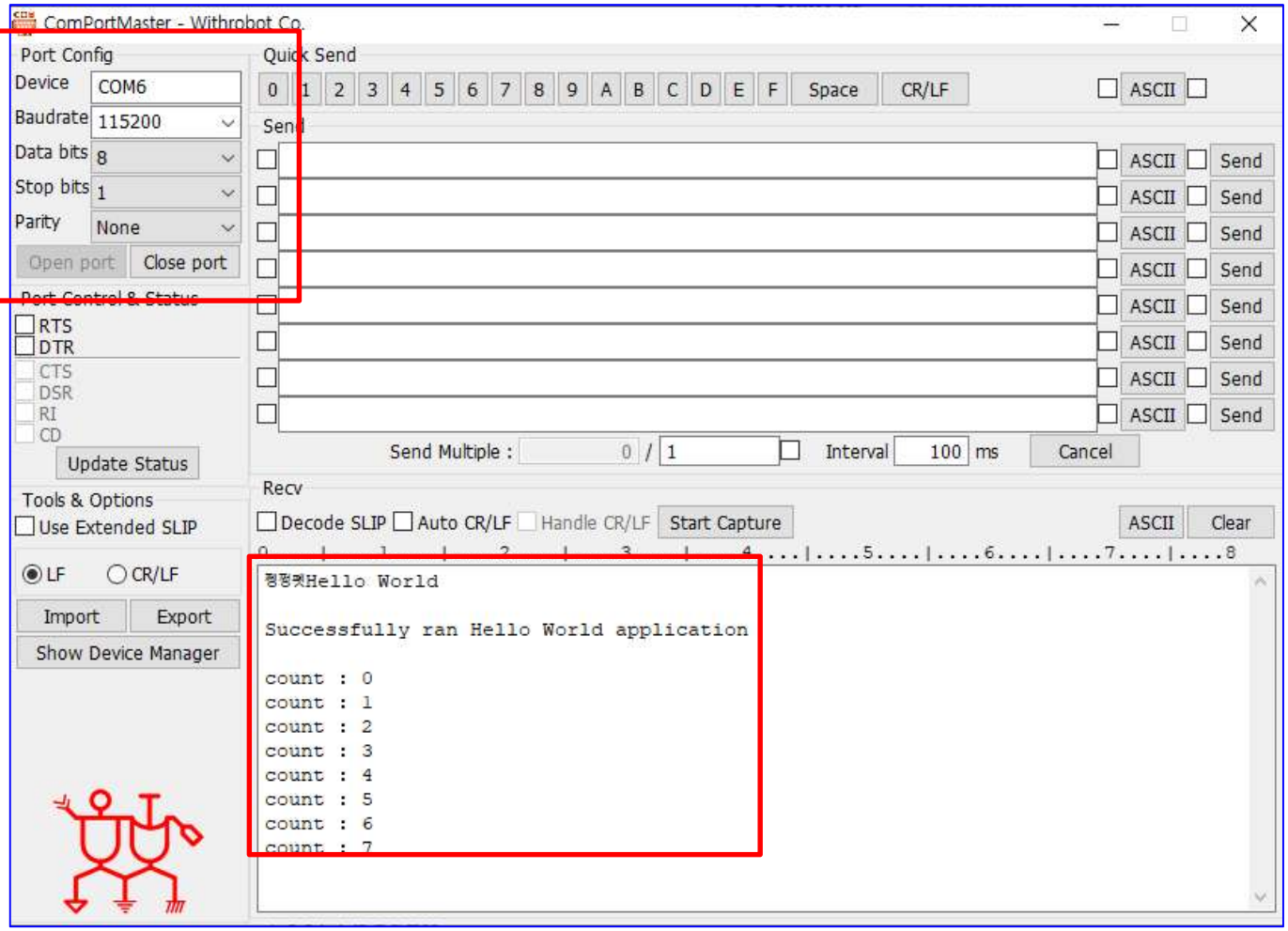


LED_Counter Implementation

- Create Project
- Create Block Design
- Add IP → MicroBlaze
- Add IP → AXI Uartlite
- Change Port Name and Interrupt Pin Connection
- Validate Design and Regenerate Layout
- Create HDL Wrapper
- User Logic Implementation → Led_On/Off_Logic → led_control
- User Logic Implementation → Led_On/Off_Logic → BlazeTop
- XDC Implementation
- Generate Bitstream
- Export Hardware
- SW Implementation → Launch Vitis IDE & Create Application Project
- SW Implementation → [helloworld.c] Code Implementation
- SW Implementation → Build Project , Debug and Program Device
- **Check The Result**

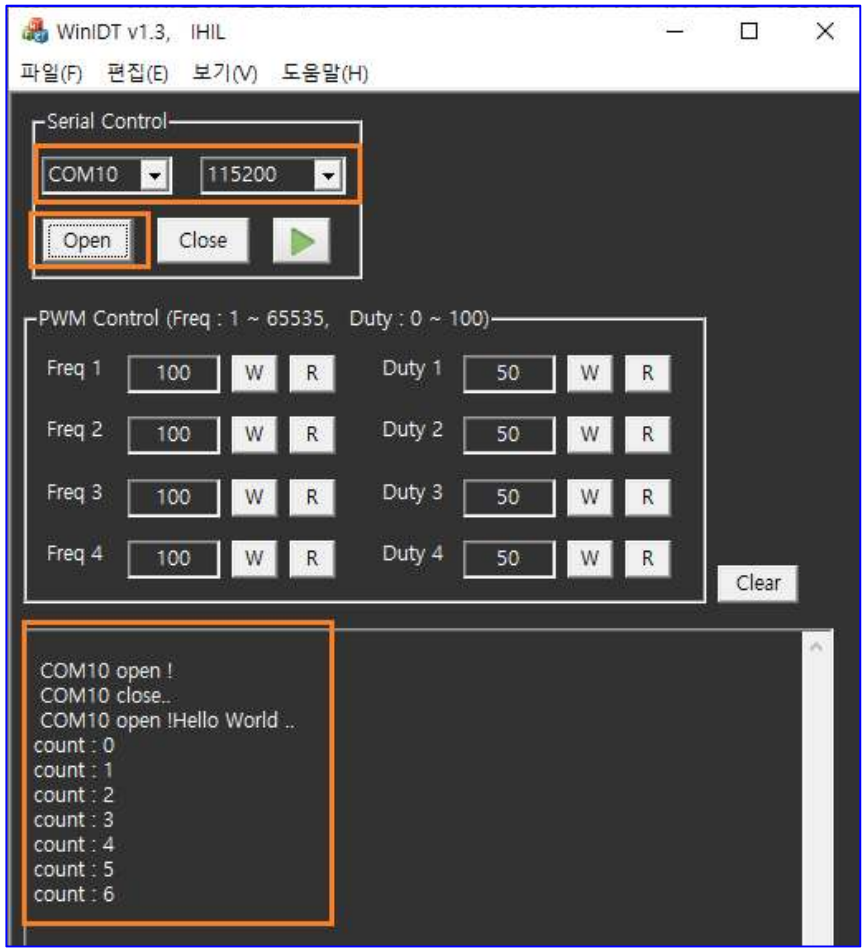
LED_Counter Implementation

- ***SW Implementation ➔ Check The Result***
  - **Resume 아이콘( )을 클릭하면 프로그램이 동작**
  - **터미널(ComPortMaster …) 프로그램을 실행하고 ComPortMaster를 Open하면 메시지가 1초마다 출력**



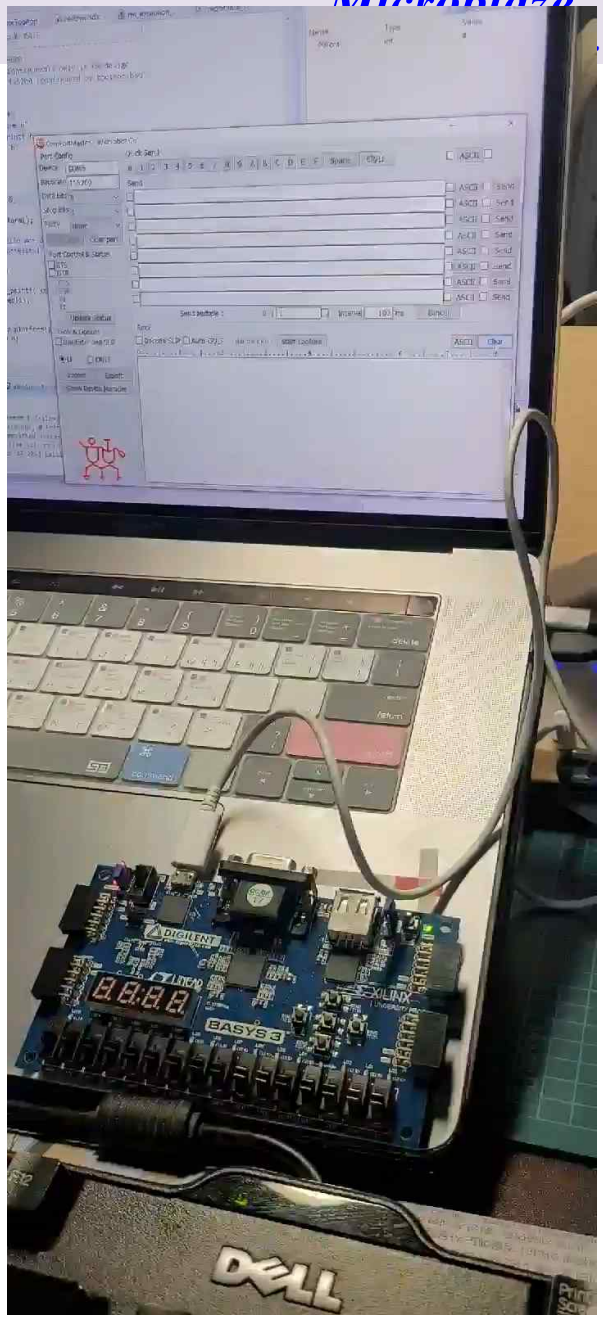
LED_Counter Implementation

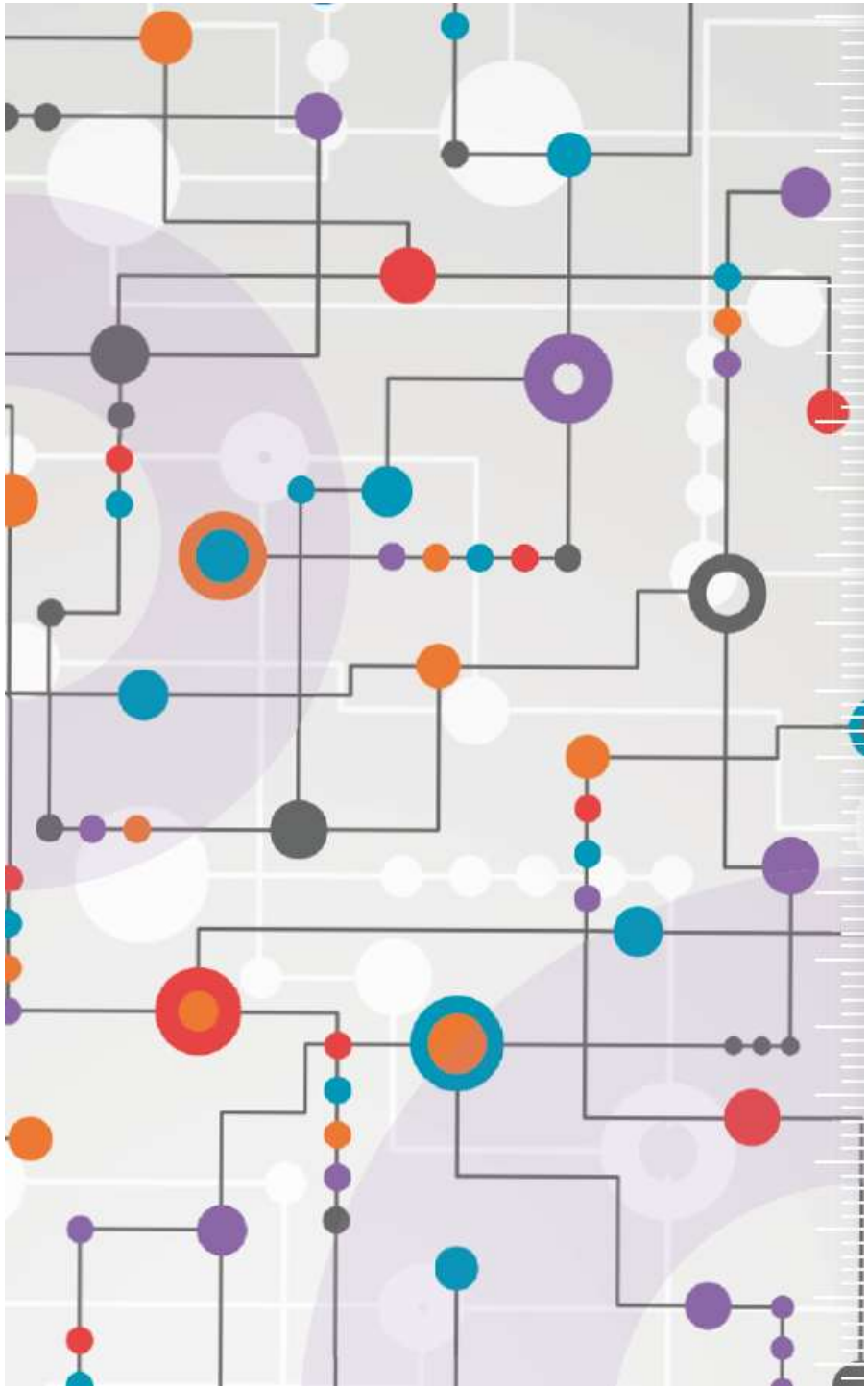
- *SW Implementation ➔ Check The Result*
  - **Resume 아이콘( )을 클릭하면 프로그램이 동작**
  - **WinIDT 프로그램을 실행하고 Comport를 Open하면 메시지가 1초마다 출력**



LED_Counter Implementation

- ***Check The Result***
  - ***LD0 ~ LD3*** 이 **0.5**초 간격으로 ***On/Off***
  - 기본적인 동작이 완료 ➔ ***led_counter*** 모듈에서 구현한 **LED on/off** 동작과 **SW**에서 구현한 **1**초마다 메시지를 표시하는 것이 완료





수고하셨습니다.